

NodeVet

DUOC UC SEDE PUENTE ALTO

Aranda Cárdenas, Catalina
Cruz Muñoz, Ricardo
Santander Leal, Sebastián

Tabla de contenidos

Tabla de contenidos	2
Introducción	4
Etapla 1: Planificación y Construcción	5
Fase 1.1: Inicio y Contexto del Proyecto	5
1. Acta de constitución	5
2. Marco Teórico	6
3. Estado del arte	7
4. Tabla de homologación	8
Fase 1.2: Definición de la Propuesta	9
1. Propuesta	9
2. Objetivo general y objetivos específicos	10
3. Supuestos y restricciones	10
4. Metodología de trabajo	11
5. Entregables	11
Fase 1.3: Ingeniería y Arquitectura	12
1. Tecnologías	12
2. Requisitos funcionales	13
3. Requisitos no funcionales	17
Fase 1.4: Gestión y Cierre de Planificación	39
1. Planificación y Cronograma de Trabajo	39
2. Gestión de Riesgos	39
Etapla 2: Cimientos del Software	40
Fase 2.1: Diseño y Documentación de la Solución	40
1. Arquitectura de Software y Tecnologías	40
2. Diagramas Relevantes	41
3. Metodología de Trabajo	62
Fase 2.2: Configuración del Ambiente de Pruebas y Procedimientos	62
1. Especificaciones del Entorno	62
2. Procedimientos de Respaldo y Restauración	63
3. Plan de Pruebas y Buenas Prácticas	63
Fase 2.3: Desarrollo de Software y Demostración	67
1. Funcionalidades Implementadas	67
2. Validaciones, Seguridad y Buenas Prácticas	86
3. Integración Front-Back	90
Fase 2.4: Métricas del proyecto	95
Etapla 3: Pruebas del software	96
Fase 3.1: Enfoque:	96
Fase 3.2: Plan de pruebas funcionales	96
1. Tipos de pruebas aplicadas	96
2. Estrategia de asignación	97
3. Cobertura de módulos	97
4. Criterios de aceptación	97
Fase 3.3: Herramientas y Software utilizados	98
Fase 3.4: Requisitos involucrados	99

Fase 3.5: Casos de pruebas	101
1. Backend	101
2. Frontend Móvil:	104
3. Frontend Web	108
Etapla 4: Cierre Final del proyecto	112
Fase 4.1: Estado final del producto:	112
Fase 4.2: Atributos de calidad consolidados:	114
Cierre y Conclusión	116
Conclusión etapa 1:	116
Conclusión etapa 2:	116
Conclusión etapa 3:	117
Conclusión final del proyecto:	118
Anexos	119

Introducción

La administración de centros de salud veterinaria se enfrenta constantemente a desafíos operativos que limitan su eficiencia y rentabilidad. En el modelo tradicional, es una práctica común la entrega de fichas clínicas físicas a los tutores, lo que frecuentemente resulta en la pérdida parcial o total de la información médica de los pacientes. A esto se suma una problemática crítica y recurrente en el rubro: el ausentismo en las consultas agendadas, lo que genera una pérdida significativa de tiempo y recursos económicos para las clínicas veterinarias.

Como respuesta a estas deficiencias, surge el proyecto "NodeVet", una plataforma digital integral compuesta por una aplicación web y una aplicación móvil nativa, orientada a optimizar la gestión clínica y administrativa. A diferencia de otras soluciones en el mercado que priorizan módulos contables o de inventario, NodeVet concentra su innovación inicial en mitigar el ausentismo y asegurar la rentabilidad del tiempo médico. Para lograrlo, implementa un innovador sistema de agendamiento que exige la integración de un pago externo para confirmar la cita de manera definitiva, apoyado por una regla de negocio que bloquea temporalmente el bloque horario por 10 minutos para evitar colisiones entre usuarios.

Además de modernizar el control de la agenda, la plataforma busca empoderar a los tutores al otorgarles acceso centralizado e inmediato al historial clínico digital de sus mascotas mediante sus dispositivos móviles. Por su parte, los médicos veterinarios contarán con un portal robusto diseñado para reducir la carga administrativa, permitiéndoles registrar evoluciones, diagnósticos, recetas y adjuntar archivos médicos importantes de forma eficiente.

Para materializar esta solución en un plazo estricto de 4 a 5 meses, el proyecto se ejecuta bajo la metodología ágil Scrum, lo que permite dividir el trabajo en ciclos iterativos (Sprints) y garantizar entregas de valor continuo. A nivel de ingeniería, NodeVet se construye sobre una arquitectura por capas escalable y segura, utilizando Java y Spring Boot para el desarrollo del backend mediante una API RESTful. La capa de presentación se desarrolla utilizando React para la versión web y React Native para el entorno móvil, alojando finalmente la infraestructura en Amazon Web Services (AWS) con una base de datos relacional MySQL.

Etapa 1: Planificación y Construcción

Fase 1.1: Inicio y Contexto del Proyecto

1. Acta de constitución

El presente documento constituye el acta formal de inicio del proyecto “NodeVet”, cuyo desarrollo comienza el 10/03/2026. Este proyecto surge con el propósito de crear una plataforma digital que facilite el registro clínico de pacientes animales y optimice el agendamiento de horas médicas. Además, busca resolver el problema de ausentismo en las consultas proponiendo métodos efectivos para evitar que los clientes falten a citas previamente agendadas.

Stakeholders

Los stakeholders principales para este proyecto son:

- **Stakeholders Internos (Equipo de desarrollo):**
 - Aranda Cárdenas, Catalina Paz.
 - Cruz Muñoz, Ricardo Ariel.
 - Santander Leal, Sebastián Ignacio.
- **Stakeholders externos (Usuarios finales):**
 - Médicos Veterinarios
 - Tutores de Mascotas

Lineamientos Generales y Objetivos

El objetivo central del proyecto es desplegar una aplicación web y una aplicación móvil en un plazo estricto de 4 meses. Esto con la finalidad de agilizar el uso de las horas médicas y otorgar a los tutores la posibilidad de controlar la salud de sus mascotas mediante un historial clínico accesible. Para lograr esto, se implementará la regla de negocio donde el usuario podrá bloquear temporalmente un horario (10 minutos), registrándose la cita en el sistema de manera definitiva solo una vez que el pago externo haya sido realizado.

Alcance Inicial:

El alcance del proyecto contempla el desarrollo de los siguientes módulos funcionales:

- **Gestión de Usuarios:** Sistema de registro y autenticación basado en roles (tutor y médico), cada uno con vistas personalizadas.

- **Perfil de Cliente y Gestión de Mascotas:** Capacidad para que el tutor agregue múltiples mascotas con su información básica (nombre, especie, raza, sexo, edad y peso). Los tutores podrán visualizar el historial clínico de sus mascotas, así como sus citas agendadas y pasadas.
- **Agendamiento de Horas y Pagos:** Calendario interactivo con bloques horarios disponibles. Permite selección de médico de preferencia e integración con un sistema de pagos externos.
- **Portal Médico Veterinario:** Panel de control con calendario de citas (diario, semanal, mensual). Acceso a los detalles del paciente con capacidad de editar el historial clínico, añadiendo notas, diagnósticos, recetas, vacunas y archivos adjuntos como radiografías o exámenes.

2. Marco Teórico

El desarrollo e implementación de la plataforma "NodeVet" se sustenta en la evolución y estandarización de los sistemas de información aplicados a la medicina veterinaria. Este marco teórico aborda los ejes centrales del proyecto: la digitalización de historiales médicos, la optimización de las agendas y el impacto de las tecnologías web en la administración clínica.

- **Sistema de Información y la Ficha Clínica Electrónica**

La transición hacia el registro digital centralizado responde a deficiencias operativas crónicas en el modelo tradicional. En los sistemas convencionales, es una práctica común que las fichas físicas sean entregadas a los dueños de las mascotas, quienes frecuentemente las pierden parcial o totalmente. Según [Padilla Espino \(2021\)](#), esta situación genera procesos ineficientes cuyas causas deben ser eliminadas de manera prioritaria.

Para resolver esta fragmentación de datos, [Uriol Paredes \(2016\)](#) respalda el enfoque de un desarrollo dual (mediante aplicaciones web y móviles), justificando que centralizar el acceso a la información clínica permite agilizar de manera significativa los diagnósticos. Adicionalmente, se destaca la necesidad de empoderar al tutor de la mascota; la investigación de [Semisud \(2017\)](#) indica que un sistema eficiente debe permitir a los dueños tener a su alcance el expediente médico completo. Esto facilita que los tutores observen el registro de citas, vacunas y la ficha integral de sus mascotas, siendo una herramienta de alto valor en caso de que necesiten cambiar de veterinario.

- **Gestión de Agendamiento y Control de Citas**

El correcto flujo de atención es un pilar fundamental en los centros de salud animal. [Chávez Yagual \(2022\)](#) argumenta que la implementación de aplicaciones web enfocadas en el agendamiento busca mejorar directamente la administración de los procesos y el control del servicio al cliente. En su investigación, sostiene que el uso de un calendario interactivo es clave para eliminar los cuellos de botella que suelen producirse en el área de recepción. En

concordancia, [Llor García \(2021\)](#) señala que la automatización de la agenda médica garantiza un rendimiento eficiente, evitando el choque de horarios y mejorando, en última instancia, la atención brindada a las mascotas.

- **Impacto Administrativo y Tecnológico**

La incorporación de plataformas modernas trasciende la labor netamente clínica y optimiza la rentabilidad del recinto. [Silva \(2021\)](#) fundamenta que la adopción de aplicaciones web en el entorno veterinario logra reducir la carga administrativa que recae sobre el personal. Además, concluye que el uso de estas herramientas genera reportes críticos que permiten analizar información, detectar fallas operativas y ejercer un mejor control. Finalmente, este nivel de gestión integral permite asegurar la información del recinto, brindar un servicio más calificado y fortalecer la fidelización de los clientes.

En cuanto a la construcción de este tipo de plataformas, [Córdova Urriola \(2018\)](#) detalla en su análisis de sistemas veterinarios que la aplicación de metodologías ágiles, como Scrum, resulta idónea para guiar la estructuración y desarrollo integral tanto de la capa del backend como del frontend.

3. Estado del arte

En la actualidad, la administración de clínicas veterinarias cuenta con diversas herramientas digitales orientadas a la automatización de procesos administrativos y clínicos. Al analizar el mercado actual, se identifican plataformas que ofrecen soluciones integrales, de las cuales destacan las siguientes:

- **Qvet:** Es un sistema enfocado en la gestión de horas y el control de productos, permitiendo el manejo de stock, proveedores y seguimiento de ingresos. Cuenta con una aplicación orientada al cliente para el envío de recordatorios de citas, aunque se reporta que actualmente se encuentra desactualizada.
- **Ponkis:** Se presenta como una solución basada en la nube (sin instalación de escritorio) que destaca por su integración nativa con WhatsApp, utilizada para la gestión de citas y notificaciones. Ofrece un control de inventario automatizado basado en máximos y mínimos, sistema de caja (POS), y funcionalidades administrativas avanzadas como consentimientos automatizados con firma digital y una página web prediseñada con portal para pacientes.
- **Okvet:** Plataforma que ofrece una amplia variedad de módulos específicos para el rubro, abarcando desde historias clínicas, vacunación, imágenes diagnósticas y exámenes de laboratorio, hasta servicios de peluquería y spa. Incorpora herramientas de CRM para clientes, perfiles de usuario y una agenda dual (personal y general) que soporta notificaciones y recordatorios.
- **DodoZoof:** Es un software veterinario robusto que digitaliza y centraliza los procesos clínicos y administrativos. A nivel médico, permite gestionar historias clínicas digitales,

hospitalizaciones, interconsultas y la elaboración de la epicrisis para el alta médica. A nivel administrativo, incluye una multiagenda, documentos inteligentes personalizables (recetas, pagarés), facturación electrónica, y un punto de venta integrado que actualiza automáticamente el stock y genera reportes financieros.

4. Tabla de homologación

A partir del análisis del estado del arte, se presenta a continuación una matriz de homologación. Esta tabla contrasta las funcionalidades clave de las soluciones comerciales existentes (Qvet, Ponkis, Okvet, DodoZooft) frente a la propuesta de valor del proyecto NodeVet. El objetivo es evidenciar las ventajas competitivas de la solución propuesta, destacando especialmente las brechas de mercado que busca cubrir en relación al compromiso del cliente y la reducción del ausentismo.

Funcionalidad/ Módulo	NodeVet	QVet	Ponkis	Okvet	DodoZooft
Agendamiento de horas médicas	Si	Si	Si	Si	Si (Multiagenda)
Historial clínico digital	Si	No especificado	Si	Si	Si
App Móvil nativa para dueños (Tutores)	Si	Si (Desactualizada)	No (Usa WhatsApp)	No	No
Bloqueo temporal de agenda (Prevención choques)	Si	No	No	No	No
Integración de pago previo para confirmar cita	Si	No	No	No	No
Roles de usuario personalizados	Si (Tutor, Veterinario y Admin)	Si	Si	Si	Si
Gestión de Inventario y Punto de Venta (POS)	No	Si	Si	No especificado	Si
Notificaciones	Si	Si	Si (Vía WhatsApp)	Si	Si (Masivas)

Análisis de la Homologación:

Como se puede observar en la tabla, si bien las soluciones del mercado son robustas en términos de gestión administrativa integral (como control de stock, inventario y punto de venta), presentan un enfoque tradicional frente al agendamiento. NodeVet se diferencia estratégicamente al no intentar competir inicialmente en módulos contables, sino que concentra su innovación en resolver un dolor crítico de las clínicas: el ausentismo. Al implementar un flujo que requiere la integración de pagos externos para asegurar la cita (y un bloqueo temporal de 10 minutos), NodeVet asegura la rentabilidad del tiempo médico. Además, la inclusión de una aplicación móvil nativa para Android otorga a los tutores una herramienta moderna y accesible para el control de sus mascotas, superando a plataformas que solo dependen de correos o mensajes de WhatsApp.

Fase 1.2: Definición de la Propuesta

1. Propuesta

La propuesta del equipo es desarrollar una plataforma digital llamada “NodeVet”, la cual se compone de una aplicación web y una aplicación móvil, orientada a optimizar el registro clínico de pacientes animales y la administración de horas médicas online.

“NodeVet” se diferencia de las soluciones actuales del mercado al no enfocarse inicialmente en control de stock, sino en resolver un problema crítico para las clínicas veterinarias: el ausentismo (el hecho de que las personas reserven una hora online y al momento de la cita no aparezcan, generando pérdida de tiempo y dinero para la veterinaria). Para lograr esto, se propone integrar un flujo de agendamiento con el sistema de pagos Flow, el cual es de carácter obligatorio para confirmar la cita (las veterinarias tendrán la opción de activar o no esta función). Esto se complementa con una regla de negocio que bloquea temporalmente el bloque horario por un rango de 10 minutos, evitando las colisiones entre usuarios y asegurando la rentabilidad del veterinario.

Además, esta propuesta busca mantener un seguimiento más activo y consciente de los tutores de las mascotas y eliminar las deficiencias del modelo tradicional donde las fichas físicas suelen perderse. A partir de la aplicación móvil y/o web, los tutores podrán registrar a sus mascotas y tener acceso centralizado a su historial clínico digital, permitiéndoles visualizar más fácilmente el registro de vacunas, diagnósticos, recetas y citas.

Por otro lado, en el caso de los médicos veterinarios, se propone un portal robusto que busca reducir la carga administrativa. Este módulo les permitirá gestionar su agenda de disponibilidad de manera eficiente, registrar detalladamente la evolución médica de cada paciente en la ficha clínica electrónica, y además poder adjuntar archivos médicos importantes, tales como exámenes diversos y/o radiografías.

En términos técnicos, la propuesta contempla principalmente una arquitectura por capas desarrollando el backend con Java y Spring Boot, consumida a través de una API RESTful. Esto garantiza que el sistema sea escalable, seguro y con tiempos de respuesta óptimos bajo alta concurrencia.

2. Objetivo general y objetivos específicos

Objetivo General:

Desarrollar e implementar una plataforma digital (aplicación web y móvil) en un plazo de 4 meses aproximadamente, orientada a optimizar el registro clínico y agilizar el uso de horas médicas. Esta solución busca resolver el problema del ausentismo mediante un sistema de agendamiento con bloqueo temporal y pago obligatorio previo, al mismo tiempo que le da mayor facilidad a los tutores al acceso centralizado al historial de salud de sus mascotas.

Objetivos específicos:

- Desarrollar un sistema interactivo de agendamiento de horas que permita el bloqueo temporal de bloques horarios.
- Integrar un sistema de pagos externos en el flujo de reserva para confirmar definitivamente las citas y asegurar la rentabilidad del tiempo médico.
- Implementar un módulo de Ficha Clínica Electrónica para que los médicos veterinarios puedan registrar: evolución del paciente, diagnósticos, recetas, vacunas y subir archivos adjuntos.
- Construir una aplicación móvil nativa (para Android e iOS) orientada a los tutores, permitiéndoles gestionar perfiles de múltiples mascotas y visualizar su historial médico
- Desplegar el backend del sistema utilizando Java con Spring Boot bajo una arquitectura de API RESTful, implementando JWT para la autenticación segura de usuarios.

3. Supuestos y restricciones

Supuestos:

- Los usuarios cuentan con un dispositivo móvil inteligente compatible con sistemas operativos iOS 14 o superior, y Android 10 o superior, para ejecutar la aplicación.
- El proveedor de la plataforma de pagos externos (Flow) mantiene sus servicios operativos y devuelve respuestas estructuradas en formato JSON.
- El personal médico está dispuesto a adoptar un sistema digital para reemplazar la entrega de fichas físicas a los dueños de mascotas

Restricciones:

- El proyecto debe ser desarrollado y desplegado en un plazo máximo de 4 a 5 meses.
- El sistema no competirá inicialmente con módulos contables complejos; funciones como la gestión de inventario y el punto de venta (POS) quedan fuera del alcance inicial.
- El sistema debe garantizar un rendimiento óptimo, cargando el historial clínico completo en un máximo de 2 segundos, incluso con 500 usuarios concurrentes.
- La arquitectura debe asegurar una alta disponibilidad, limitando el tiempo total de caída o inaccesibilidad a un máximo de 9 horas al año (aproximadamente 43 minutos al mes).
- Todas las comunicaciones hacia proveedores externos deben viajar obligatoriamente por el protocolo HTTPS. Además, las contraseñas de los usuarios deben transformarse utilizando algoritmos de hash robustos antes de ser almacenadas en la base de datos.
- Los archivos adjuntos médicos, como radiografías o exámenes en PDF, deben guardarse en un servicio en la nube dedicado (AWS).

4. Metodología de trabajo

Para la gestión del proyecto de “NodeVet”, se adoptará la metodología ágil Scrum. El porqué de esta decisión se fundamenta en la necesidad de desarrollar e implementar la plataforma en un tiempo limitado de 4 meses. Scrum nos permite dividir el proyecto en ciclos iterativos e incrementales (Sprints), garantizando entregas de valor continuo, mitigando riesgos de forma temprana y otorgando flexibilidad ante posibles cambios en los requerimientos. Además, tal como se menciona en el marco teórico del proyecto, la aplicación de metodologías ágiles resulta idónea para guiar la estructuración y el desarrollo, tanto del backend, como del frontend en la construcción de sistemas veterinarios.

5. Entregables

I. Etapa de inicio y planificación:

- Product Backlog:
 - Traduce todos los requisitos funcionales y no funcionales en Historias de Usuario priorizadas por su valor de negocio
- Definición de la Arquitectura Base:
 - Esquema formal de la arquitectura en capas (N-Tier), modelo relacional de la base de datos y diseño inicial de la infraestructura en AWS.
- Prototipos de interfaz (UI/UX):
 - Diseños y/o mockups de las pantallas principales para la versión web y la aplicación móvil, validados previo a la fase de programación

II. Etapa de Desarrollo Iterativo (Sprints)

- Entregable Sprint 1 (Seguridad y Perfiles):

- Módulo de registro e inicio de sesión operativo utilizando JWT , junto con la creación de los perfiles base para Tutores de Mascotas y Médicos Veterinarios.
- Entregable Sprint 2 (Mascotas y Calendarios):
 - Panel habilitado para que los tutores registren múltiples mascotas, acompañado del despliegue del calendario interactivo utilizando el componente FullCalendar.
- Entregable Sprint 3 (Flujo de Agendamiento):
 - Sistema de reservas operativo con la regla de negocio que bloquea temporalmente el horario por 10 minutos , logrando la integración exitosa con el sistema de pagos externos.
- Entregable Sprint 4 (Portal Médico):
 - Módulo de Ficha Clínica Electrónica para registrar evoluciones y vacunas y realizar seguimiento de las citas agendadas.

III. Etapa de Despliegue y Cierre (Release)

- Aplicaciones y Servidor Empaquetado:
 - Entrega del backend compilado en un archivo ejecutable .jar , junto con la aplicación web (React) optimizada y la aplicación móvil nativa (React Native y Expo) lista para dispositivos móviles.
- Documentación Interactiva de la API:
 - Archivo de especificación (OpenAPI) integrado visualmente mediante herramientas como Swagger UI, exponiendo una ruta web para documentar y probar todos los endpoints del sistema.
- Repositorio y Calidad de Código:
 - Entrega del código fuente versionado.

Fase 1.3: Ingeniería y Arquitectura

1. Tecnologías

Para el desarrollo de la aplicación web se ha decidido utilizar el modelo de arquitectura por capas (N-Tier Architecture), donde la capa de aplicación expondrá sus funcionalidades hacia la capa de presentación a través de una interfaz de servicios basada en REST (API RESTful). Esto se realiza con el propósito de distribuir y separar las responsabilidades del sistema, asegurando una comunicación estandarizada para el desarrollo de una solución escalable.

La capa de aplicación será desarrollada en Java (JDK 21). Se eligió este lenguaje debido a que es robusto, completo y se adapta de manera óptima a las necesidades del sistema. Junto a este, se utilizará Spring Boot como framework gracias a la gran cantidad de herramientas y librerías que ofrece. Una de las librerías fundamentales en este proyecto será Spring Security, la cual proporciona un nivel de protección acorde a los estándares actuales de la industria mediante sus funciones de autenticación y autorización de usuarios.

En la capa de presentación —es decir, la interfaz de usuario— se empleará JavaScript/TypeScript utilizando React, mientras que para la presentación móvil se usará React Native y Expo. Se eligió este stack tecnológico por su simplicidad, escalabilidad y por su capacidad de migración adaptable a entornos móviles. Gracias a que el frontend será desarrollado en React, la transición a un entorno móvil será ágil, permitiendo reutilizar el código con cambios mínimos en su estructura. Junto a este framework se hará uso de JSON Web Tokens (JWT) para añadir una capa adicional de seguridad, protegiendo las rutas de la web y limitando el acceso a información considerada delicada. Por último, el sistema de agendamiento de horas se implementó como un componente de calendario propio, construido a medida mediante el uso de arreglos para meses y días en TypeScript, sin depender de librerías de terceros. Este componente despliega una vista mensual que permite la selección del día, seguido de un acordeón interactivo que muestra los horarios disponibles por cada profesional.

Finalmente, la aplicación web se alojará y desplegará en una máquina virtual instanciada en la nube de Amazon Web Services (AWS). Para la base de datos se utilizará el motor MySQL, garantizando el almacenamiento relacional, estructurado y persistente de toda la información generada en la plataforma.

2. Requisitos funcionales

Código RF	RF-01	Código RF	RF-02
Nombre RF	Crear perfil tutor	Nombre RF	Modificar Perfil
Tipo de Requerimiento	Funcional Usuario	Tipo de Requerimiento	Funcional Usuario
Actores Relacionados	Tutor de Mascota	Actores Relacionados	Tutor de Mascota / Veterinario
Descripción	Apartado de Registro para Tutores.	Descripción	Opción para modificar su perfil.
Criterios de Aceptación	-El sistema debe validar que el correo tenga formato válido (nombre@dominio.com) y que no exista en la base de datos. -Las contraseñas ingresadas en ambos campos deben coincidir obligatoriamente. -Debe ingresar su nombre completo y teléfono adicionalmente. -Al registrarse con éxito, el sistema debe redirigir al usuario a la pantalla de inicio de sesión.	Criterios de Aceptación	-Podrá modificar su nombre, teléfono y foto de perfil. -Para cambiar la contraseña, el sistema debe exigir ingresar la "contraseña actual" antes de pedir la nueva dos veces

Código RF	RF-03	Código RF	RF-04
Nombre RF	Adjuntar Imagen	Nombre RF	Inhabilitar Perfil
Tipo de Requerimiento	Funcional Usuario	Tipo de Requerimiento	Funcional Usuario
Actores Relacionados	Tutor de Mascota / Veterinario	Actores Relacionados	Tutor de Mascota / Veterinario
Descripción	Opción para agregar imagen a su perfil.	Descripción	Opción de Inhabilitar su perfil.
Criterios de Aceptación	-Validará que el tamaño máximo por archivo no exceda el límite de 5 MB -Al seleccionar un archivo, la interfaz debe mostrar una vista previa de la imagen. -Al confirmar la subida, el sistema debe renombrar el archivo inmediatamente (utilizando la ID del usuario_fecha).	Criterios de Aceptación	-El sistema realizará un "borrado lógico" (cambio de estado a inactivo) del dueño y sus mascotas para no perder el historial de atenciones médicas pasadas. -Una vez inhabilitado, el usuario no podrá volver a iniciar sesión ni aparecerá en las búsquedas activas del sistema.

Código RF	RF-05	Código RF	RF-06
Nombre RF	Crear Perfil Médico	Nombre RF	Iniciar Sesión
Tipo de Requerimiento	Funcional Usuario	Tipo de Requerimiento	Funcional Usuario
Actores Relacionados	Administrador	Actores Relacionados	Tutor de Mascota / Veterinario / Admin
Descripción	Crear cuenta para veterinario.	Descripción	Autenticación en la plataforma.
Criterios de Aceptación	-El sistema debe validar que el correo contenga el dominio oficial de la clínica. -Deberá ingresar contraseña (2 veces para confirmar), nombre completo, teléfono y seleccionar una especialidad de una lista predefinida. -Solo un usuario con rol de Administrador podrá acceder a esta vista y ejecutar esta acción.	Criterios de Aceptación	-Deberá ingresar correo electrónico y contraseña. -El sistema validará credenciales con la base de datos. -Si son correctas, redirigirá al dashboard correspondiente al rol. -Si son incorrectas, mostrará la alerta de "Credenciales inválidas".

Código RF	RF-07	Código RF	RF-08
Nombre RF	Adjuntar Imagen	Nombre RF	Agregar Mascota
Tipo de Requerimiento	Funcional Usuario	Tipo de Requerimiento	Funcional Usuario
Actores Relacionados	Tutor de Mascota / Veterinario	Actores Relacionados	Tutor de Mascota
Descripción	Opción para agregar imagen a su perfil.	Descripción	Registro de una nueva mascota.
Criterios de Aceptación	-Validará que el tamaño máximo por archivo no exceda el límite de 5 MB. -Al seleccionar un archivo, la interfaz debe mostrar una vista previa de la imagen. -Al confirmar la subida, el sistema debe renombrar el archivo inmediatamente (utilizando la ID del usuario_fecha).	Criterios de Aceptación	-El formulario exigirá: nombre, especie, raza, sexo, edad aprox. y peso. -Los campos numéricos no aceptarán valores negativos. -La mascota quedará asociada automáticamente al ID del dueño que la registra.

Código RF	RF-09	Código RF	RF-10
Nombre RF	Editar Mascota	Nombre RF	Ver Historial Clínico
Tipo de Requerimiento	Funcional Usuario	Tipo de Requerimiento	Funcional Usuario
Actores Relacionados	Tutor de Mascota	Actores Relacionados	Tutor de Mascota
Descripción	Modificación de los datos de la mascota.	Descripción	Visualización de atenciones médicas pasadas.
Criterios de Aceptación	-Podrá actualizar peso, edad, o corregir raza / nombre. -No podrá modificar el historial clínico asociado a la mascota (esto es exclusivo del veterinario).	Criterios de Aceptación	-Mostrará una lista cronológica (más reciente primero) de vacunas, diagnósticos y recetas. -La vista será de "solo lectura" (el cliente no puede editar ni borrar estos registros).

Código RF	RF-11	Código RF	RF-12
Nombre RF	Ver Mis Citas	Nombre RF	Ver Calendario Disponible
Tipo de Requerimiento	Funcional Usuario	Tipo de Requerimiento	Funcional Usuario
Actores Relacionados	Tutor de Mascota	Actores Relacionados	Tutor de Mascota
Descripción	Panel de listado de citas futuras y pasadas.	Descripción	Visualización de bloques horarios para agendar.
Criterios de Aceptación	-Mostrará dos pestañas o secciones: "Próximas Citas" e "Historial de Citas". -Cada registro mostrará fecha, hora, mascota asignada y médico.	Criterios de Aceptación	-El sistema mostrará un calendario interactivo. -Sólo se mostrarán habilitados los bloques horarios que no estén ocupados ni bloqueados temporalmente por otro usuario.

Código RF	RF-13	Código RF	RF-14
Nombre RF	Iniciar Agendamiento	Nombre RF	Dashboard Médico
Tipo de Requerimiento	Funcional Usuario	Tipo de Requerimiento	Funcional Usuario
Actores Relacionados	Tutor de Mascota	Actores Relacionados	Veterinario
Descripción	Selección de datos para la nueva cita.	Descripción	Visualización de la agenda diaria / semanal.
Criterios de Aceptación	-Deberá seleccionar una mascota previamente registrada. -Deberá ingresar motivo de consulta. -Opcionalmente podrá seleccionar un médico de preferencia.	Criterios de Aceptación	-Mostrará un calendario con todas las citas asignadas a su ID. -Podrá filtrar la vista por día, semana o mes. -Las citas mostrarán estado (Pendiente, En curso, Finalizada).

Código RF	RF-15	Código RF	RF-16
Nombre RF	Ver Detalle de Atención	Nombre RF	Registrar Evolución Médica
Tipo de Requerimiento	Funcional Usuario	Tipo de Requerimiento	Funcional Usuario
Actores Relacionados	Veterinario	Actores Relacionados	Veterinario
Descripción	Acceso a la información de la cita seleccionada.	Descripción	Ingreso de datos clínicos durante/después de la consulta.
Criterios de Aceptación	-Al hacer clic en una cita, mostrará los datos del cliente, la mascota y el motivo de la consulta ingresado en el agendamiento.	Criterios de Aceptación	-Permitirá ingresar notas, diagnóstico, indicaciones y vacunas aplicadas. -Al guardar, el registro se añadirá al historial de la mascota con fecha y firma (ID del médico). -Una vez guardada la evolución, el estado de la cita pasará a "Finalizada".

Código RF	RF-17
Nombre RF	Subir Archivos Adjuntos
Tipo de Requerimiento	Funcional Usuario
Actores Relacionados	Veterinario
Descripción	Carga de exámenes o radiografías al historial.
Criterios de Aceptación	-El sistema permitirá adjuntar (imágenes, PDF) con un máximo de MB. -Validará que el tamaño máximo por archivo no exceda el límite definido. -Los archivos quedarán vinculados permanentemente a esa consulta específica.

3. Requisitos no funcionales

Código RNF	RNF 1.1	Código RNF	RNF 1.2
Nombre RNF	Bloqueo temporal de espacio en agenda	Nombre RNF	Optimización de Respuesta en Consultas de Datos de Alta Demanda
Tipo de Requerimiento	Rendimiento	Tipo de Requerimiento	Rendimiento
Actores Relacionados	Tutor de Mascota	Actores Relacionados	Veterinario, Administrador
Descripción	El sistema debe asegurar la disponibilidad de un turno seleccionado, bloqueándolo en la base de datos durante el proceso de reserva para evitar colisiones entre usuarios.	Descripción	El sistema debe garantizar una respuesta rápida y estable al cargar el historial clínico y el dashboard médico, incluso bajo condiciones de alta concurrencia.
Criterios de Aceptación	-El bloqueo debe registrarse en la base de datos en un tiempo máximo de 1 segundo después de que el usuario haga clic en el horario. -El sistema debe liberar automáticamente el horario si la reserva no se confirma (realizando el pago, si así lo decide la Veterinaria) en un lapso de 10 minutos. -Ningún otro usuario debe poder seleccionar o visualizar como	Criterios de Aceptación	-El tiempo de respuesta (latencia) para visualizar el historial clínico completo o el dashboard no debe exceder los 2 segundos desde la petición. -El rendimiento de 2 segundos debe mantenerse estable incluso cuando haya 500 usuarios realizando consultas simultáneamente. -La rapidez no debe comprometer la carga completa de la información; todos los módulos del

	"disponible" el mismo horario mientras el bloqueo esté activo.		dashboard y registros del historial deben renderizar correctamente.
--	--	--	---

Código RNF	RNF 1.3	Código RNF	RNF 2.1
Nombre RNF	Optimización de Generación y Descarga de PDF	Nombre RNF	Hashing Unidireccional de Contraseñas
Tipo de Requerimiento	Rendimiento	Tipo de Requerimiento	Seguridad
Actores Relacionados	Veterinario, Tutor, Administrador	Actores Relacionados	Administrador, Tutor, Veterinario
Descripción	El backend debe procesar y entregar archivos PDF (recetas y comprobantes) de manera ágil para evitar esperas prolongadas en la interfaz de usuario.	Descripción	El sistema debe transformar las contraseñas de los usuarios en un código irreversible (hash) antes de guardarlas, garantizando que nadie pueda ver la clave original en la base de datos.
Criterios de Aceptación	-El tiempo desde que se solicita el documento hasta que el backend termina de generarlo no debe exceder los 3 segundos. -El archivo generado debe ser un PDF estándar, compatible con cualquier lector común, manteniendo el diseño y las firmas digitales (si aplica). -Una vez generado el archivo en el backend, el enlace de descarga o la apertura automática del documento debe ser inmediata y sin errores de "archivo no encontrado".	Criterios de Aceptación	-El sistema debe utilizar algoritmos de hash robustos y modernos (como Argon2 o BCrypt) que impidan técnicamente obtener la contraseña original a partir del hash almacenado. -Cada contraseña debe combinarse con un valor aleatorio único (salt) antes de procesar el hash, evitando así ataques de diccionario o tablas de arcoíris. -El proceso de autenticación debe comparar el hash generado en el momento del inicio de sesión con el hash guardado en la base de datos, sin que la contraseña en texto plano sea persistida en ningún log o archivo temporal.

Código RNF	RNF 2.2	Código RNF	RNF 2.3
Nombre RNF	Autenticación Segura mediante Tokens de Expiración Dinámica	Nombre RNF	Bloqueo Preventivo contra Ataques de Fuerza Bruta
Tipo de Requerimiento	Seguridad	Tipo de Requerimiento	Seguridad
Actores Relacionados	Veterinario, Tutor, Administrador	Actores Relacionados	Administrador, Tutor, Veterinario
Descripción	Implementar un sistema de validación basado en JWT (JSON Web Tokens) que proteja	Descripción	El sistema debe salvaguardar las cuentas bloqueando temporalmente el acceso tras detectar repetidos

	las comunicaciones entre el cliente y el servidor, forzando el cierre de sesión tras 30 minutos de inactividad.		intentos de inicio de sesión fallidos, evitando que atacantes prueben contraseñas masivamente.
Criterios de Aceptación	-Todas las rutas de la API (REST) deben exigir un token válido en el encabezado de la petición. Si el token no está presente o es inválido, el sistema debe responder con un error 401 Unauthorized. -El sistema debe invalidar el token y rechazar cualquier petición posterior si han transcurrido más de 30 minutos desde la última interacción registrada del usuario con el backend. -Los tokens JWT deben estar firmados digitalmente (usando una clave secreta robusta o certificados) para asegurar que no han sido manipulados por terceros durante el tránsito de datos.	Criterios de Aceptación	-El sistema debe llevar un contador temporal y restringir el acceso a la cuenta en el instante en que se registre el quinto intento fallido consecutivo. -Si el usuario ingresa sus credenciales correctamente en el primer, segundo o tercer intento, el contador de fallos de esa cuenta debe reiniciarse automáticamente a cero. -La cuenta debe permanecer inaccesible por un periodo de gracia definido (por ejemplo, 15 o 30 minutos), tras el cual el sistema debe levantar el bloqueo de forma automática sin intervención manual.

Código RNF	RNF 3.1	Código RNF	RNF 3.2
Nombre RNF	Acceso Directo al Historial Clínico desde la Agenda	Nombre RNF	Flujo Guiado y Optimizado de Recuperación de Contraseña
Tipo de Requerimiento	Usabilidad	Tipo de Requerimiento	Usabilidad
Actores Relacionados	Veterinario	Actores Relacionados	Administrador, Tutor, Veterinario
Descripción	El panel médico debe optimizar la navegación para que el expediente de una mascota se pueda abrir rápidamente desde la vista del calendario, minimizando la fricción y el tiempo de clics.	Descripción	El sistema debe ofrecer un proceso intuitivo, rápido y con fricción mínima para restablecer credenciales olvidadas, limitando toda la interacción a un máximo de dos pantallas.

Criterios de Aceptación	-La ruta completa de navegación, partiendo desde la vista principal del calendario de citas hasta la visualización total del historial clínico de la mascota, no debe requerir más de 3 o 4 interacciones (clics o toques) -Al hacer clic en el acceso directo de la cita, el sistema debe cargar automáticamente el perfil exacto de la mascota programada, listo para registrar o visualizar datos (como las vacunas), sin necesidad de usar un buscador intermedio. -La tarjeta de la cita en el calendario o su modal de resumen debe contener un botón o enlace primario claramente identificable (ej. "Ver Historial") para ejecutar esta acción de forma intuitiva.	Criterios de Aceptación	-El proceso completo, desde la solicitud inicial hasta el guardado de la nueva clave, no debe requerir que el usuario navegue por más de 2 pantallas o modales distintos. -La interfaz debe proveer instrucciones directas y validaciones en tiempo real en ambas pantallas (por ejemplo, indicar si el correo existe o mostrar los requisitos de seguridad mientras se teclea la nueva contraseña) para guiar al usuario sin confusión. -Al enviar el formulario en la segunda pantalla, el sistema debe procesar el cambio y mostrar un mensaje claro de éxito, habilitando inmediatamente la redirección a la pantalla principal de inicio de sesión.
--------------------------------	---	--------------------------------	---

Código RNF	RNF 3.3	Código RNF	RNF 4.1
Nombre RNF	Accesibilidad y Claridad Visual en el Calendario	Nombre RNF	Compatibilidad Multiplataforma y Diseño Responsivo
Tipo de Requerimiento	Usabilidad	Tipo de Requerimiento	Usabilidad
Actores Relacionados	Veterinario, Tutor	Actores Relacionados	Tutor
Descripción	La interfaz del calendario debe garantizar una lectura rápida y sin esfuerzo de los estados de las citas, utilizando colores con alto contraste e iconos que eviten depender únicamente del color para transmitir información.	Descripción	La interfaz orientada al cliente debe operar correctamente en navegadores web (React) y contar con una versión móvil nativa independiente (React Native + Expo).
Criterios de Aceptación	-La paleta de colores utilizada para los fondos y los textos/iconos de los bloques horarios debe cumplir, como mínimo, con el nivel de conformidad AA de las pautas de accesibilidad WCAG -La interfaz debe implementar FullCalendar (React version), adaptando sus estilos para garantizar un alto contraste. Cada estado de cita debe usar patrones visuales y no depender	Criterios de Aceptación	-Todos los elementos visuales (botones, calendarios, textos) deben redimensionarse y apilarse automáticamente para ajustarse al ancho de la pantalla del dispositivo móvil, eliminando por completo la necesidad de hacer scroll horizontal. -La aplicación móvil debe ejecutarse de forma nativa con todas sus funcionalidades intactas, sin errores de compatibilidad, en dispositivos que utilicen iOS 14 o

	solo del color dentro de los eventos renderizados por FullCalendar. -Los tres estados deben ser claramente diferenciables entre sí a simple vista, tanto en monitores de alta resolución como en pantallas móviles con brillo reducido o condiciones de mucha luz exterior.		superior y Android 10 o superior. -El sistema debe entregar una aplicación móvil construida y empaquetada con React Native y Expo, ejecutable en iOS y Android de forma nativa, eliminando la necesidad de PWA o Service Workers.
--	---	--	---

Código RNF	RNF 4.2	Código RNF	RNF 4.3
Nombre RNF	Empaquetado y Ejecución Autónoma del Backend	Nombre RNF	Portabilidad y Compatibilidad Multi-Navegador
Tipo de Requerimiento	Portabilidad	Tipo de Requerimiento	Usabilidad
Actores Relacionados	Administrador	Actores Relacionados	Administrador, Tutor, Veterinario
Descripción	El código del servidor y sus dependencias deben empaquetarse en un único archivo ejecutable (JAR) utilizando el servidor web embebido de Spring Boot, garantizando que la aplicación se ejecute en cualquier infraestructura con Java instalado.	Descripción	La aplicación debe poder ejecutarse de forma consistente en diferentes navegadores web estándar, asegurando que el código no dependa de un entorno exclusivo para funcionar.
Criterios de Aceptación	-El proyecto debe generar automáticamente un archivo .jar compilado mediante Maven o Gradle, que contenga toda la lógica, dependencias y el servidor web (Tomcat) embebido. -El empaquetado debe poder ejecutarse en los tres entornos (Desarrollo, Pruebas, Producción) utilizando exactamente el mismo archivo .jar, cambiando el comportamiento mediante variables de entorno o perfiles de Spring (application.properties). -El sistema backend debe levantar y funcionar correctamente al ejecutar un único comando estándar (ej. java -jar app.jar) en el servidor anfitrión de AWS, sin requerir	Criterios de Aceptación	-Las funciones críticas (como agendar, ver historial o descargar los PDFs) deben operar correctamente y sin errores de consola en las últimas versiones estables de Chrome, Firefox y Edge. -Los elementos de la interfaz (como el calendario accesible o los formularios) deben mantener su estructura, sin que los botones se deformen o los textos se solapen al cambiar de un navegador a otro. -El código frontend no debe utilizar funciones experimentales o características exclusivas que solo existan en un navegador específico, garantizando que el estándar web (HTML/CSS/JS) sea universal.

	configuraciones adicionales del servidor web.		
--	---	--	--

Código RNF	RNF 5.1	Código RNF	RNF 5.2
Nombre RNF	Alta Disponibilidad del Servicio	Nombre RNF	Gestión de Despliegues en Ventanas de Mantenimiento
Tipo de Requerimiento	Disponibilidad	Tipo de Requerimiento	Disponibilidad
Actores Relacionados	Veterinario, Tutor	Actores Relacionados	Administrador
Descripción	El sistema debe mantenerse operativo y accesible de forma casi ininterrumpida, garantizando que el tiempo total de caída o inaccesibilidad no supere las 9 horas en un año calendario.	Descripción	Las actualizaciones de software y tareas de mantenimiento del servidor deben ejecutarse estrictamente en horarios de mínima o nula actividad operativa para evitar la interrupción del servicio.
Criterios de Aceptación	-El backend debe exponer una ruta de verificación (ej. un endpoint /health) que permita a herramientas externas comprobar constantemente si el servidor está vivo y respondiendo. -En caso de que el proceso principal de la aplicación falle o se congele, el entorno de despliegue (como el gestor de procesos del servidor o servicio de AWS) debe reiniciar el sistema automáticamente para restaurar el servicio en segundos. -Los registros del servidor y las herramientas de monitoreo deben evidenciar que las interrupciones no planificadas del servicio suman un máximo de ~43 minutos al mes (para mantenerse dentro de la métrica anual de las 9 horas).	Criterios de Aceptación	-Los procesos de actualización automatizados (como pipelines de CI/CD) o las tareas del servidor deben restringir su ejecución a producción exclusivamente dentro de la franja de 02:00 AM a 05:00 AM. -Si el sistema necesita reiniciarse o estar inactivo temporalmente durante esa ventana, debe interceptar cualquier petición entrante y mostrar una pantalla amigable de "Mantenimiento Programado" (devolviendo un código HTTP 503), evitando que la web quede cargando al infinito. -El panel administrativo debe contar con un mecanismo (como la activación de un banner global en la interfaz) para notificar al personal médico con horas de anticipación sobre el corte nocturno planificado.

Código RNF	RNF 5.3	Código RNF	RNF 6.1
Nombre RNF	Conmutación por Error Automática	Nombre RNF	Exposición y Documentación Automática de la API RESTful
Tipo de Requerimiento	Disponibilidad	Tipo de Requerimiento	Interoperabilidad

Actores Relacionados	Administrador	Actores Relacionados	Administrador
Descripción	El sistema debe utilizar un balanceador de carga que detecte caídas en el servidor principal y desvíe el tráfico de los usuarios hacia un servidor de respaldo en menos de 60 segundos, evitando la interrupción total del servicio.	Descripción	El backend debe proveer sus servicios de datos mediante una API estructurada, integrando una herramienta que genere y actualice su manual de uso automáticamente a partir del código fuente.
Criterios de Aceptación	-El balanceador de carga debe sondear continuamente (por ejemplo, cada 5 o 10 segundos) el estado de salud del servidor principal para identificar rápidamente si deja de responder. -Al confirmarse la caída del servidor primario, el sistema debe actualizar sus reglas de enrutamiento y enviar todas las peticiones entrantes a la instancia secundaria (respaldo) en un tiempo máximo de 1 minuto. -La instancia de respaldo debe estar conectada a la misma base de datos centralizada que el servidor principal, asegurando que el tráfico redirigido no se encuentre con un sistema vacío o con datos desactualizados.	Criterios de Aceptación	-Todas las rutas del backend deben respetar las convenciones RESTful, utilizando correctamente los verbos HTTP (GET para obtener datos, POST para crear, PUT/PATCH para actualizar, DELETE para borrar) y devolviendo los códigos de estado adecuados (200, 201, 400, 404, 500) -El proyecto debe tener configurada una librería (como Swagger UI o Redoc) que lea los comentarios o anotaciones directamente en el código del backend y construya el archivo de especificación (OpenAPI) sin necesidad de redactar documentos manuales en Word o PDF. -La documentación debe exponerse en una ruta web (por ejemplo, /api-docs) que muestre visualmente todos los endpoints disponibles, los parámetros que requieren, y que permita probar las peticiones directamente desde el navegador.

Código RNF	RNF 6.2	Código RNF	RNF 6.3
Nombre RNF	Comunicación Segura con Pasarela de Pagos (JSON sobre HTTPS)	Nombre RNF	Integración Automatizada de Correo Electrónico (SMTP/API)
Tipo de Requerimiento	Interoperabilidad	Tipo de Requerimiento	Interoperabilidad
Actores Relacionados	Tutor	Actores Relacionados	Tutor, Veterinario
Descripción	El backend debe enviar y recibir los datos de las transacciones financieras hacia el proveedor externo utilizando el estándar de formato JSON, operando estrictamente a través de un	Descripción	El sistema backend debe conectarse de forma segura a un proveedor de correo externo para automatizar el envío de notificaciones transaccionales (confirmación de citas,

	canal cifrado para proteger la información.		recuperación de claves o envío de recetas).
Criterios de Aceptación	-Toda petición de red enviada hacia la API de pagos (y los webhooks de confirmación que regresen al servidor) debe viajar obligatoriamente por el protocolo HTTPS, bloqueando cualquier intento de transmisión en texto plano. -La información de la orden (monto, identificador de la compra, moneda) debe empaquetarse y enviarse como un objeto JSON válido, y el sistema debe ser capaz de parsear correctamente la respuesta JSON del proveedor. -El sistema debe procesar adecuadamente las respuestas de rechazo (fondos insuficientes, timeout de la pasarela) devueltas en el JSON, cancelando el flujo de reserva y notificando al usuario sin exponer detalles técnicos del error.	Criterios de Aceptación	-Las credenciales de conexión al servidor SMTP o las claves de la API (host, puerto, usuario, API Key) deben inyectarse de forma dinámica mediante variables de entorno, prohibiendo estrictamente dejarlas escritas en texto plano en el código fuente. -El módulo de envío debe ser capaz de procesar y despachar plantillas de correo en formato HTML (no solo texto plano), permitiendo que los mensajes incluyan el logo de la clínica, enlaces de botones y un diseño profesional. -El intento de envío del correo no debe congelar la pantalla del usuario. Si el servidor de SendGrid o Gmail tarda un par de segundos en responder, el sistema debe confirmarle la acción al usuario inmediatamente en la interfaz y procesar el correo en segundo plano.

Código RNF	RNF 7.1	Código RNF	RNF 7.2
Nombre RNF	Control de Concurrencia en Reserva de Citas	Nombre RNF	Auditoría de Cambios y Borrado Lógico (Soft Delete) en Expedientes
Tipo de Requerimiento	Integridad	Tipo de Requerimiento	Integridad
Actores Relacionados	Tutor	Actores Relacionados	Administrador, Veterinario
Descripción	El sistema debe prevenir el "overbooking" (doble reserva) bloqueando temporalmente el horario seleccionado en la base de datos mientras el usuario completa el proceso de agendamiento.	Descripción	El sistema debe mantener un registro histórico de quién y cuándo modifica un historial clínico, ocultando los datos eliminados de la interfaz en lugar de borrarlos físicamente de la base de datos.

Criterios de Aceptación	-En el instante en que un usuario selecciona un bloque de horario, el sistema debe marcarlo como "En proceso" en la base de datos, impidiendo que cualquier otro usuario lo vea como disponible en sus pantallas. -El sistema debe contar con un temporizador que libere automáticamente el bloque horario (volviéndolo al estado "Disponible") si el usuario abandona la página o no completa la reserva dentro de un tiempo límite predefinido (ej. 10 minutos). -Si el usuario decide retroceder o presiona un botón de "Cancelar" durante el flujo de reserva, el sistema debe liberar el bloqueo de la base de datos inmediatamente, sin esperar a que se cumpla el tiempo límite del temporizador.	Criterios de Aceptación	-Cada vez que se inserte o actualice una entrada en el historial clínico, el sistema debe guardar automáticamente la fecha y hora exacta del servidor (timestamp) y el identificador único (ID) de la cuenta que realizó la acción. -Al accionar el botón de "Eliminar" en la interfaz, el sistema no debe ejecutar el comando DELETE en SQL. En su lugar, debe actualizar un atributo específico de la tabla (por ejemplo, cambiar is_deleted a verdadero, o llenar un campo deleted_at con la fecha actual). -Todas las consultas (queries) regulares que alimentan la interfaz del expediente deben excluir automáticamente aquellos registros que estén marcados como eliminados, asegurando que el usuario vea un historial limpio sin información obsoleta o descartada.
--------------------------------	---	--------------------------------	---

Código RNF	RNF 7.3	Código RNF	RNF 8.1
Nombre RNF	Validación de Integridad de Datos Numéricos en Backend	Nombre RNF	Escalabilidad Horizontal del Backend
Tipo de Requerimiento	Integridad	Tipo de Requerimiento	Escalabilidad
Actores Relacionados	Veterinario, Tutor	Actores Relacionados	Administrador
Descripción	El servidor debe verificar obligatoriamente que los atributos físicos de la mascota (como edad, peso o temperatura) estén dentro de rangos matemáticos y biológicamente posibles antes de guardarlos en la base de datos.	Descripción	La arquitectura del servidor debe estar diseñada para soportar la creación rápida de múltiples copias (instancias) de la aplicación, distribuyendo el tráfico de usuarios para mantener el rendimiento durante eventos de alta demanda.

Criterios de Aceptación	-El sistema debe retornar un error de validación (ej. código HTTP 400 Bad Request) si recibe peticiones donde campos como peso o edad sean menores o iguales a 0. -El backend debe tener topes máximos configurados (por ejemplo, rechazar una edad mayor a 30 años o un peso superior a 150 kg para mascotas convencionales), evitando que un error de tipeo ensucie las estadísticas de la clínica. -Esta validación lógica debe ejecutarse en el servidor de forma inquebrantable, incluso si la interfaz gráfica web o móvil ya cuenta con bloqueos visuales (como inputs numéricos que no dejan escribir el signo menos).	Criterios de Aceptación	-El código del backend no debe guardar información temporal (como sesiones de usuario o archivos subidos a medias) en el disco duro o memoria local de ese servidor en particular, permitiendo que cualquier instancia nueva pueda procesar cualquier petición sin perder el contexto. -El entorno de despliegue debe contar con un balanceador de carga configurado que, al detectar que una nueva instancia se ha encendido y está lista, comience a enviarle peticiones de los clientes de manera equitativa. -El arranque de una nueva instancia del servidor (ej. un nuevo servidor virtual en AWS EC2 o Elastic Beanstalk) debe demorar solo unos segundos o pocos minutos, conectándose a la base de datos centralizada sin requerir la intervención de un programador o la reestructuración del código.
--------------------------------	---	--------------------------------	--

Código RNF	RNF 8.2	Código RNF	RNF 8.3
Nombre RNF	Diseño de Capa de Datos para Separación de Lectura y Escritura	Nombre RNF	Almacenamiento Externo de Archivos Clínicos (Object Storage)
Tipo de Requerimiento	Escalabilidad	Tipo de Requerimiento	Escalabilidad
Actores Relacionados	Veterinario, Administrador	Actores Relacionados	Administrador, Veterinario
Descripción	El código del backend en Spring Boot debe estructurarse de manera que permita, en el futuro, configurar conexiones independientes para leer datos y para guardar datos, evitando cuellos de botella en la base de datos MySQL principal.	Descripción	El sistema debe guardar los archivos adjuntos (como imágenes y PDFs médicos) en un servicio de almacenamiento en la nube dedicado (Cloudinary), manteniendo el disco del servidor principal libre y optimizado.

Criterios de Aceptación	-El sistema de acceso a datos implementado con Spring Data JPA e Hibernate debe soportar la configuración de al menos dos orígenes de datos (DataSources) hacia MySQL: uno exclusivo para operaciones de modificación (INSERT, UPDATE, DELETE) y otro para consultas de solo lectura (SELECT). -La transición de usar una única base de datos a usar un modelo de base de datos principal/réplica debe lograrse modificando únicamente las variables de entorno (.env), sin tener que reescribir la lógica de los controladores o servicios del backend. -La lógica del frontend y backend no debe romperse si hay un ligero retraso (milisegundos) entre el momento en que se guarda un dato en la base de escritura y el momento en que aparece disponible en la base de lectura.	Criterios de Aceptación	-Al adjuntar un examen o radiografía, el backend en Spring Boot debe enviar el archivo físico directamente al servicio de almacenamiento en la nube Cloudinary, guardando únicamente la URL de referencia en la base de datos MySQL. -Dado que son documentos médicos, el servicio no debe exponer los archivos de forma pública. El sistema debe generar enlaces temporales de acceso (Signed URLs o tokens de acceso) que expiren tras unos minutos para que solo el usuario autenticado pueda verlos o descargarlos. -El volumen de almacenamiento del servidor de la aplicación (donde corre tu código) no debe aumentar de forma permanente con el uso diario, asegurando que si el servidor o la instancia de AWS se reinicia o destruye, no se pierda ninguna radiografía.
--------------------------------	--	--------------------------------	--

Código RNF	RNF 9.1	Código RNF	RNF 9.2
Nombre RNF	Automatización y Gestión Centralizada de Dependencias	Nombre RNF	Integración y Entrega Continua (CI/CD)
Tipo de Requerimiento	Instalabilidad	Tipo de Requerimiento	Instalabilidad
Actores Relacionados	Administrador	Actores Relacionados	Administrador
Descripción	El código fuente del proyecto debe integrar herramientas que administren, versionen y descarguen automáticamente las librerías de terceros necesarias para compilar y ejecutar el ecosistema (Spring Boot y React).	Descripción	El proceso de lanzar nuevas versiones del sistema a producción debe realizarse mediante secuencias de comandos automatizadas (pipelines) que publiquen el código de forma segura en un máximo de 15 minutos.

Criterios de Aceptación	-El código debe incluir archivos de configuración raíz válidos según el entorno: pom.xml o build.gradle para la gestión de librerías en Java/Spring Boot, y package.json para el entorno Node.js en los proyectos de React y React Native. -Un desarrollador o el servidor en la nube debe poder descargar y vincular todas las dependencias del ecosistema ejecutando un único comando estándar por consola (ej. mvn clean install o npm install) al clonar el proyecto por primera vez, sin necesidad de instalaciones manuales. -El código fuente subido al sistema de control de versiones (como GitHub o GitLab) debe ignorar estrictamente las carpetas donde se descargan estas librerías (ej. usando .gitignore para omitir node_modules o la carpeta target), manteniendo el repositorio ligero y limpio.	Criterios de Aceptación	-El proceso de despliegue debe iniciarse automáticamente en el servidor (ej. usando herramientas como GitHub Actions, GitLab CI o Jenkins) en el instante en que el código nuevo sea integrado (merge) a la rama principal (main/master) del repositorio. -El tiempo total medido por la herramienta de automatización, desde que inicia el proceso de compilación del código (creación del ejecutable) hasta que la aplicación está corriendo y respondiendo en producción, no debe superar la barrera de los 15 minutos. -Durante la ventana de despliegue, ningún desarrollador o administrador debe tener que conectarse al servidor manualmente por consola (SSH) ni subir archivos sueltos (ej. vía FTP) para que la actualización se aplique exitosamente.
--------------------------------	---	--------------------------------	--

Código RNF	RNF 9.3	Código RNF	RNF 10.1
Nombre RNF	Automatización de Migraciones de Base de Datos	Nombre RNF	Copias de Seguridad Automáticas e Incrementales (Backups)
Tipo de Requerimiento	Instalabilidad	Tipo de Requerimiento	Confiabilidad
Actores Relacionados	Administrador	Actores Relacionados	Administrador
Descripción	El sistema debe integrar una herramienta de control de versiones para la base de datos (migraciones) que construya y actualice la estructura de las tablas de forma automática al iniciar o desplegar la aplicación.	Descripción	El sistema debe proteger la información crítica de atenciones y usuarios mediante la ejecución programada de respaldos cada 12 horas, guardando únicamente los datos nuevos o modificados para optimizar el almacenamiento y el tiempo de procesamiento.

Criterios de Aceptación	-Al arrancar el servidor backend, el sistema debe leer los archivos de migración y ejecutar automáticamente los cambios pendientes (crear tablas, añadir columnas) sin requerir que un humano copie y pegue scripts de SQL manualmente. -La herramienta elegida debe generar una tabla de control (oculta al usuario) dentro de la base de datos que registre el historial exacto de qué migraciones ya fueron aplicadas. Esto garantiza que un mismo cambio no intente ejecutarse dos veces y cause errores al iniciar el sistema. -El uso de migraciones debe garantizar de forma estricta que la estructura (esquema) de la base de datos sea idéntica tanto en la computadora de desarrollo local como en el servidor de producción de la clínica.	Criterios de Aceptación	-El servidor debe contar con un disparador de tiempo (ej. un cron job) que inicie el proceso de respaldo de la base de datos de forma automática exactamente cada 12 horas, sin requerir clics ni intervención humana. -El proceso de respaldo no debe generar un archivo con toda la historia de la clínica desde cero en cada ejecución. Solo debe capturar e indexar los registros (usuarios, atenciones, pagos) que hayan sido creados, editados o eliminados desde el último respaldo exitoso. -El archivo resultante del respaldo debe guardarse en un volumen o servicio externo (para que no se pierda si el servidor principal se incendia) y el sistema debe generar un registro (log) que indique si la copia fue exitosa o si ocurrió un error.
-------------------------	---	-------------------------	--

Código RNF	RNF 10.2	Código RNF	RNF 10.3
Nombre RNF	Procesamiento Asíncrono y Tolerancia a Fallos en Correos	Nombre RNF	Tiempo de Recuperación ante Fallos
Tipo de Requerimiento	Confiabilidad	Tipo de Requerimiento	Confiabilidad
Actores Relacionados	Tutor, Administrador	Actores Relacionados	Administrador
Descripción	El sistema debe utilizar una cola de mensajes en segundo plano para gestionar el envío de correos, garantizando que los mensajes no se pierdan y se reintenten automáticamente si el proveedor externo falla.	Descripción	Tras sufrir un cierre inesperado o un colapso por falta de recursos (ej. memoria RAM llena), el sistema debe reiniciarse automáticamente y restaurar todos sus servicios en un tiempo máximo de 5 minutos.

Criterios de Aceptación	-Al momento de disparar un correo (ej. al guardar una cita), el sistema debe registrar la tarea en la cola y devolver el control al usuario inmediatamente en la interfaz, sin obligarlo a mirar una pantalla de carga mientras el servidor se comunica con el proveedor de correos. -Si el servidor SMTP o la API de correos (ej. SendGrid) devuelve un error de conexión o timeout, la cola de tareas debe interceptar la falla y reprogramar el envío automáticamente (por ejemplo, reintentar a los 5, 15 y 30 minutos) sin intervención humana. -Si un correo agota su número máximo de reintentos permitidos sin éxito, el sistema debe retirarlo de la cola principal, moverlo a un registro de "tareas fallidas" y guardar el error exacto en los logs del servidor para su posterior análisis.	Criterios de Aceptación	-El entorno de ejecución (ej. mediante gestores de procesos o servicios del sistema operativo como Systemd) debe detectar la caída del proceso principal e iniciar una nueva instancia de la aplicación inmediatamente sin intervención humana. -El tiempo total cronometrado desde que el proceso muere hasta que el servidor vuelve a aceptar tráfico y responde exitosamente a un health check (verificación de estado) no debe superar la marca de los 5 minutos. -Al arrancar nuevamente, el backend debe ser capaz de restablecer sus conexiones con la base de datos, colas de tareas y servicios externos de forma automática, quedando listo para operar sin necesidad de reconfiguraciones manuales.
-------------------------	--	-------------------------	---

Código RNF	RNF 11.1	Código RNF	RNF 11.2
Nombre RNF	Reversión Automática de Citas por Fallo de Pago	Nombre RNF	Validación Restrictiva y Manejo de Excepciones en Carga de Archivos
Tipo de Requerimiento	Robustez	Tipo de Requerimiento	Robustez
Actores Relacionados	Tutor, Administrador	Actores Relacionados	Veterinario
Descripción	El sistema debe anular la retención del bloque horario y registrar la incidencia interna si la comunicación con el proveedor de pagos falla, notificando al usuario de manera comprensible y segura.	Descripción	El sistema debe inspeccionar el peso y la extensión de cualquier archivo adjunto antes de procesarlo, bloqueando aquellos que superen los 5MB o posean formatos no autorizados, y devolviendo un mensaje de error controlado.

Criterios de Aceptación	-Si la pasarela de pagos (ej. Flow) no responde o devuelve un error de timeout, el backend debe ejecutar una actualización inmediata en la base de datos para cambiar el estado de ese turno de "Bloqueado" a "Disponible" nuevamente. -El sistema debe capturar el código del error exacto devuelto por la pasarela y guardarlo silenciosamente en los logs del servidor (archivos de registro) junto con la fecha y hora, sin exponer esta información sensible en el navegador del cliente. -El frontend debe interceptar la falla y mostrar una alerta o ventana modal (ej. "Hubo un problema de conexión con el banco. Tu cita no fue agendada ni cobrada. Por favor, inténtalo de nuevo"), ocultando cualquier código de error técnico (como un Error 500) que pueda confundir al usuario.	Criterios de Aceptación	-El backend (o un middleware intermedio) debe interceptar y rechazar automáticamente cualquier petición que contenga un archivo cuyo tamaño sobrepase los 5MB, devolviendo un código HTTP adecuado (como 413 Payload Too Large o 400 Bad Request). -El sistema debe verificar la firma interna del archivo (no solo la extensión en el nombre) para permitir únicamente formatos seguros y predefinidos (ej. PDF, JPG, PNG), bloqueando estrictamente ejecutables, scripts o documentos corruptos. -Ante un archivo inválido, el código debe atrapar el error antes de que el proceso colapse. Debe evitar arrojar un error genérico de servidor (HTTP 500 Internal Server Error) y, en su lugar, retornar un mensaje claro y en formato JSON que el frontend pueda traducir en una alerta amigable (ej. "El archivo supera los 5MB permitidos").
--------------------------------	---	--------------------------------	--

Código RNF	RNF 11.3	Código RNF	RNF 12.1
Nombre RNF	Sanitización de Entradas y Prevención de Inyecciones (SQLi / XSS)	Nombre RNF	Arquitectura de Capas en el Backend
Tipo de Requerimiento	Robustez	Tipo de Requerimiento	Modificabilidad
Actores Relacionados	Administrador	Actores Relacionados	Administrador
Descripción	El sistema debe limpiar, validar y escapar toda la información de texto enviada por los clientes hacia el servidor, neutralizando cualquier intento de inyectar código malicioso en la base de datos o en el navegador de otros usuarios.	Descripción	Estructuración del código del lado del servidor utilizando un patrón de arquitectura por capas (Controladores, Servicios y Repositorios), con el objetivo de separar las responsabilidades, aislar la lógica de negocio de la infraestructura y asegurar un código fácil de mantener y escalar.

Criterios de Aceptación	-El acceso a datos en el backend (ya sea a través de un ORM o consultas crudas) debe utilizar exclusivamente declaraciones preparadas o parametrizadas, garantizando que el texto ingresado por el usuario nunca se concatene directamente con la sintaxis SQL. -Todo texto que el sistema reciba (como las observaciones de una consulta) y deba mostrarse posteriormente en la interfaz, debe ser procesado por una librería de sanitización que neutralice o elimine etiquetas HTML y scripts ejecutables (ej. <code><script></code>). -Si el backend o el middleware detectan un patrón de ataque evidente en el cuerpo (payload) de la petición, el sistema debe abortar el proceso inmediatamente, devolver un error HTTP 400 (Bad Request) y guardar una alerta en los logs de seguridad.	Criterios de Aceptación	-Dado que llega una solicitud web a la API, cuando el sistema reciba la petición, entonces la capa de Controladores debe limitarse a capturar los datos, delegar la ejecución a la capa inferior y retornar la respuesta HTTP, sin incluir lógica de negocio. -Dado que se requiere procesar un caso de uso específico, cuando el sistema aplique las reglas de la aplicación, entonces la capa de Servicios debe resolver toda la lógica de forma centralizada, sin interactuar con los objetos de la petición HTTP ni escribir consultas directas a la base de datos. -Dado que se necesita persistir o consultar información, cuando el sistema interactúe con la base de datos, entonces debe hacerlo única y exclusivamente a través de la capa de Repositorios, retornando la información estructurada hacia los servicios.
-------------------------	---	-------------------------	--

Código RNF	RNF 12.2	Código RNF	RNF 12.3
Nombre RNF	Desacoplamiento del Proveedor de Pagos (Patrón de Diseño)	Nombre RNF	Adherencia a Principios SOLID y Extensibilidad del Código
Tipo de Requerimiento	Modificabilidad	Tipo de Requerimiento	Modificabilidad
Actores Relacionados	Administrador	Actores Relacionados	Administrador
Descripción	Implementación de un patrón de diseño estructural o de comportamiento (como Adapter o Strategy) en la integración de pagos, garantizando que el módulo de agendamiento opere de forma completamente independiente del proveedor externo y facilitando la adición o migración de pasarelas en el futuro.	Descripción	Aplicación de los principios de diseño de software SOLID en la arquitectura del proyecto para asegurar un código mantenible y escalable, permitiendo la integración de nuevas funcionalidades (como un inventario) sin riesgo de afectar o romper los módulos ya existentes.

Criterios de Aceptación	-Dado que se toma la decisión de cambiar la pasarela de pagos, cuando el desarrollador integre el nuevo servicio, entonces debe poder habilitarlo creando una nueva implementación del patrón, sin necesidad de modificar ni alterar el código existente del módulo de agendamiento. -Dado que se confirma una reserva y se inicia el cobro, cuando el sistema procese la transacción, entonces debe comunicarse con el proveedor configurado utilizando exclusivamente los métodos definidos en la interfaz común del patrón, logrando completar el pago con éxito. -Dado que la pasarela externa devuelve una respuesta nativa o un error específico de su plataforma, cuando el sistema reciba esta información a través del adaptador, entonces debe traducirla inmediatamente a un formato estándar interno para no acoplar la lógica de la aplicación a los detalles del proveedor.	Criterios de Aceptación	-Dado que se requiere agregar una nueva funcionalidad o módulo, cuando el desarrollador implemente la solución, entonces debe lograrlo añadiendo nuevo código (extendiendo clases o implementando interfaces) sin necesidad de modificar el código fuente de los módulos que ya están operando. -Dado que se ha integrado un nuevo módulo al ecosistema, cuando el sistema CI/CD ejecute las pruebas de regresión automatizadas, entonces el sistema debe confirmar que las funcionalidades previas siguen operando correctamente y sin alteraciones. -Dado que el código se envía para revisión, cuando el revisor de código (o líder técnico) evalúe la solicitud de integración (Pull Request), entonces debe validar que los nuevos componentes cumplen con tener una única responsabilidad y mantienen un bajo nivel de dependencia con el resto del sistema.
--------------------------------	--	--------------------------------	---

Código RNF	RNF 13.1	Código RNF	RNF 13.2
Nombre RNF	Componentes Visuales Independientes y Reutilizables (Frontend)	Nombre RNF	Encapsulación de Validaciones Transversales en Clases Utilitarias
Tipo de Requerimiento	Reusabilidad	Tipo de Requerimiento	Reusabilidad
Actores Relacionados	Administrador	Actores Relacionados	Tutor, Veterinario
Descripción	Desarrollo de la interfaz gráfica mediante una arquitectura modular, creando elementos (botones, modales, formularios) autónomos que puedan ser instanciados y reutilizados a lo largo de las distintas vistas de la aplicación.	Descripción	Centralización de las lógicas de validación de uso común (como formato de RUT, correos electrónicos y cálculo de edad) en clases utilitarias compartidas, con el fin de reutilizar el código y evitar su duplicación a lo largo del proyecto.

Criterios de Aceptación	-Dado que se requiere un elemento visual preexistente en una nueva pantalla, cuando el desarrollador importe dicho componente, entonces este debe integrarse y renderizar correctamente sin necesidad de duplicar o reescribir su código base. -Dado que se necesita modificar el diseño o comportamiento de un elemento (ej. cambiar el estilo de todos los botones primarios), cuando el desarrollador actualice el archivo principal del componente, entonces los cambios deben reflejarse automáticamente en todas las vistas de la plataforma donde esté implementado. -Dado que un componente reusable requiere mostrar información o acciones distintas según el contexto (como un modal con diferentes mensajes), cuando el desarrollador lo instancie en una vista, entonces debe poder pasarle propiedades o parámetros para adaptar su contenido sin alterar la estructura interna del componente original.	Criterios de Aceptación	-Dado que se ingresa un documento de identidad (por ejemplo, al registrar al dueño de un paciente), cuando el sistema requiera verificarlo, entonces debe invocar obligatoriamente la clase utilitaria para validar su formato y verificar que el dígito verificador sea el correcto. -Dado que se recibe una dirección de email en cualquier formulario, cuando el sistema procese los datos, entonces debe utilizar la función transversal encapsulada para confirmar que el correo cumple con una estructura válida. -Dado que se necesita conocer la edad a partir de una fecha de nacimiento (como la de una mascota), cuando el sistema ejecute la acción, entonces debe llamar al método utilitario centralizado que calcule y devuelva la edad exacta, garantizando que esta lógica no se reescriba en otros módulos.
--------------------------------	---	--------------------------------	--

Código RNF	RNF 13.3	Código RNF	RNF 14.1
Nombre RNF	Manejo Global e Interceptado de Excepciones	Nombre RNF	Paginación Obligatoria en Consultas de Listados
Tipo de Requerimiento	Reusabilidad	Tipo de Requerimiento	Eficiencia
Actores Relacionados	Veterinario, Tutor, Administrador	Actores Relacionados	Administrador, Veterinario
Descripción	Implementación de un mecanismo centralizado utilizando <code>@ControllerAdvice</code> y <code>@ExceptionHandler</code> en Spring Boot para capturar, procesar y formatear todas las excepciones generadas, garantizando una respuesta de error unificada.	Descripción	Implementación de un mecanismo de paginación estricto para todas las consultas que devuelvan múltiples registros utilizando Spring Data JPA, limitando la respuesta a un máximo de 20 elementos por página para optimizar memoria en MySQL.

Criterios de Aceptación	-Dado que ocurre un fallo inesperado en el servidor, cuando el cliente de la API reciba la respuesta, el sistema debe retornar un objeto JSON con un formato estándar predefinido gestionado por Spring Boot. -Dado que se busca un registro que no existe, cuando el cliente de la API realice la consulta, el sistema debe interceptar la EntityNotFoundException de JPA usando @ExceptionHandler y devolver un error HTTP 404 estructurado. -Dado que se envían datos incorrectos en el cuerpo de una petición, cuando el cliente intente procesarla, el sistema debe capturar las fallas de validación y responder con un código HTTP 400 estructurado en el formato estándar de la API.	Criterios de Aceptación	-Dado que una tabla contiene más de 20 registros, cuando el sistema reciba una solicitud de listado, debe resolverse a nivel de base de datos utilizando la interfaz Pageable de Spring Data JPA (pasando un objeto PageRequest). -Dado que el volumen de datos supera el límite establecido, cuando el sistema solicite una página específica, debe garantizar que la consulta SQL generada incluya sentencias LIMIT y OFFSET en MySQL, devolviendo la metadata útil para la navegación. -Dado que existen menos de 20 registros en el historial de un paciente, cuando el sistema ejecute la consulta, entonces debe retornar la totalidad de esos elementos en una única página de forma exitosa y sin arrojar errores.
--------------------------------	---	--------------------------------	---

Código RNF	RNF 14.2	Código RNF	RNF 14.3
Nombre RNF	Minificación y Compresión de Recursos Estáticos	Nombre RNF	Capa de Caché en Memoria para Datos Estáticos
Tipo de Requerimiento	Eficiencia	Tipo de Requerimiento	Eficiencia
Actores Relacionados	Tutor	Actores Relacionados	Administrador, Tutor
Descripción	Reducción del tamaño de los archivos del lado del cliente (HTML, CSS y JS) mediante técnicas de minificación y compresión, con el objetivo de minimizar el consumo de datos en dispositivos móviles durante el uso de la aplicación.	Descripción	Las consultas a datos estáticos o que rara vez cambian (como la lista de especialidades médicas o tipos de especies) deben almacenarse en una capa de caché en memoria para reducir las peticiones a la base de datos.

Criterios de Aceptación	-Dado que el código está listo para producción, cuando se construya la aplicación cliente, entonces todos los archivos estáticos (HTML, CSS, JS) deben generarse sin espacios, saltos de línea ni comentarios innecesarios. -Dado que los recursos están optimizados, cuando la aplicación cliente (o el servidor que la aloja) responda a una petición web, entonces debe entregar los archivos utilizando un algoritmo de compresión estándar (como Gzip o Brotli). -Dado que la compresión está activa, cuando el usuario acceda específicamente al flujo de agendamiento desde una red móvil, entonces el peso total de los recursos descargados en esa vista debe mantenerse al mínimo posible.	Criterios de Aceptación	-Dado que la información estática aún no ha sido solicitada, cuando el sistema realice la primera consulta de estos datos, entonces debe obtenerlos desde la base de datos y guardarlos inmediatamente en la capa de caché. -Dado que la lista de especialidades o especies ya fue consultada y almacenada previamente, cuando el sistema vuelva a requerir esta información, entonces debe recuperarla directamente desde el caché sin ejecutar una nueva petición a la base de datos. -Dado que se realiza una modificación o inserción en los datos estáticos originales, cuando el sistema detecte este cambio, entonces debe limpiar o actualizar el registro correspondiente en el caché para evitar entregar información obsoleta.
--------------------------------	---	--------------------------------	--

Código RNF	RNF 15.1	Código RNF	RNF 15.2
Nombre RNF	Cobertura Mínima de Pruebas (Testing)	Nombre RNF	Sistema de Logs Centralizado
Tipo de Requerimiento	Verificabilidad	Tipo de Requerimiento	Verificabilidad
Actores Relacionados	Administrador	Actores Relacionados	Administrador
Descripción	Garantizar al menos un 80% de cobertura de código mediante pruebas unitarias y de integración, asegurando el correcto funcionamiento de flujos críticos como el agendamiento, los pagos y el registro.	Descripción	Implementación de un registro centralizado que capture y almacene trazas detalladas de errores, accesos denegados y transacciones fallidas para permitir la auditoría y depuración del sistema.

Criterios de Aceptación	-Dado que el código está listo para integrarse, cuando el sistema CI/CD (o el desarrollador) ejecute el análisis de código, entonces el reporte debe confirmar una cobertura de pruebas igual o superior al 80%. -Dado que se implementan los módulos de agendamiento y confirmación de pago, cuando el desarrollador corra la suite de pruebas, entonces los tests unitarios y de integración deben cubrir y validar exitosamente todos los escenarios de estos flujos. -Dado que el registro es una función crítica, cuando el desarrollador verifique los tests, entonces debe existir evidencia de que la creación de usuarios está completamente validada por las pruebas automatizadas.	Criterios de Aceptación	-Dado que ocurre una falla interna o excepción en la aplicación, cuando el auditor revise el sistema centralizado, entonces debe poder visualizar la traza detallada del error registrado. -Dado que se rechaza un intento de acceso no autorizado, cuando el auditor consulte los logs, entonces el sistema debe mostrar los detalles específicos de ese evento de seguridad. -Dado que una operación o transacción no se completa con éxito, cuando el auditor examine los registros, entonces debe poder identificar la transacción fallida y sus datos asociados para su posterior revisión.
-------------------------	--	-------------------------	---

Código RNF	RNF 15.3
Nombre RNF	Endpoint de Estado del Sistema (Health Check)
Tipo de Requerimiento	Verificabilidad
Actores Relacionados	Administrador
Descripción	Implementación de un endpoint público mediante Spring Boot Actuator para permitir la verificación continua de la disponibilidad operativa de la API y su conexión con MySQL.
Criterios de Aceptación	-Dado que la API y MySQL funcionan correctamente, cuando una herramienta externa realiza una petición GET al endpoint /actuator/health, el sistema debe retornar un código de estado HTTP 200 (OK). -Dado que MySQL está accesible, cuando una herramienta externa consulta el endpoint /actuator/health, la respuesta de Actuator debe detallar e incluir explícitamente que el estado de la conexión a la base de datos es exitoso. -Dado que existe una caída en la base de datos o en la API, cuando una herramienta externa intenta verificar el estado mediante el endpoint, entonces el sistema debe retornar un código de error HTTP correspondiente (ej. 503 Service Unavailable) indicando la falla.

Fase 1.4: Gestión y Cierre de Planificación

1. Planificación y Cronograma de Trabajo

Tal como se definió en la metodología, el proyecto se ejecutará bajo el marco de trabajo **Scrum** con un plazo de 4 a 5 meses. El trabajo del equipo de desarrollo se dividirá en Sprints funcionales, garantizando entregas de valor continuo:

- Mes 1 - Sprint 1: Cimientos y Seguridad
 - Configuración inicial de la arquitectura Cloud en AWS y repositorios (GitHub/GitLab).
 - Desarrollo del backend base (Spring Boot) y conexión a la base de datos (MySQL).
 - Implementación de la autenticación segura (JWT) y creación de perfiles para Tutores y Médicos Veterinarios.
- Mes 2 - Sprint 2: Gestión y Agendamiento Base
 - Desarrollo del panel del tutor para la gestión de mascotas.
 - Integración del componente interactivo FullCalendar (React y React Native).
 - Desarrollo de la lógica de visualización de bloques horarios disponibles.
- Mes 3 - Sprint 3: Flujos Críticos y Pagos
 - Implementación de la regla de negocio crítica: Bloqueo temporal del horario por 10 minutos.
 - Integración con la pasarela de pagos externa (Flow) mediante comunicación segura HTTPS y formato JSON.
 - Pruebas de concurrencia y validación del flujo completo de reserva.
- Mes 4 - Sprint 4: Portal Médico y Despliegue (Release):
 - Desarrollo del Dashboard del Médico Veterinario.
 - Implementación de la Ficha Clínica Electrónica y subida de archivos adjuntos hacia AWS.
 - Empaquetado final (archivo `.jar`), despliegue automatizado en producción y entrega del producto final a los stakeholders.

2. Gestión de Riesgos

Se han identificado posibles obstáculos durante el ciclo de vida del proyecto y se han establecido medidas de mitigación:

- Caída del proveedor de pagos: Si la pasarela de pagos falla, el sistema cuenta con reversión automática, anulando el bloqueo del horario y notificando al usuario sin romper la plataforma.
- Pérdida de datos clínicos: Mitigado mediante la configuración de copias de seguridad automáticas e incrementales cada 12 horas en un volumen externo.

- Alta concurrencia: Si muchos tutores intentan agendar al mismo tiempo, la arquitectura escalable y el balanceador de carga distribuirán el tráfico para mantener el tiempo de respuesta por debajo de los 2 segundos.
- Riesgo de Aprobación en Tiendas (App/Play Store): Los tiempos de revisión de Apple y Google pueden retrasar el lanzamiento. Se mitigará enviando la aplicación a revisión semanas antes de la fecha límite del cuarto mes.

Etapa 2: Cimientos del Software

Fase 2.1: Diseño y Documentación de la Solución

Wireframes

1. Arquitectura de Software y Tecnologías

NodeVet adopta una arquitectura N-Tier clásica dividida en tres grandes capas: presentación, lógica de negocio y datos. Esta separación estructural permite que los clientes (Web y Móvil) operen de manera completamente independiente del backend.

- Capas de la arquitectura:
 - **Capa de Presentación:** Desarrollada con React (para la versión Web) y React Native junto a Expo (para la versión Móvil). Ambos clientes interactúan y consumen la API REST del backend mediante peticiones HTTP/JSON, utilizando Axios como cliente HTTP.
 - **Capa de Lógica de Negocio:** Construida sobre Spring Boot 3 e integrada con Spring Security. Es responsable de gestionar la autenticación mediante tokens JWT, ejecutar las reglas de negocio (tales como el registro de mascotas o la recuperación de contraseñas mediante OTP) y coordinar la comunicación con la base de datos a través de Spring Data JPA.
 - **Capa de Datos:** Implementada con MySQL 8 y gestionada mediante el ORM Hibernate/JPA. El esquema de datos se mantiene sincronizado de forma automática utilizando la propiedad “ddl-auto=update”.
 -
- Justificación tecnológica:
 - **Spring Boot:** Proporciona un ecosistema maduro y robusto con soporte nativo para la gestión de seguridad con JWT (Spring Security), persistencia de datos (JPA), envío de correos (Spring Mail) y generación de documentación automática (Swagger). Su uso reduce drásticamente la configuración boilerplate gracias a la implementación de anotaciones como “@RestController”, “@Service” y herramientas como Lombok.
 - **React Native / Expo:** Facilita el desarrollo al permitir escribir una única base de código en TypeScript que compila de forma nativa para iOS y Android. Expo optimiza el ciclo de

desarrollo ofreciendo recarga en caliente (hot reload) e integra librerías como expo-image-picker para la eficiente captura y gestión de fotos de las mascotas.

- **MySQL:** Seleccionado por ser un motor relacional robusto que garantiza el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). Esta elección responde directamente a la naturaleza altamente relacional del modelo de datos del proyecto (donde un Usuario está vinculado a un Tutor, y este a una Mascota).
- **Cloudinary:** Implementado como el único servicio externo dedicado al almacenamiento y optimización de imágenes en todo el proyecto. El backend está diseñado para recibir únicamente la URL de la imagen ya procesada en la nube, evitando así almacenar archivos binarios en la base de datos relacional y superando con éxito las limitaciones de tamaño de carga del servidor.

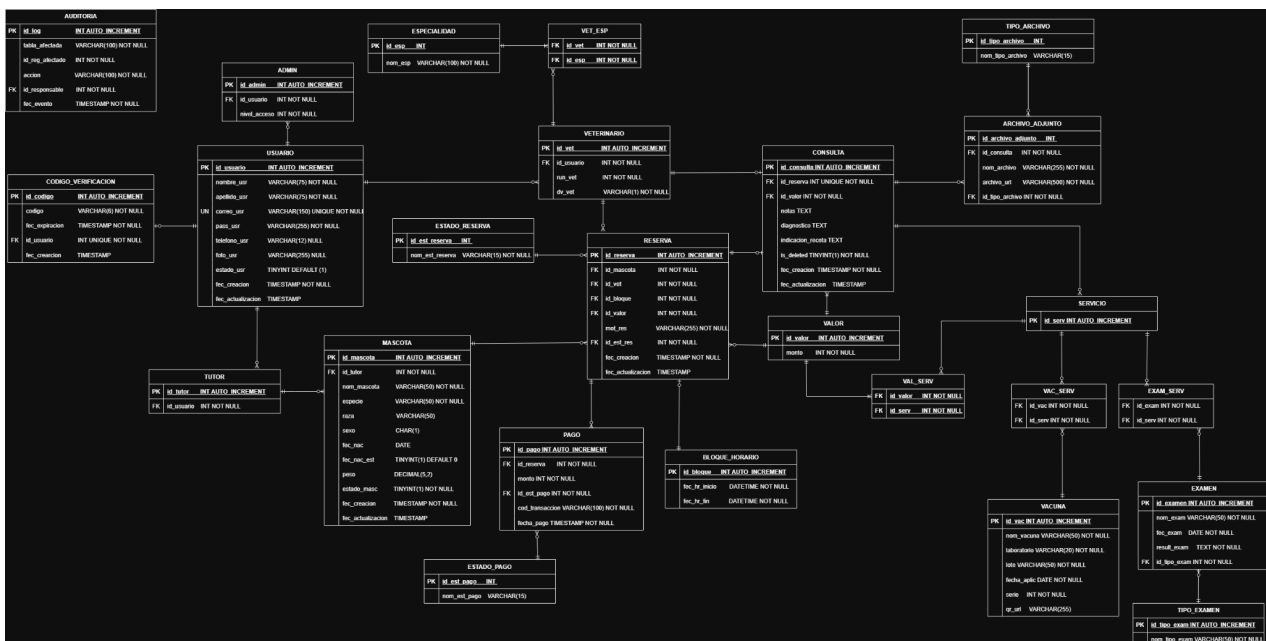
2. Diagramas Relevantes

- **Modelo Relacional de la Base de Datos**

Arquitectura de Datos Actual (Estructura Base)

El modelo de persistencia del proyecto NodeVet se sustenta en una base de datos relacional utilizando el motor de MySQL. Adjunto el modelo de datos:

[Modelo de la Base de Datos](#)



El diseño del esquema se encuentra normalizado y estructurado para soportar eficientemente los flujos operativos, clínicos y de concurrencia de la plataforma, destacando las siguientes implementaciones clave:

Gestión de Usuarios y Roles: El sistema emplea un modelo que centraliza la autenticación en la tabla de **USUARIO**, la cual se ramifica limpiamente hacia las tablas específicas de **TUTOR** y **VETERINARIO**. Esto facilita el manejo de perfiles, permisos y escalabilidad de nuevos roles en el futuro.

Gestión Clínica y Optimización de Almacenamiento: El núcleo del negocio se articula mediante la relación entre **MASCOTA**, **RESERVA** y los registros clínicos. Para el manejo de radiografías y exámenes, se diseñó la tabla **ARCHIVO_ADJUNTO**, la cual almacena exclusivamente la URL de referencia del documento, delegando el almacenamiento físico a un servicio externo (Cloudinary) para no saturar el servidor de la base de datos.

Trazabilidad y Auditoría: Para salvaguardar la integridad de los expedientes médicos, el modelo integra una tabla dedicada de **AUDITORIA**. Esta estructura permite registrar históricamente el identificador del usuario y la marca de tiempo exacta en que se modifica cualquier historial clínico.

Borrado Lógico (Soft Delete): El diseño de la base de datos prescinde del uso de comandos **DELETE** destructivos. En su lugar, el modelo incorpora atributos de estado (tales como **ESTADO_MASC** y **ESTADO_USR**) que permiten cambiar el estado de un registro a inactivo, el cual evita borrar el dato en cuestión (**MASCOTA** por ejemplo) y en su lugar lo deja deshabilitado.

Control de Concurrencia y Pagos: Para dar soporte a la prevención del ausentismo, la tabla **RESERVA** se vincula transaccionalmente con **BLOQUE_HORARIO** y la entidad de **PAGO**. Esta arquitectura de datos permite registrar un bloqueo temporal del horario durante 10 minutos, confirmando la cita de manera definitiva únicamente tras recibir la validación de la pasarela de pagos.

Modelo Futuro (Escalabilidad y Separación de Responsabilidades)

Considerando el potencial crecimiento transaccional de la plataforma, el diseño del backend y la base de datos han sido proyectados para escalar de manera horizontal y vertical.

- Patrón de Lectura y Escritura:** El sistema se adaptará en el futuro para separar las operaciones de la base de datos. Utilizando Spring Data JPA, se configurarán múltiples orígenes de datos hacia MySQL: una instancia principal (Master) configurada exclusivamente para operaciones de escritura y modificación (**INSERT**, **UPDATE**, **DELETE**), y una o más instancias de réplica dedicadas únicamente a consultas de lectura (**SELECT**).
- Prevención de Cuellos de Botella:** Esta separación garantizará que las operaciones de consulta masiva (como la carga del historial clínico o la visualización de la agenda) no degraden el rendimiento transaccional de los flujos críticos de negocio, como el bloqueo de citas y la confirmación de pagos.

- **Modelo 4+1:** [PDF](#)

- **Vista Lógica - Diagrama de clases:**

Jerarquía de usuarios:

El diagrama de clases representa la estructura estática del dominio de NodeVet, modelando las entidades del sistema, sus atributos, responsabilidades y las relaciones que las vinculan.

La clase **USUARIO** se declara como `<<Abstract>>`, lo que significa que no puede ser instanciada directamente, ya que su propósito es centralizar los atributos comunes a todos los actores del sistema (id_usuario, rut, nombre, apellido, correo, password y estado). Todos estos atributos usan la visibilidad protegida (`#`), lo que permite que las subclases los hereden sin exponerlos públicamente.

De **USUARIO** heredan tres subclases concretas:

- **Administrador:** Representa al gestor del sistema, Su único comportamiento diferenciador es `“gestionarSistema()”` que encapsula las operaciones de configuración y supervisión de la plataforma.
- **Tutor:** Corresponde al dueño de la mascota. Agrega el atributo propio `“id_tutor”` (privado `-`), y el método `“agendarCita()”`, que es el punto de entrada al flujo de reserva.
- **Veterinario:** Es el profesional que atiende las consultas. Agrega `“id_vet”` y el método `“registrarEvolución()”`, a través del cual documenta el avance clínico del paciente.

Esta jerarquía aplica el principio de generalización o especialización, lo que significa que los atributos e identificación se definen una sola vez en la clase padre, evitando redundancia.

Gestión de mascotas:

La clase **MASCOTA** concentra la ficha de identificación del paciente animal. Sus atributos cubren los datos clínicos y de registro necesarios para la operación veterinaria: especie, raza, fecha de nacimiento, peso, y una URL de imagen almacenada en la nube (`“imagenUrl”`). El atributo `“estado”` permite implementar soft delete, conservando el historial sin eliminar físicamente el registro.

La relación entre Tutor y Mascota es una asociación dirigida de multiplicidad 1-N, un tutor puede registrar una o varias mascotas, pero cada mascota pertenece a exactamente un tutor. La navegabilidad está etiquetada con `“registra”`, describiendo la dirección de la acción.

Núcleo del sistema:

RESERVA es la clase central del modelo. Actúa como nodo de convergencia entre los actores y los recursos del sistema, acumulando las siguientes relaciones:

- Tutor → Reserva (1-N) con etiqueta “realiza”: un tutor puede tener múltiples reservas a lo largo del tiempo.
- Mascota → Reserva (N-N) con etiqueta “asignada a”: una reserva puede involucrar a una mascota concreta, y una mascota puede tener múltiples reservas históricas.
- Veterinario → Reserva (1-N) con etiqueta “atiende”: un veterinario puede atender múltiples reservas; cada reserva es atendida por un único veterinario.
- Reserva → BloqueHorario (N-1) con etiqueta “ocupa”: varias reservas pueden referir al mismo bloque horario en el tiempo, pero una reserva concreta ocupa un único bloque. El atributo disponible en “BloqueHorario” refleja el estado de ocupación del slot.
- Reserva → Pago (1-1) con etiqueta “requiere”: toda reserva exige exactamente un pago asociado, modelando la obligatoriedad del cobro como condición del agendamiento.
- Reserva → Consulta (1-0-1) con etiqueta “deriva en”: una reserva puede o no derivar en una consulta clínica. La multiplicidad 0-1 es deliberada, ya que una reserva cancelada o no presentada no genera ficha clínica.

Ficha Clínica:

La clase **CONSULTA** lleva el estereotipo <<Ficha Clínica>>, indicando que cumple el rol semántico de registro médico oficial de la atención. Sus atributos recogen la fecha de la consulta y el diagnóstico emitido por el veterinario. El método “finalizarAtencion()” encapsula el cierre formal de la ficha.

CONSULTA mantiene tres relaciones hacia clases complementarias:

- Agregación “o—” con **SERVICIO** (1-N) etiquetada “incluye”: una consulta puede incluir uno o más servicios veterinarios (vacuna adicional, cirugía menor, etc.). Se modela como agregación porque los servicios existen con independencia de una consulta específica; son un catálogo reutilizable.
- Composición “*--” con **ArchivoAdjunto** (1-N) etiquetada “contiene”: una consulta puede contener múltiples archivos adjuntos (radiografías, exámenes de laboratorio, imágenes). Se modela como composición porque los archivos no tienen existencia significativa fuera del contexto de su consulta; si la consulta desaparece, los archivos también. El método “subirAWS()” indica que estos archivos persisten en un servicio de almacenamiento en la nube (Amazon S3).

- Asociación con Vacuna (1-N) etiquetada “aplica”: durante una consulta pueden aplicarse una o varias vacunas, registrando el historial de vacunación del paciente.

Decisiones de diseño relevantes:

Uso de <<Abstract>> en **USUARIO** garantiza que nunca se cree un usuario sin rol definido en el sistema, reforzando la integridad del modelo de autenticación.

Soft delete mediante atributo “estado” en **Usuario** y **Mascota** permite conservar la trazabilidad histórica de registros sin comprometer la integridad referencial de la base de datos, alineándose con el requisito no funcional RNF 11.1 del proyecto.

Distinción composición vs. agregación en las relaciones de **Consulta** refleja una decisión de diseño consciente: los **Servicio** son entidades del catálogo del negocio (independientes), mientras que los **ArchivoAdjunto** son documentos clínicos cuyo ciclo de vida está atado a la consulta que los generó.

La multiplicidad 0-1 en “Reserva → Consulta” modela correctamente la realidad operativa: no toda cita agendada se concreta en atención clínica, y el sistema debe ser capaz de representar ese estado sin forzar la creación de una ficha vacía.

○ **Vista de despliegue - Diagrama de componentes:**

El diagrama representa la arquitectura física y lógica de despliegue de NodeVet, describiendo cómo los distintos componentes del sistema se organizan en capas, cómo se comunican entre sí y qué servicios externos integran. La estructura sigue un patrón N-Tier distribuido en cuatro grandes zonas.

1. Capa de presentación - Frontend

Es el punto de entrada del sistema para los usuarios finales. Contiene dos clientes independientes que coexisten:

- Aplicación Web (React): interfaz de escritorio/navegador orientada principalmente a la gestión interna (veterinarios, administradores). Permite visualizar el calendario de citas, gestionar fichas clínicas y administrar el sistema.
- App Móvil Nativa (React Native + Expo): aplicación instalable en dispositivos Android e iOS, orientada al tutor de mascotas. Permite registrar mascotas, agendar citas y realizar seguimiento desde el teléfono.

Ambos clientes se comunican con el backend exclusivamente mediante HTTPS, garantizando el cifrado del tráfico en tránsito y cumpliendo con el requisito de seguridad en las comunicaciones.

2. Infraestructura Cloud - AWS

Toda la lógica de servidor se despliega dentro del ecosistema de Amazon Web Services, lo que proporciona escalabilidad, alta disponibilidad y gestión centralizada de la infraestructura.

Balanceador de carga:

Es el primer componente que recibe el tráfico entrante desde ambos frontends. Su función es distribuir las peticiones entre las instancias disponibles del backend, evitando sobrecargar un único servidor y habilitando la escalabilidad horizontal del sistema. Actúa como punto de entrada único (single entry point) a la infraestructura cloud.

Backend — Java + Spring Boot:

Es el núcleo del sistema. Implementa una arquitectura N-Tier con separación estricta de responsabilidades en tres capas internas:

- API RESTful: capa de entrada que expone los endpoints del sistema. Recibe las peticiones HTTP provenientes del balanceador, las enruta hacia los controladores correspondientes y devuelve las respuestas en formato JSON.
- Módulo de Seguridad (JWT): intercepta todas las peticiones antes de que lleguen a los controladores. Valida la firma y expiración del token JWT incluido en el header Authorization: Bearer. Si el token es inválido o está expirado, rechaza la petición con un 401 Unauthorized sin llegar a ejecutar lógica de negocio. Este módulo está representado con borde diferenciado en el diagrama, indicando su carácter transversal.
- Estructura N-Tier interna:
 - Capa de Controladores:

Recibe las peticiones ya autenticadas, extrae los parámetros y delega el procesamiento a la capa de servicios. No contiene lógica de negocio.
 - Capa de Servicios:

implementa las reglas de negocio del sistema (validaciones, control de concurrencia, generación de tokens OTP, encolado de correos, etc.). Es la capa más relevante desde el punto de vista funcional.
 - Capa de Repositorios:

Gestiona el acceso a los datos mediante JPA/Hibernate. Abstrae las consultas SQL y provee métodos tipados para operar sobre las entidades del dominio.

3. Capa de datos y almacenamiento

Recibe solicitudes desde la capa de repositorios del backend a través de dos canales diferenciados:

- Base de Datos Relacional (MySQL): almacena toda la información estructurada del sistema: usuarios, mascotas, reservas, bloques horarios, consultas, pagos y vacunas. La comunicación se realiza mediante Consultas SQL directas gestionadas por el ORM de Spring Data JPA.
- Almacenamiento de Archivos (Cloudinary): Servicio de almacenamiento de objetos en la nube donde se persisten los archivos binarios asociados a las consultas clínicas: imágenes, radiografías y PDFs de confirmación de citas. La comunicación desde el backend a Cloudinary se realiza mediante el SDK de Cloudinary, etiquetada en el diagrama como Envío de Imágenes/PDF. Esta decisión arquitectónica desacopla el almacenamiento de archivos del servidor de aplicaciones, evitando que el backend gestione directamente archivos binarios pesados.

4. Servicios y proveedores externos

Componentes de terceros integrados mediante APIs sobre canales seguros:

- Pasarela de Pagos (Flow): integración mediante JSON sobre HTTPS. El backend genera una orden de pago firmada y la envía a la pasarela; el resultado del cobro retorna al backend mediante un callback webhook, desacoplando el flujo de pago de la latencia de la transacción bancaria.
- Proveedor de Correos (SMTP Gmail): integración mediante Conexión Segura (TLS/SSL sobre SMTP). El backend encola los correos de confirmación de citas y notificaciones, que son procesados y despachados a través del servidor SMTP de Gmail. El encolado asíncrono garantiza que el tiempo de envío del correo no impacte en el tiempo de respuesta de la API.

Además, la separación frontend/backend desacoplada mediante API RESTful permite que ambos clientes (web y móvil) consuman los mismos endpoints sin duplicar lógica de servidor, y que en el futuro se incorporen nuevos clientes sin modificar el backend. También, JWT como mecanismo de autenticación stateless elimina la necesidad de sesiones en servidor, lo que es especialmente relevante en arquitecturas cloud donde múltiples instancias del backend pueden estar corriendo en paralelo detrás del balanceador de carga.

○ **Vista de procesos - Diagrama de actividad:**

■ **Agendamiento:**

El diagrama modela el flujo de actividades del proceso de agendamiento online en NodeVet, describiendo la secuencia de acciones, decisiones y responsabilidades distribuidas entre los cinco carriles del sistema. A diferencia de un diagrama de secuencia (que se centra en la interacción entre objetos), este diagrama de actividad enfatiza el flujo de control, las condiciones de bifurcación y los estados transitorios del sistema durante una operación crítica.

- **Etapas 1 — Selección de mascota y visualización del calendario:**
 - El flujo arranca cuando el Tutor selecciona la mascota y el motivo de la consulta. Simultáneamente, el Frontend realiza una solicitud de disponibilidad hacia una API externa que provee los datos del calendario. Con esta información, renderiza el componente FullCalendar, mostrando visualmente los bloques horarios disponibles. Este paso garantiza que el tutor solo pueda interactuar con slots efectivamente libres, reduciendo la probabilidad de conflictos en la siguiente etapa.
- **Etapas 2 — Selección del bloque y control de concurrencia:**
 - Cuando el Tutor selecciona un bloque horario, el Backend ejecuta un intento de lock sobre ese registro en la base de datos (RNF 7.1 — Control de Concurrencia). Este mecanismo resuelve el problema de que dos tutores intenten reservar el mismo bloque simultáneamente.

El resultado del intento de lock genera la primera bifurcación del flujo:

- **Éxito (Libre):** el bloque no está tomado. El backend actualiza su estado a EN_PROCESO y lanza un timer asíncrono de 10 minutos que corre en segundo plano (Monitoreo en Segundo Plano). El flujo continúa hacia el proceso de pago.
 - **Conflicto (Ocupado):** otro usuario ya tomó ese bloque. El Frontend muestra una alerta de horario ocupado al tutor para que seleccione otro slot. El flujo retrocede a la selección del calendario sin haber modificado ningún estado.
- **Etapas 3 — Generación de la orden y proceso de pago:**
 - Con el bloque en estado EN_PROCESO, el Backend genera una orden de pago en formato JSON sobre HTTPS (RNF 6.2), cumpliendo el estándar de comunicación segura requerido por las pasarelas de pago. Esta orden es enviada a la Interfaz de Pago Externa (Flow), que presenta al tutor el formulario para ingresar sus datos bancarios. La pasarela procesa la transacción de forma

autónoma y comunica el resultado al backend mediante un callback webhook.

- Etapa 4 — Validación del resultado y timer asíncrono:
 - Esta es la etapa más compleja del flujo, con dos condiciones evaluadas en paralelo:
 - Condición A — ¿Timer asíncrono > 10 minutos?
El proceso de pago tiene un límite de tiempo. Si el tutor tarda más de 10 minutos (por abandono, lentitud bancaria o inactividad), el timer dispara automáticamente una reversión a DISPONIBLE mediante soft delete (RNF 11.1): el estado del bloque vuelve a estar libre sin eliminar el registro histórico. El Frontend notifica al tutor que el tiempo expiró y muestra el modal de error/cancelación. El flujo termina sin reserva.
 - Condición B — Resultado del webhook.
Si el pago se resuelve dentro del tiempo límite, el Backend valida el callback recibido y evalúa:
 - HTTP 200 OK (pago exitoso): el backend ejecuta una operación SELECT + validate + UPDATE atómica en MySQL para actualizar el bloque a estado AGENDADA y persistir el estado final de la reserva. Acto seguido, encola un correo de confirmación SMTP (RNF 10.2) para notificar al tutor de forma asíncrona. El Frontend muestra la confirmación de la cita junto con un PDF descargable. Fin exitoso.
 - Error / Tutor Cancela: si la transacción falla o el tutor cancela el pago, el bloque es revertido a DISPONIBLE (soft delete RNF 11.1) y el Frontend muestra el modal de error/cancelación. Fin sin reserva.

- **Añadir mascota:**

El diagrama modela el flujo de registro de una nueva mascota, organizado en cuatro carriles: Tutor de Mascota, Capa de Presentación (Frontend), Capa de Aplicación (Backend Spring Boot) y Capa de Datos (MySQL).

- Flujo del Tutor: el actor inicia seleccionando la opción "Agregar Mascota" en la interfaz. Completa el formulario con los datos de la mascota (nombre, especie, raza, sexo, edad y peso) y presiona "Guardar". Si la validación del frontend detecta errores, el formulario le es devuelto con una alerta visual y el tutor debe corregirlo antes de continuar.
- Flujo del Frontend: al recibir la acción de guardar, el frontend ejecuta una validación visual sobre el formulario, evaluando si existen campos vacíos o con valores negativos. Si los datos son inválidos, muestra una alerta visual inmediata sin realizar ninguna petición al servidor, reduciendo la carga

innecesaria sobre la red y el backend. Si los datos son válidos, empaqueta la información en formato JSON y envía un POST `/api/auth/pets` incluyendo el header JWT en la cabecera de autorización.

- **Flujo del Backend:** el backend ejecuta dos niveles de validación secuenciales. Primero valida el token JWT y extrae el ID del Tutor contenido en el payload del token; si el token es inválido o ha expirado, retorna inmediatamente un HTTP 401 Unauthorized sin procesar la petición. Si el token es válido, ejecuta una Validación de Integridad vía Clase Utilitaria, evaluando si los datos son biológicamente coherentes (por ejemplo, que la edad no supere los 30 años). Si los datos son ilógicos, devuelve un HTTP 400 Bad Request. Si los datos son íntegros, construye la entidad Mascota y la asocia al ID del Tutor extraído del token, garantizando que cada mascota quede vinculada a su dueño sin depender de un parámetro manipulable por el cliente. Finalmente, delega la persistencia a la capa de datos y retorna HTTP 201 Created junto con el DTO de la mascota creada.
- **Flujo de la Base de Datos:** ejecuta la sentencia INSERT INTO mascotas (...) con los datos de la entidad construida, persistiendo el nuevo registro. El resultado retorna al backend, que construye la respuesta final hacia el frontend.
- **Flujo de retorno:** el frontend recibe la respuesta del backend. Si es exitosa, muestra un mensaje de éxito y actualiza la lista de mascotas. Si es un error de servidor o autorización, muestra el mensaje de error correspondiente. Ambos caminos llevan al fin del flujo.

○ **Ver mascota**

Este diagrama modela dos flujos de lectura encadenados: el listado de mascotas del tutor y la visualización del historial clínico de una mascota específica. Se organiza en los mismos cuatro carriles que el diagrama de creación.

- **Flujo del Tutor:** el actor ingresa a la sección "Mis Mascotas", visualiza el panel de tarjetas con sus mascotas asociadas y puede seleccionar una de ellas para acceder al historial clínico en modo solo lectura, que presenta un timeline cronológico con vacunas, diagnósticos y recetas.
- **Flujo del Frontend — Listado:** al ingresar a la sección, el frontend envía un GET al backend para obtener las mascotas del tutor autenticado. Con la respuesta, renderiza las tarjetas de mascotas en pantalla. Cuando el tutor selecciona una mascota, el frontend envía una segunda petición GET para obtener la información detallada de esa mascota específica.
- **Flujo del Backend — Primera validación (listado):** el backend valida el token JWT y extrae el ID del Tutor. Si el token es válido, ejecuta un SELECT en

MySQL filtrado por el ID del dueño y el estado activo de la mascota, garantizando que el tutor solo pueda ver sus propias mascotas.

- Flujo del Backend — Segunda validación (detalle con seguridad anti-IDOR): para el acceso al detalle de una mascota específica, el backend ejecuta una validación de propiedad que verifica que la mascota solicitada pertenece efectivamente al tutor que realiza la petición. Este mecanismo previene ataques de tipo IDOR (Insecure Direct Object Reference), donde un usuario malintencionado podría modificar el identificador de la mascota en la URL para acceder a datos de otro tutor. Si la verificación falla, retorna HTTP 403 Forbidden. Si está autorizado, construye un PageRequest con la paginación correspondiente y ejecuta el SELECT del historial clínico de la mascota.
- Control de rendimiento: el diagrama incluye explícitamente una bifurcación que evalúa si el tiempo de respuesta de la consulta es inferior a 2 segundos. Si se cumple el umbral (Rendimiento Óptimo), el frontend renderiza el timeline cronológico. Si la consulta supera el tiempo esperado (Degradación de Servicio), se muestra una pantalla de error informativa. Este nodo de decisión documenta formalmente un requisito no funcional de rendimiento dentro del flujo operativo.
- Flujo de errores: token inválido, no autorización por IDOR o degradación de servicio convergen en el mismo nodo de error del frontend, que muestra la pantalla de error o sesión expirada con los códigos HTTP 401 o HTTP 403 según corresponda.

○ **Editar mascota**

El diagrama modela la edición de los datos permitidos de una mascota, incorporando controles de seguridad, validaciones biológicas y auditoría de cambios. Se organiza en los cuatro carriles estándar.

- Flujo del Tutor: el actor selecciona la mascota y presiona "Editar mascota". Modifica únicamente los campos habilitados (peso, fecha de nacimiento, raza, nombre e imagen) y presiona "Guardar Cambios". Si el backend rechaza la operación, recibe una notificación de error controlado y visualiza un modal de error específico que indica la causa (datos inválidos o no autorizado). Si la operación es exitosa, ve el mensaje "Perfil Actualizado" y el dashboard se refresca automáticamente.
- Flujo del Frontend: al iniciar la edición, solicita los datos actuales de la mascota a la API para poblar el formulario. Renderiza el formulario con algunos inputs bloqueados (aquellos correspondientes al historial clínico que no pueden ser modificados por el tutor), dejando editables solo los campos autorizados. Antes de enviar la petición PUT, ejecuta una validación local de

campos vacíos o no permitidos, como primera barrera de protección. Si supera la validación, envía la petición PUT al backend.

- Flujo del Backend — Validación de autorización: el backend intercepta la petición validando el JWT y verificando que la mascota pertenece al tutor que realiza la solicitud. Si el token es inválido o el tutor no es el dueño, retorna HTTP 403 Forbidden o HTTP 400 Bad Request directamente. Este control es análogo al anti-IDOR del flujo de lectura.
- Flujo del Backend — Validación biológica (RNF 7.3): si el tutor está autorizado, el backend ejecuta una validación de rangos biológicos vía Clase Utilitaria (RNF 13.2), verificando que los valores ingresados sean biológicamente posibles (por ejemplo, que el peso no sea negativo o igual a cero). Si los valores son ilógicos, retorna el código HTTP de error correspondiente. Esta validación es una segunda línea de defensa que complementa la validación visual del frontend.
- Flujo del Backend — Mapeo, auditoría y persistencia: si los datos son íntegros y validados, el backend mapea los cambios al objeto entidad y ejecuta el registro de auditoría (RNF 7.2), capturando qué campos fueron modificados, por quién y en qué momento. Este registro de trazabilidad es fundamental para el cumplimiento normativo del sistema. A continuación, el repositorio ejecuta el UPDATE mascotas SET nombre=?, peso=?, edad=? WHERE id=? en MySQL y retorna la confirmación al backend, que construye la respuesta HTTP 200 OK con el DTO actualizado.

○ **Eliminar mascota**

El diagrama modela la eliminación lógica de una mascota, incorporando una confirmación explícita del usuario, auditoría automática y soft delete. Se organiza en los cuatro carriles estándar.

- Flujo del Tutor: el actor presiona el botón "Eliminar mascota". El sistema le presenta una advertencia visual de eliminación y espera su confirmación explícita. Solo si confirma la acción presionando "Sí, eliminar" se desencadena el flujo de eliminación. Este paso de confirmación es una salvaguarda de UX que previene eliminaciones accidentales.
- Flujo del Frontend: al detectar el clic en el botón de eliminación, el frontend intercepta el evento y renderiza un Modal de Confirmación antes de ejecutar cualquier operación, delegando la decisión final al usuario. Si el tutor confirma, envía la petición DELETE al backend con el JWT en el header. Al recibir la confirmación del backend, remueve visualmente la tarjeta de la mascota del listado y redirige al tutor a la sección "Mis Mascotas". Si recibe un error, muestra el Modal de Error y cancela el flujo.

- Flujo del Backend — Validación de autorización: el backend intercepta la petición, valida el JWT y verifica que la mascota a eliminar pertenece al tutor solicitante. Si el token es inválido o el tutor no es el dueño, retorna HTTP 403 Forbidden o HTTP 404 Not Found según corresponda, sin ejecutar ninguna modificación en la base de datos.
- Flujo del Backend — Auditoría y eliminación lógica: si el tutor está autorizado, el backend ejecuta la auditoría capturando el timestamp y el usuario que realizó la operación, dejando trazabilidad del evento de eliminación. A continuación, en lugar de ejecutar un DELETE físico, ejecuta un UPDATE mascota en MySQL que modifica el campo estadoMasc a inactivo (soft delete), preservando el registro histórico de la mascota y toda su ficha clínica asociada en la base de datos. Este mecanismo garantiza la integridad referencial del historial médico y cumple con los requisitos de trazabilidad del sistema. Finalmente, construye y retorna HTTP 200 OK o HTTP 204 No Content al frontend confirmando la operación.

■ Inicio Sesión

El diagrama modela el proceso de autenticación del sistema, organizado en cuatro carriles: Tutor de Mascota, Capa de Presentación (Frontend), Capa de Aplicación (Backend) y Capa de Datos (MySQL).

- Flujo del Tutor: El actor ingresa a la pantalla de login, escribe su correo y contraseña, y presiona "Ingresar". Dependiendo del resultado del proceso, verá un mensaje de error por credenciales inválidas o será redirigido al Dashboard Principal.
- Flujo del Frontend: Al presionar "Ingresar", el frontend empaqueta las credenciales y envía una petición POST /api/auth/login al backend. No ejecuta validación previa de formato en este flujo, delegando toda la lógica de autenticación al servidor. Si el backend retorna éxito, el frontend guarda el JWT en LocalStorage (web) o SecureStore (móvil) según la plataforma, y redirige al Dashboard Principal. Si recibe un error, muestra el mensaje "Credenciales Inválidas" al usuario. La distinción entre LocalStorage y SecureStore es relevante: SecureStore de Expo cifra el token en el dispositivo móvil, mientras que LocalStorage es la alternativa disponible en el navegador web.
- Flujo del Backend — Tres niveles de validación secuencial:
- El backend implementa una cadena de validaciones que se interrumpe en el primer fallo encontrado, retornando siempre HTTP 401 Unauthorized sin revelar cuál validación específica falló. Esta decisión de diseño es una buena

práctica de seguridad, ya que evita que un atacante pueda distinguir si el correo existe en el sistema o si simplemente la contraseña es incorrecta.

- Nivel 1 — Búsqueda del usuario: Extrae las credenciales del payload JSON y ejecuta la consulta en MySQL. Si el usuario no existe, retorna HTTP 401 directamente.
- Nivel 2 — Verificación de contraseña: Si el usuario existe, compara la contraseña ingresada en texto plano contra el hash almacenado usando Bcrypt. Este algoritmo de hashing es resistente a ataques de fuerza bruta por su factor de costo configurable. Si la contraseña no coincide, retorna HTTP 401.
- Nivel 3 — Verificación de estado de cuenta: Si la contraseña es correcta, verifica que la cuenta esté activa. Si la cuenta está inhabilitada o bloqueada (por el administrador o por el propio tutor mediante borrado lógico o soft delete), retorna HTTP 401 sin generar token.

Generación del JWT: Solo si las tres validaciones son superadas, el backend genera un Token JWT firmado con la clave secreta (Secret Key), incluyendo en el payload el ID del usuario y su rol. Este token será utilizado en todas las peticiones subsiguientes para autenticar al usuario sin necesidad de consultar la base de datos en cada request. Finalmente, retorna HTTP 200 OK con el token.

○ **Registro de cuenta**

El diagrama modela la creación de una nueva cuenta de tutor, con validaciones en dos capas: frontend y backend. Se organiza en los cuatro carriles estándar.

- Flujo del Tutor: el actor ingresa a la pantalla de registro, completa el formulario con RUT, nombre, correo y contraseña, y presiona "Crear Cuenta". Si hay errores de formato, el formulario le es devuelto con alertas. Si el registro es exitoso, recibe un mensaje de éxito y es redirigido al Login.
- Flujo del Frontend: renderiza el formulario de registro y ejecuta una Validación Local de formato sobre tres campos antes de enviar cualquier petición al servidor: formato de correo (expresión regular), formato de RUT (dígito verificador) y complejidad de contraseña (mínimo de caracteres, mayúsculas y caracteres especiales). Si alguno es inválido, muestra las alertas de formato correspondientes sin realizar petición de red. Si todos los formatos son válidos, envía POST /api/auth/auth/register con el payload JSON.
- Flujo del Backend — Detección de duplicados:
- El backend recibe el payload y ejecuta inmediatamente una consulta de unicidad en MySQL: `SELECT count(*) WHERE correo = X OR rut = Y`. Esta

consulta verifica en una sola operación atómica si ya existe un usuario registrado con ese correo o con ese RUT. Si existe un duplicado, retorna HTTP 409 Conflict, que el frontend presenta como mensaje de error (por ejemplo, "El correo ya existe"). La elección de HTTP 409 es semánticamente precisa: indica un conflicto con el estado actual del recurso, diferenciándose del HTTP 400 que señalaría un error en el formato del request.

- Construcción del usuario: si no hay duplicados, el backend ejecuta dos operaciones en secuencia. Primero hashea la contraseña con Bcrypt antes de almacenarla, garantizando que nunca se persista texto plano en la base de datos. Luego construye la entidad Usuario asignándole el rol 'Tutor' automáticamente, sin que el cliente pueda influir en el rol asignado. Esta asignación automática del rol es una medida de seguridad: impide que un usuario pueda autoasignarse privilegios de veterinario o administrador mediante manipulación del payload.
- Persistencia: el repositorio ejecuta INSERT INTO usuarios (rut, correo, password_hash...) en MySQL y el backend retorna HTTP 201 Created, que el frontend interpreta para mostrar el mensaje de éxito y redirigir al Login.

○ **Recuperar contraseña**

Este diagrama es el más complejo de los tres e involucra cinco carriles: Tutor de Mascota, Frontend, Backend, Servicio Externo (AWS SES / Gmail) y Capa de Datos (MySQL). Modela un flujo de recuperación de contraseña organizado en las fases de solicitud, verificación y restablecimiento.

- Fase 1 — Solicitud del código de recuperación
 - Flujo del Tutor: El actor presiona "Olvidé mi contraseña", ingresa su correo registrado y presiona "Enviar Enlace" o "Enviar Código".
 - Flujo del Frontend: Envía una petición POST /api/auth/recuperacion con el correo ingresado. En este paso inicial, el frontend muestra siempre un mensaje genérico: "Si el correo existe, recibirá instrucciones". Esta decisión de diseño funciona como una medida de seguridad efectiva contra la enumeración de usuarios en la primera etapa, impidiendo que un atacante deduzca si una cuenta existe o no en base a respuestas diferenciadas del servidor.
 - Flujo del Backend (Validación y generación del token): El backend consulta en MySQL si el correo existe. Si es así, ejecuta de forma paralela la generación de un Token Temporal con una expiración corta de 15 minutos vinculado al usuario y almacena el registro con su respectiva fecha de expiración en la base de datos, limitando

significativamente la ventana de tiempo operativa ante posibles interceptaciones.

- Flujo del Servicio Externo: El backend compila una plantilla HTML con el código o enlace de recuperación y la transfiere al servicio de correo (AWS SES o Gmail SMTP). El servicio externo encola y despacha el correo seguro al tutor, garantizando legibilidad y reduciendo la probabilidad de que sea clasificado como spam.

■ Fase 2 — Verificación del código o token

- Flujo del Frontend: Una vez que el usuario recibe el código o accede al enlace, el sistema procesa o solicita la validación enviando una petición POST `/api/auth/verificacion`.
- Flujo del Backend (Matiz de Seguridad): En este punto medio del flujo, la protección global contra la enumeración de usuarios presenta una vulnerabilidad parcial. A diferencia del hermetismo logrado en la Fase 1, el endpoint `/api/auth/verificacion` lanza una excepción específica 404 Not Found ("Usuario no encontrado") si el correo ingresado no existe en el sistema, lo cual difiere del comportamiento de error devuelto cuando el correo sí existe pero el código es incorrecto. Este diseño revela indirectamente la existencia de la cuenta, quedando registrado en la arquitectura técnica como un hallazgo y una mejora de seguridad pendiente por resolver.

■ Fase 3 — Restablecimiento de la contraseña

- Flujo del Tutor: El actor ingresa su nueva contraseña en el formulario correspondiente y presiona "Actualizar Contraseña".
- Flujo del Frontend: Envía una petición empleando el método HTTP PUT `/api/auth/password`, incluyendo en el cuerpo el token validado y la nueva credencial. Si la operación es exitosa, muestra un mensaje de confirmación y redirige al Login; si el backend responde con error, se notifica que el token es inválido o ha expirado.
- Flujo del Backend (Validación y actualización): El backend evalúa la autenticidad de la firma del token y comprueba que se encuentre dentro del rango de los 15 minutos de vigencia. Si el token es válido, se ejecutan tres operaciones atómicas y encadenadas:
 - Aplica el algoritmo de hashing Bcrypt a la nueva contraseña para asegurar resistencia contra ataques de fuerza bruta.

- Invalida inmediatamente el token utilizado en la base de datos, marcándolo como consumido para garantizar que los enlaces sean de un solo uso e impedir que un tercero intente reutilizarlo de forma maliciosa.
- Ejecuta un UPDATE en la tabla de usuarios en MySQL para persistir la nueva credencial y confirma el éxito de la transacción al frontend.

○ **Vista física - Diagrama de despliegue:**

El diagrama de despliegue representa la topología física de ejecución de NodeVet, mostrando cómo los artefactos del sistema se distribuyen sobre nodos de hardware y servicios de infraestructura reales. A diferencia del diagrama de componentes (que describe qué hace cada parte), este diagrama responde a dónde se ejecuta cada artefacto y cómo se comunican los nodos entre sí a nivel de red y protocolo.

El diagrama se organiza en cuatro zonas físicas claramente delimitadas.

1. Nodos cliente - Capa de presentación

Representa los dispositivos físicos de los usuarios finales. Son nodos externos a la infraestructura controlada por el sistema y se comunican con el backend únicamente a través de Internet:

- App Web «React JS»: se ejecuta en el navegador del usuario (PC o laptop). Consume la API mediante peticiones HTTPS (REST/JSON), lo que garantiza el cifrado de toda la comunicación y la independencia del cliente respecto al servidor.
- App Móvil Nativa «React Native / Expo»: se ejecuta como aplicación instalada en el dispositivo móvil del tutor (Android o iOS). Igualmente se comunica mediante HTTPS (REST/JSON), usando el mismo canal seguro que la app web.

Ambos clientes son thin clients: no contienen lógica de negocio ni acceso a datos; toda la inteligencia del sistema reside en el backend.

2. Infraestructura Cloud — Amazon Web Services

Todo el backend del sistema se despliega dentro de AWS, lo que proporciona alta disponibilidad, escalabilidad gestionada y aislamiento de red. Dentro de esta zona existen tres subzonas físicas:

Balanceador de Carga «AWS ELB»

Es el único punto de entrada público al backend. Recibe las peticiones HTTPS de ambos clientes y las distribuye entre las instancias de aplicación disponibles mediante un algoritmo de balanceo (round-robin o least connections). Si la instancia principal no responde, el ELB activa la redirección automática hacia la instancia de respaldo, representada en el diagrama con una flecha punteada. Este mecanismo implementa failover automático, garantizando continuidad del servicio ante fallos individuales de instancia.

Instancias de Aplicación

Esta subzona contiene dos nodos de ejecución del backend, ambos desplegados como artefactos .jar compilados de Spring Boot:

- API Principal «Spring Boot (.jar)»: instancia activa que procesa la totalidad del tráfico en condiciones normales de operación. Gestiona autenticación JWT, lógica de negocio, control de concurrencia para el agendamiento y generación de órdenes de pago.
- API Respaldo «Spring Boot (.jar)»: instancia en modo standby que el balanceador activa automáticamente ante la caída de la instancia principal. Mantiene una conexión de solo lectura a MySQL (para consultas no críticas) mientras está en espera, y conexión completa de lectura/escritura cuando asume el rol activo.

La separación en dos instancias idénticas implementa un esquema de alta disponibilidad activo-pasivo, asegurando que un fallo en el servidor principal no interrumpa el servicio a los usuarios.

Almacenamiento y Datos

Subzona que agrupa los dos servicios de persistencia del sistema:

- Base de Datos «MySQL»: nodo de base de datos relacional que recibe conexiones desde ambas instancias de aplicación. La instancia principal se conecta por el puerto 3306 con permisos de lectura y escritura completos. La instancia de respaldo mantiene una conexión de solo lectura (MySQL read replica), lo que le permite responder consultas sin comprometer la integridad transaccional mientras está en modo pasivo.
- Object Storage «Cloudinary»: Servicio de almacenamiento de objetos para archivos binarios (imágenes de mascotas, radiografías). Ambas instancias de aplicación se comunican con Cloudinary

mediante HTTPS usando el SDK/API de Cloudinary, garantizando una transferencia cifrada y segura de los activos.

3. Servicios Externos

Proveedores de terceros que el backend integra mediante protocolos estándar sobre canales seguros. Ambas instancias de aplicación pueden invocarlos, aunque en condiciones normales solo lo hace la instancia principal:

- Pasarela de Pagos «Flow»: se integra mediante HTTPS (JSON), siguiendo el estándar de comunicación de ambas pasarelas. El backend envía la orden de pago firmada y recibe el resultado de la transacción mediante un webhook callback sobre el mismo canal seguro.
- Servidor de Correos «SMTP / Gmail»: se integra mediante SMTP sobre HTTPS (TLS), canal cifrado para el envío de correos transaccionales (confirmaciones de cita, notificaciones de cancelación, códigos OTP de recuperación de contraseña).

○ **Vista de escenarios - Diagrama de casos de usos:**

■ Tutor:

Es el actor principal del sistema desde la perspectiva del usuario final. Sus casos de uso se organizan en tres paquetes funcionales.

Cuenta:

Agrupar la gestión del ciclo de vida de la sesión y el perfil personal. El caso de uso Registrar es el punto de entrada al sistema para nuevos usuarios; permite crear una cuenta con datos personales y credenciales. Iniciar sesión habilita el acceso a las funcionalidades protegidas. Recuperar clave extiende (<<extend>>) a Iniciar sesión, activándose opcionalmente cuando el usuario no recuerda su contraseña, implementando el flujo OTP de tres pasos descrito en el diagrama de secuencia. Editar perfil extiende a Visualizar perfil, reflejando que solo es posible editar una vez que se visualizan los datos actuales. Inhabilitar perfil permite al tutor desactivar su propia cuenta mediante soft delete, preservando el historial sin eliminar el registro. Cerrar sesión invalida el token JWT activo y elimina el estado local de autenticación.

Gestión de Mascotas:

Concentra el CRUD de mascotas implementado en la aplicación móvil. Agregar mascota extiende a Visualizar mascotas, dado que tras registrar una

nueva mascota el sistema redirige al listado actualizado. Del mismo modo, EditarMascota se extiende a Visualizar mascotas, pues la edición parte siempre desde la vista de detalle de una mascota existente. Ambas relaciones <<extend>> modelan correctamente que Visualizar mascotas es el caso de uso base y que agregar/editar son extensiones opcionales del mismo.

Agenda:

Contiene casos de uso relacionados con el agendamiento de citas. Agendar cita incluye (<<include>>) obligatoriamente a Bloquear hora temporalmente: sin importar el camino que tome el tutor, el proceso de agendamiento siempre requiere ejecutar el lock del bloque horario para gestionar la concurrencia (RNF 7.1). La relación <<include>> es la correcta aquí porque el bloqueo no es opcional; es parte estructural del flujo. Ver detalle cita extiende a Visualizar atenciones, modelando que el tutor puede opcionalmente profundizar en el detalle de una cita concreta desde el historial general de atenciones.

■ **Veterinario:**

Opera principalmente sobre la ficha clínica y el control de su agenda de citas. Sus casos de uso se dividen en dos paquetes especializados más la gestión de cuenta.

Consultas y Ficha Clínica

Es el paquete de mayor riqueza funcional del actor. Ingresar a detalle cita actúa como caso de uso central de este paquete: es el punto de acceso obligatorio (<<include>>) desde el cual se activan todos los demás. Una vez dentro del detalle de una cita, el veterinario puede ejecutar opcionalmente (<<extend>>):

- Registrar evolución médica (Diagnóstico e indicaciones): documenta el diagnóstico clínico y las indicaciones terapéuticas del paciente, constituyendo el núcleo de la ficha clínica.
- Registrar vacunas aplicadas: registra las vacunas administradas durante la consulta, vinculándolas al historial de vacunación de la mascota.
- Subir archivos médicos (Exámenes / Radiografías): permite adjuntar documentos binarios a la ficha clínica mediante integración con Cloudinary
- Derivar a exámenes externos: registra una derivación formal a un laboratorio o especialista externo, documentando la continuidad del tratamiento.

Consultar historial clínico y Buscar tutor y mascota son casos de uso independientes que no dependen de una cita activa, permitiendo al veterinario acceder al historial completo de un paciente en cualquier momento.

Control de Citas y Agenda

Visualizar citas de la clínica (Dashboard Médico) es el caso de uso base del paquete, que presenta al veterinario la agenda completa de la clínica. Ingresar a detalle cita extiende a este caso de uso, activándose cuando el veterinario selecciona una cita específica del dashboard. Cambiar estado de la cita extiende también al dashboard, permitiendo gestionar los estados del flujo de agendamiento (DISPONIBLE → EN_PROCESO → AGENDADA → ATENDIDA) directamente desde la vista de agenda.

Cuenta

El veterinario no puede registrarse por sí mismo; su cuenta es creada por el Administrador. Por ello este paquete carece del caso de uso Registrar y se limita a Iniciar sesión, Recuperar clave (que extiende al login), Visualizar perfil, Editar perfil y Cerrar sesión.

■ Administrador:

Es el actor con mayor nivel de privilegio del sistema. No interactúa con mascotas ni citas; su dominio es la configuración, el control de acceso y la auditoría.

Gestión de Usuarios y Acceso

Visualizar listado de usuarios es el caso de uso base del paquete. Suspende / Bloquear cuenta y Reactivar cuenta de usuario se extienden a este caso de uso: ambas acciones solo son posibles una vez que el administrador visualiza y selecciona un usuario del listado. Esta modelación con <<extend>> es correcta porque la suspensión y reactivación son acciones opcionales que enriquecen el caso de uso base sin ser parte obligatoria de él. Crear y configurar cuenta de Veterinario es un caso de uso independiente que permite al administrador asignar a nuevos profesionales en el sistema, asignándoles credenciales y configuración inicial.

Auditoría y Cumplimiento

Auditar cambios en expedientes médicos incluye (<<include>>) a Consultar registros de logs globales: auditar los cambios en fichas clínicas siempre requiere acceder primero al log global del sistema. La relación <<include>> es precisa: no es posible realizar una auditoría sin ejecutar la consulta de logs como paso obligatorio previo.

Configuración del Sistema

Agrupar las capacidades de parametrización de la plataforma. Configurar bloques de horarios base permite al administrador definir la estructura de la agenda (días, franjas horarias, duración de bloques) sobre la cual se generan los slots disponibles para los tutores. Activar/desactivar el pago en el agendamiento online es un interruptor operacional que permite habilitar o deshabilitar el cobro durante el proceso de reserva, útil para periodos de mantenimiento, atenciones de urgencia gratuitas o ajustes de pasarela.

3. Metodología de Trabajo

El equipo aplica Scrum con sprints de una semana. A la fecha se han ejecutado los siguientes sprints:

Sprint	Objetivo	Estado
Sprint 1	Arquitectura y Diseño	Completado
Sprint 2	Reglas de Negocio y Ambiente	Completado
Sprint 3 (actual)	Seguridad y Gestión de Usuarios	Completado
Sprint 4 (planificado)	Datos Maestros y Calidad	Completado

- Herramienta de gestión: Trello (tablero documentado en Documentacion/Trello.txt).
- Reuniones diarias (dailys) documentadas en Documentacion/Dailys/.
- Control de versiones con Git, ramas por integrante: main, RamaSeba, RamaCata, RamaRicardo.

Fase 2.2: Configuración del Ambiente de Pruebas y Procedimientos

1. Especificaciones del Entorno

Para garantizar que el ambiente de pruebas sea un reflejo fiel de producción, la arquitectura del proyecto NodeVet se sustenta en las siguientes tecnologías y configuraciones:

- **Lenguajes de Programación:** Java (JDK 21) para el servidor y TypeScript para las aplicaciones cliente.
- **Frameworks Principales:** Spring Boot para el desarrollo de la API RESTful, React para el frontend web, y React Native junto con Expo para el entorno móvil.
- **Base de Datos:** MySQL para el almacenamiento relacional estructurado.
- **Bibliotecas Clave (Seguridad):** Spring Security y JWT (JSON Web Tokens) para la autenticación y autorización segura de los usuarios.
- **Almacenamiento Externo:** Integración exclusiva con Cloudinary como el único servicio externo dedicado para la carga y almacenamiento unificado tanto de imágenes y documentos

médicos (radiografías, exámenes), como de las imágenes de perfil de los usuarios y sus mascotas.

- **Gestión de Variables de Entorno:** Las credenciales sensibles (claves de AWS, base de datos y pasarelas de pago) se inyectan dinámicamente mediante el archivo `application.properties` y variables `.env` garantizando que no se expongan en el código fuente.
- **Herramientas de Testing Automatizado:** Se integró `spring-boot-starter-test` (que incluye JUnit 5 y Mockito) para la creación de pruebas unitarias y de integración.
- **Aislamiento de Base de Datos (Replicación de Entorno):** Para evitar la corrupción de datos en el entorno de desarrollo, se configuró una base de datos en memoria **H2** (`application.properties` exclusivo en `src/test/resources`). Esto permite que Hibernate cree y destruya una réplica exacta de las tablas de MySQL en la memoria RAM cada vez que se ejecutan los test automáticos, logrando un ambiente efímero y seguro.

2. Procedimientos de Respaldo y Restauración

Para garantizar la integridad y disponibilidad de los datos persistentes del sistema, los procedimientos aplican exclusivamente sobre nuestro entorno de base de datos principal (MySQL). No se aplican sobre el entorno de pruebas, dado que este utiliza tecnología en memoria (H2) de carácter destructivo. Los pasos para el flujo de producción son los siguientes:

- **Generación de Respaldo (Backup):**

Para extraer una copia de seguridad exacta (esquemas y registros) de nuestro entorno relacional, utilizamos las herramientas nativas del motor en consola. El comando utilizado es:

```
mysqldump -u root -p nodevet_db > respaldo_nodevet.sql
```

- **Procedimiento de Restauración (Restore):**

En caso de una contingencia, corrupción de datos, o para trasladar la base a un servidor virgen, el respaldo se inyecta mediante el siguiente comando en consola:

```
mysql -u root -p nodevet_db < respaldo_nodevet.sql
```

3. Plan de Pruebas y Buenas Prácticas

El aseguramiento de la calidad del proyecto se divide en dos enfoques: validación interactiva y automatización de código, aplicando el patrón AAA (Arrange, Act, Assert).

- **Pruebas Automatizadas y de Integración (Backend)**

Se han configurado pruebas de integración automatizadas operando sobre Maven. Como evidencia, se implementó `UsuarioRepositoryTest` empleando la anotación `@DataJpaTest`.

- **Buenas prácticas aplicadas:** Este test levanta exclusivamente la capa de persistencia (Spring Data JPA) conectada a la base de datos H2 de la carpeta de testing. El test crea un usuario simulado usando el patrón Builder, lo guarda en la persistencia temporal, busca directamente por su correo y valida los resultados mediante AssertJ. Una vez finalizado el ciclo, la base de datos revierte los cambios, manteniendo intacto el ecosistema real.
- **Ejecución:** Estas pruebas operan como guardianes de la calidad y se ejecutan obligatoriamente al construir el proyecto con el comando `./mvnw clean test`, retornando **BUILD SUCCESS** si todas las validaciones son correctas.

Imágenes de pruebas:

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 9.583 s -- in com.nodevet.app.repository.UsuarioRepositoryTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.12:report (report) @ app ---
[INFO] Loading execution data file C:\Users\elric\Documents\NodeVet\NodeVet\Producto\backend\target\jacoco.exec
[INFO] Analyzed bundle '' with 15 classes
[INFO] BUILD SUCCESS
[INFO] Total time: 20.020 s
[INFO] Finished at: 2026-05-23T15:24:27-04:00
[INFO]
```

```
# Indicar que es un entorno de pruebas
spring.application.name=app-test

# Conexión a la base de datos en memoria H2 (RAM)
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Dialecto de H2 para JPA
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Muy importante: Crear y destruir la base de datos con cada prueba
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
```

- **Pruebas Operativas API y Verificación (Validación Manual)**

Para validar el funcionamiento operativo de la API y asegurar la correcta integración, las pruebas se ejecutan directamente utilizando la interfaz interactiva de **Swagger UI** (OpenAPI) generada automáticamente por el backend. El plan de pruebas incluye:

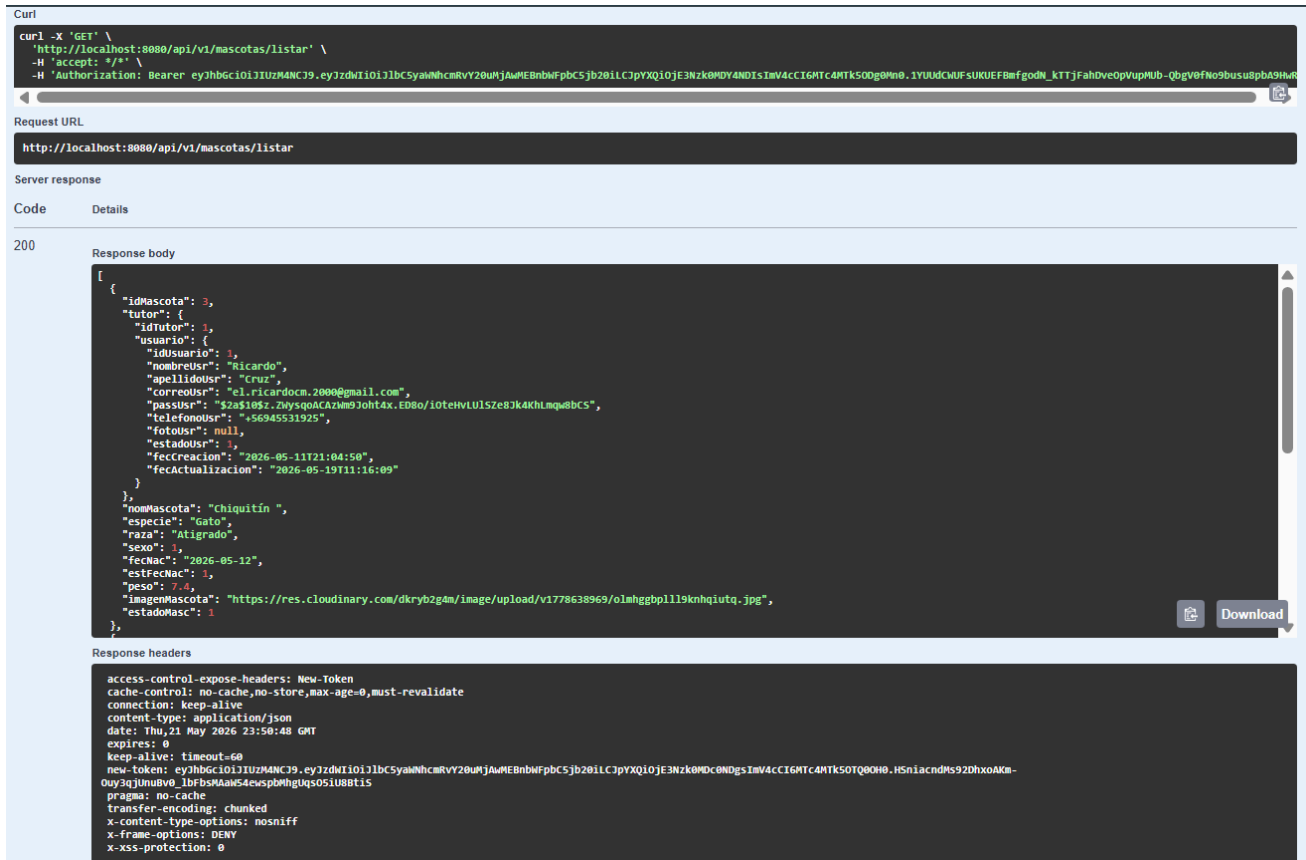
- **Validación de Endpoints Públicos:** Ejecución de peticiones **POST** en el endpoint de autenticación (ej. [/api/login](#)) a través de la interfaz visual de Swagger para verificar que el servidor valida correctamente las credenciales en la base de datos y retorna el token JWT correspondiente.
- **Validación de Endpoints Protegidos (Autorización):** Utilizando el botón "Authorize" nativo de Swagger, se inyecta el token JWT para simular peticiones **GET** hacia las rutas protegidas (como el historial clínico) y documentar las respuestas HTTP 200 (Éxito). Para validar la robustez de Spring Security, se ejecutan pruebas enviando las mismas peticiones sin el token (o con uno alterado), evidenciando que el sistema intercepta la petición y retorna correctamente el código HTTP 401 (No Autorizado).

Imágenes de pruebas:

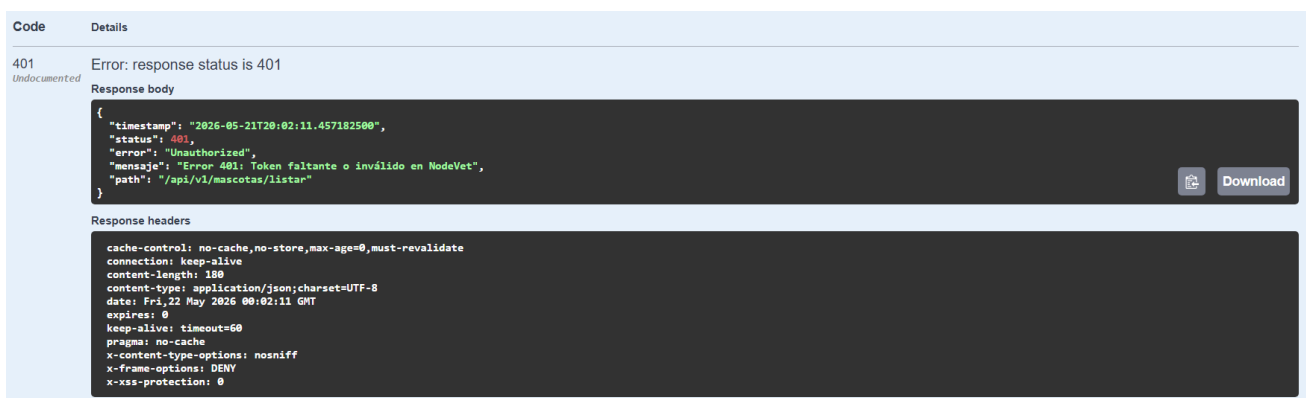
- **Validación de Autenticación en Swagger UI:** Se demuestra la correcta interacción con el endpoint [/api/auth/login](#), donde el sistema valida las credenciales y responde con un código **200 OK**, entregando el token JWT necesario para el acceso a rutas protegidas.

Code	Details
200	Response body <pre>{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1b20uMjAwMEBnbWVpbC5jb20iLCJpYXQiOiJlY3NkZk0MDY4NDIsImV4cCI6MTc4MTk5ODg0Mn0.1YUudCWUFsUKUEF8mfrodN_kTTjFahDveOpVupMUb-QbgV0fNo9busu8pbA9HwRM" }</pre> Response headers <pre>cache-control: no-cache,no-store,max-age=0,must-revalidate connection: keep-alive content-type: application/json date: Thu,21 May 2026 23:40:42 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache transfer-encoding: chunked x-content-type-options: nosniff x-frame-options: DENY x-xss-protection: 0</pre>

- **Prueba de Acceso a Endpoints Protegidos:** Se valida la correcta autorización del usuario mediante el envío de token JWT en la cabecera de la petición **GET** [/api/auth/mascotas/listar](#). El sistema responde con un código **200 OK** y entrega la estructura JSON con la información de las mascotas, confirmando que la integración entre Spring Security y el servicio de gestión de mascotas opera satisfactoriamente.



- **Verificación de Seguridad (Acceso Denegado):** Se demuestra la robustez del módulo de seguridad ante intentos de acceso no autorizados. Al realizar una petición a un endpoint protegido sin un token válido, el sistema intercepta la solicitud y retorna un código **401 Unauthorized** junto con un mensaje JSON estructurado, evitando la exposición de errores internos y cumpliendo con las políticas de seguridad del sistema.



Fase 2.3: Desarrollo de Software y Demostración

1. Funcionalidades Implementadas

App Móvil:

La aplicación móvil de NodeVet se construye sobre React Native como framework base multiplataforma y Expo como plataforma de herramientas y APIs nativas. Implementa tres flujos de navegación independientes según el rol autenticado (Tutor, Veterinario, Administrador), además del flujo público de bienvenida y autenticación.

- React Native - Framework: Es el núcleo de la aplicación. Provee los componentes de UI nativos, el sistema de estilos, la lógica de formularios y la gestión del estado de pantalla.
 - Componentes de interfaz:
 - View, Text, TextInput, ScrollView:
 - Bloques fundamentales del layout y entrada de datos presentes en todas las pantallas (Login, Registro, Recuperar contraseña, Mascotas).
 - TouchableOpacity:
 - Empleado en todos los botones, las “PetCard” y elementos interactivos, con retroalimentación visual de opacidad al presionar.
 - FlatList:
 - Renderiza el listado de mascotas en “MascotasScreen” y los modales de selección de especie/raza en “RegistroMascota” con virtualización de la lista para performance óptima.
 - KeyboardAvoidingView:
 - Usado en los formularios de Login y Registro para que el teclado virtual no tape los campos de entrada, con comportamiento diferenciado según “Platform.OS” ('padding' en iOS).
 - Image:
 - Renderiza las fotos de mascotas desde URLs remotas de Cloudinary en “DetalleMascotaScreen”, “PetCard” y “EditarMascota”. Si no hay imagen registrada, se muestra un placeholder con ícono de pata.
 - ActivityIndicator:
 - Indicador de carga que reemplaza el texto de los botones durante peticiones HTTP, implementado en todas las pantallas con operaciones asíncronas.
 - RefreshControl:
 - Permite hacer “pull-to-refresh” en el listado de mascotas para recargar los datos desde el servidor sin reiniciar la app.
 - Estilos y sistema de diseño
 - StyleSheet.create:

- Todos los estilos están definidos mediante la API nativa de React Native, con una hoja de estilos global (*"GlobalStyle.ts"*), estilos de dashboard (*"DashboardStyle.ts"*) y estilos locales por componente.
 - Fuente personalizada Fredoka:
 - Cargada mediante expo-font (ver sección Expo), disponible en 5 pesos (Light, Regular, Medium, SemiBold, Bold) referenciados desde *"typograph.ts"*.
 - Paleta de colores centralizada:
 - Archivo *"colors.ts"* con las constantes de color de la marca (darkDGreen, lightYellow, etc.) referenciado en toda la app.
- Navegación (React Navigation)
 - Stack Navigator (*"@react-navigation/stack"*):
 - Gestiona las rutas del flujo de autenticación (Welcome → Login / Registro → ForgotPassword) y las rutas protegidas (Main → DetalleMascota → EditarMascota).
 - Bottom Tab Navigator (*@react-navigation/bottom-tabs*):
 - Barra de navegación inferior con cinco pestañas: *"Mascotas"*, *"Ordenes, Home"*, *"Agenda"* y *"Perfil"*, con íconos de *@expo/vector-icons*.
 - AppNavigator reactivo:
 - Lee el estado *userToken* del *AuthContext* y decide automáticamente si renderizar el *"AuthStack"* o el *"BottomTabNavigator"*, sin necesidad de navegar manualmente al iniciar o cerrar sesión.
 - *useFocusEffect*:
 - Hook de React Navigation utilizado en *"MascotasScreen"* y *"ProfileScreen"* para recargar datos desde el backend cada vez que la pantalla gana el foco, garantizando que los cambios recientes siempre sean visibles.
- Gestión de estado y contexto
 - *AuthContext* (React Context API):
 - Contexto global que expone *"userToken"*, *"isLoading"*, *"signIn(token)"* y *"signOut()"*. Envuelve toda la aplicación desde *"App.tsx"* para que cualquier componente acceda al estado de autenticación sin prop drilling.
 - *useState* y hooks locales:
 - Cada pantalla administra su estado interno (campos de formulario, errores de validación, loading, visibilidad de modales) con hooks de React estándar.
- Lógica de formularios y validaciones
 - Validación en tiempo real en *"RegisterScreen"*:
 - Cada campo (nombre, apellido, celular, email, contraseña, confirmación) dispara su validador al cambiar, mostrando mensajes de error directamente bajo el campo afectado sin necesidad de un submit previo.
 - Checkbox de fecha estimada:

- El campo *“fecNacEst”* (0 = exacta, 1 = estimada) se alterna con un *“TouchableOpacity”* que simula un checkbox con ícono checkmark de Ionicons.
 - Radio buttons de sexo:
 - Selección de Macho/Hembra mediante dos *“TouchableOpacity”* con estilos condicionales (radioSelected) según el valor del formulario.
- Integración con el Backend (Axios)
 - axiosInstance.ts:
 - Instancia configurada con baseUrl: `http://${DEVELOPER_IP}:8080/api` y timeout de 10 segundos.
 - Interceptor de petición:
 - Lee el token JWT desde *“AsyncStorage”* e inyecta la cabecera Authorization: `Bearer <token>` en cada request automáticamente.
 - Interceptor de respuesta:
 - Detecta el header new-token en la respuesta para renovar la sesión automáticamente. Si recibe un 401, elimina el token local y el *“AuthContext”* redirige al login.
 - cancelAllRequests():
 - Función que cancela todas las peticiones Axios pendientes al cerrar sesión usando *“CancelToken”*, evitando actualizaciones de estado en componentes desmontados.
- Plataforma diferenciada
 - Platform.OS:
 - Utilizado en KeyboardAvoidingView y en el DateTimePicker para adaptar el comportamiento entre Android e iOS.
 - SafeAreaView (@react-native-safe-area-context):
 - Aplicado en *“MascotasScreen”* para respetar las zonas seguras del dispositivo (notch, barra de estado).
- Expo — APIs y Herramientas Nativas:

Expo actúa como capa de abstracción sobre las APIs nativas del dispositivo, simplificando el acceso a hardware y servicios del sistema operativo sin necesidad de código nativo.

 - expo-image-picker — Carga de Imágenes

Es la librería de Expo que permite al usuario acceder a la galería del dispositivo (o la cámara) para seleccionar una foto de mascota:

 - Configuración:
 - Se llama a *“ImagePicker.launchImageLibraryAsync()”* con *“allowsEditing”*: *“true, aspect: [1, 1], quality: 0.5”*. El recorte cuadrado asegura consistencia

visual en las tarjetas, y la compresión al 50% reduce el tiempo de subida a Cloudinary.

- Resultado:
 - Si el usuario no cancela (`!result.canceled`), la URI local de la imagen se guarda en el estado `"image"` del formulario y se previsualiza en pantalla con un componente `<Image source={{ uri: image }} />`.
- Subida a Cloudinary:
 - La URI local es empaquetada en un `FormData` y enviada al endpoint de Cloudinary vía `fetch`. Si la subida es exitosa, se recibe una `"secure_url"` HTTPS que se incluye en el payload enviado al backend Spring Boot.
- Uso de edición:
 - En `"EditarMascota.tsx"` se compara la UR seleccionada con la URL original de la mascota. Solo se sube la imagen a Cloudinary si el usuario eligió una foto nueva, evitando subidas innecesarias.
- @expo/vector-icons — Íconos Vectoriales
 - Se usa el set `Ionicons` y `"MaterialCommunityIcons"` en toda la app: `chevron-back`, `eye/eye-off`, `paw`, `calendar-outline`, `camera`, `notifications-outline`, `log-out-outline`, entre otros.
 - Los íconos son vectores SVG compilados, por lo que se escalan sin pérdida de calidad en cualquier densidad de pantalla y no requieren assets de imagen adicionales.
- expo-font — Fuente Personalizada
 - La fuente Fredoka se carga desde `"assets/fonts/"`. Su carga se gestiona en `"App.tsx"` con el hook `"useFonts"` de Expo, que retrasa el renderizado hasta que los assets estén disponibles.
 - Una vez cargada, la fuente está disponible globalmente mediante los tokens de `typography.ts`, garantizando consistencia visual en toda la aplicación.
- expo-splash-screen y expo-status-bar
 - expo-splash-screen:
 - Mantiene la splash screen visible mientras se cargan los assets (fuentes, token de `AsyncStorage`) al iniciar la app, evitando parpadeos de contenido sin cargar.
 - expo-status-bar:
 - Controla el color y estilo de la barra de estado del sistema operativo para mantener coherencia con el tema visual oscuro de NodeVet.
- AsyncStorage (@react-native-async-storage/async-storage)
 - Aunque no es exclusivo de Expo, se integra en el ecosistema Expo y es la solución estándar para persistencia clave-valor en React Native.

- Se utiliza para guardar y recuperar el JWT de sesión (userToken) entre sesiones de la app, de modo que el usuario no necesita iniciar sesión nuevamente al reabrir la aplicación (siempre que el token no haya expirado).

App Web:

- El frontend web de NodeVet es una Single Page Application (SPA) construida con React 19, TypeScript y Vite. Se comunica con el backend Spring Boot exclusivamente mediante llamadas HTTP/JSON a través de Axios, usando el mismo JWT que el cliente móvil. La arquitectura de rutas está gestionada por React Router DOM v7 y el proyecto no utiliza ningún framework de estado global (como Redux), apoyándose en el estado local de los componentes con los hooks useState y useEffect.
- Stack del Frontend Web

Tecnología	Versión	Rol en el proyecto
React	19.2.5	Librería de UI basada en componentes
TypeScript	~6.0.2	Tipado estático en todo el proyecto
Vite	^8.0.10	Bundler y servidor de desarrollo con proxy inverso
React Router DOM	^7.14.2	Enrutamiento client-side (SPA)
Axios	^1.16.0	Cliente HTTP con interceptores para JWT
react-easy-crop	^5.5.7	Recorte de imágenes circular antes de subir a Cloudinary
react-icons	^5.6.0	Íconos para la interfaz
babel-plugin-react-compiler	^1.0.0	Compilador experimental de React para optimización automática

- Arquitectura de Rutas — App.tsx
 - Toda la navegación está declarada en App.tsx usando BrowserRouter. Las rutas están organizadas por rol y funcionalidad:

```
// App.tsx — estructura completa de rutas
<BrowserRouter>
  <Routes>
    {/* Autenticación */}
    <Route path="/registro" element={<Registro />} />
    <Route path="/login" element={<Login />} />
    <Route path="/recuperar/solicitud" element={<RequestPassword />} />
    <Route path="/recuperar/restablecer" element={<ResetPassword />} />

    {/* Dashboards por rol */}
    <Route path="/dashboard/tutor" element={<DashboardTutor />} />
    <Route path="/dashboard/medico" element={<DashboardMedico />} />
    <Route path="/dashboard/admin" element={<DashboardAdmin />} />

    {/* CRUD Mascotas */}
    <Route path="/agregar-mascota" element={<AgregarMascota />} />

    {/* Citas (en desarrollo) */}
    <Route path="/agendarCita" element={<SeleccionDia />} />
    <Route path="/agendarCita/formulario" element={<FormularioAgendarCita />} />

    <Route path="/" element={<Navigate to="/home" />} />
    <Route path="*" element={<Navigate to="/home" replace />} />
  </Routes>
</BrowserRouter>
```

codesnap.dev

- La ruta comodín (*) redirige cualquier URL desconocida a /home, evitando pantallas de error vacías.
- Cliente HTTP y Gestión del JWT — [client.ts](#)
 - El módulo src/api/client.ts centraliza toda la comunicación con el backend. Implementa dos interceptores Axios que automatizan el manejo del token JWT sin que ningún componente tenga que gestionarlo manualmente.
 - Configuración base con Proxy de Vite
 - El cliente usa baseURL: '/api', lo que aprovecha el proxy inverso configurado en vite.config.ts para redirigir las peticiones al backend durante el desarrollo:

```
// vite.config.ts — proxy para evitar problemas de CORS en desarrollo
server: {
  proxy: {
    '/api': {
      target: 'http://localhost:8080',
      changeOrigin: true
    }
  }
}

// client.ts — baseURL relativa, funciona en dev y producción
const api = axios.create({ baseURL: '/api' });
```

codesnap.dev

- Esta configuración resuelve el problema de CORS en desarrollo: el browser hace peticiones a su propio origen (localhost:5173/api) y Vite las redirige transparentemente al backend en localhost:8080.
- Interceptor de Request — Adjunta el JWT

```
// Antes de cada petición, busca el token en localStorage y lo adjunta
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

codesnap.dev

- A diferencia del cliente móvil (que usa AsyncStorage), el cliente web usa localStorage del navegador. Esto permite que el token persista entre recargas de página sin requerir al usuario que inicie sesión nuevamente dentro de las 8 horas de vigencia.
- Interceptor de Response — Renovación y manejo de 401

```
api.interceptors.response.use(
  (response) => {
    // Si el servidor manda un token renovado, lo guardamos automáticamente
    const newToken = response.headers['new-token'];
    if (newToken) localStorage.setItem('token', newToken);
    return response;
  },
  (error) => {
    if (error.response?.status === 401) {
      // Token expirado (8h): limpiar y redirigir al login
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

codesnap.dev

- El manejo del 401 en el cliente web usa `window.location.href` (redirección dura) en lugar de `useNavigate`, porque el interceptor vive fuera del árbol de componentes React y no tiene acceso al contexto de routing. Esto garantiza que la sesión nunca quede en estado inconsistente.
- Módulo de Autenticación
 - Login — `Login.tsx`
 - La pantalla de login gestiona el estado del formulario con `useState`, valida credenciales contra el backend y almacena el JWT en `localStorage`:

```
// Login.tsx — flujo de autenticación
const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  const response = await api.post('/auth/login', { correoUsr, passUsr });

  // Compatibilidad con diferentes campos de respuesta del backend
  const token = response.data.jwt || response.data.token || response.data.access_token;

  localStorage.setItem('token', token);
  navigate('/dashboard/tutor'); // redirige al dashboard
};
```

codesnap.dev

- La extracción del token con tres campos alternativos (jwt, token, access_token) es una decisión defensiva que da flexibilidad ante cambios en el contrato del backend sin romper el frontend.

El formulario incluye estados de carga (loading) y error que se reflejan en la UI: el botón muestra 'Iniciando...' mientras la petición está en vuelo y el campo error se renderiza condicionalmente solo cuando hay un mensaje.

- Registro — Registro.tsx

- El formulario de registro implementa validación de contraseña en el cliente antes de enviar la petición al backend, reduciendo llamadas innecesarias a la API:

```
// Registro.tsx — validación client-side de contraseña
const passErrors: string[] = [];
if (pass.length < 6) passErrors.push('mínimo 6 caracteres');
if (!/[A-Z]/.test(pass)) passErrors.push('al menos 1 mayúscula');
if (!/[a-z]/.test(pass)) passErrors.push('al menos 1 minúscula');
if (!/[!@#$$%^&*()...]/.test(pass)) passErrors.push('al menos 1 carácter especial');

if (passErrors.length > 0) {
  setError('Contraseña inválida: ' + passErrors.join(', '));
  return; // corta antes de llamar al API
}
```

codesnap.dev

- Las reglas de validación acumulan todos los errores en un array y los muestran juntos, evitando la frustración del usuario de corregir un error a la vez. El campo confirmPassword se valida localmente y nunca se envía al backend.

La petición de registro va a POST /api/auth/usuarios/registro con el mismo DTO que el cliente móvil (nombreUsr, apellidoUsr, correoUsr, passUsr, telefonoUsr), garantizando consistencia en el contrato de la API.

- Recuperación de Contraseña — 2 pantallas, 3 pasos
 - El flujo de recuperación está dividido en dos componentes: RequestPassword.tsx maneja los dos primeros pasos en una sola pantalla con renderizado condicional, y ResetPassword.tsx gestiona el tercer paso.
 - RequestPassword.tsx — Paso 1 y 2 en una sola pantalla

```
// Un solo formulario que cambia de comportamiento según el estado
const [codigoEnviado, setCodigoEnviado] = useState(false);

// El onSubmit apunta a una función diferente según el estado
<form onSubmit={codigoEnviado ? handleVerifyCode : handleSendCode}>
  <input type="email" disabled={codigoEnviado} ... />

  {/* El campo de código aparece solo DESPUÉS de enviar el correo */}
  {codigoEnviado && (
    <input type="text" placeholder="Ingrese el código" ... />
  )}
</form>

// Validación del código: 6 dígitos exactos
if (codeToSubmit.length !== 6 || !/^\d+$/.test(codeToSubmit)) {
  setError('El código debe contener 6 dígitos numéricos');
  return;
}
```

codesnap.dev

- El campo de correo se deshabilita (disabled) una vez enviado el código para evitar que el usuario lo modifique accidentalmente. El botón 'Reenviar' permite solicitar un nuevo código sin recargar la página.
- Antes de navegar a la siguiente pantalla, el componente persiste correo y código en localStorage para que ResetPassword los pueda leer sin depender de props o estado compartido:

```
// Al verificar correctamente el código:
localStorage.setItem('resetCorreo', emailToSubmit);
localStorage.setItem('resetCodigo', codeToSubmit);
navigate('/recuperar/restablecer');
```

codesnap.dev

- ResetPassword.tsx — Paso 3

```
// Lee el correo y código guardados por la pantalla anterior
const resetCorreo = localStorage.getItem('resetCorreo');
const resetCodigo = localStorage.getItem('resetCodigo');

await api.post('/auth/reset-password', {
  correo_usr: resetCorreo,
  codigo: resetCodigo,
  nueva_pass: nuevaPassword,
});

// Limpieza obligatoria: elimina los datos temporales
localStorage.removeItem('resetCorreo');
localStorage.removeItem('resetCodigo');
navigate('/login');
```

codesnap.dev

- La limpieza de los datos temporales en localStorage después del éxito es una buena práctica de seguridad: evita que el código OTP quede almacenado en el navegador indefinidamente.

- Dashboard por roles
 - El frontend web implementa tres dashboards diferenciados según el rol del usuario autenticado, cada uno consultando endpoints distintos del backend:

Dashboard	Ruta	Endpoints consumidos	Funcionalidades
DashboardTutor	/dashboard/tutor	GET /v1/usuarios/perfil, GET /v1/mascotas/listar	Lista de mascotas con PetCards, próximas citas, últimos controles, navegación a agregar/editar mascota
DashboardMedico	/dashboard/medico	GET /v1/medico/perfil, GET /v1/medico/citas	Vista de citas asignadas al médico autenticado
DashboardAdmin	/dashboard/admin	GET /v1/admin/perfil, stats, usuarios, mascotas, citas	Panel de control con estadísticas globales: n° médicos, tutores, mascotas y listados completos

- DashboardTutor — Detalle de implementación
 - Es el dashboard más completo implementado a la fecha. Al montar el componente, dispara dos peticiones en paralelo mediante useEffect:

```
// DashboardTutor.tsx – carga de datos al montar
useEffect(() => {
  const fetchData = async () => {
    try {
      const userData = await api.get('/v1/usuarios/perfil');
      setUsuario(userData.data);

      const mascotasData = await api.get('/v1/mascotas/listar');
      setMascotas(mascotasData.data);
    } catch (error: any) {
      if (error.response?.status === 401) {
        localStorage.removeItem('token');
        navigate('/login'); // sesión expirada
      } else {
        setError('Error al cargar los datos...');
      }
    } finally {
      setLoading(false);
    }
  };
  fetchData();
}, [navigate]);
```

codesnap.dev

- El manejo de errores es explícito: el 401 representa sesión expirada (limpia token y redirige), cualquier otro error muestra un mensaje en la UI sin romper la aplicación.
- La lista de mascotas se renderiza con el componente PetCard. Las acciones de editar y eliminar se pasan como callbacks:

```
// Navegar a AgregarMascota en modo edición, pasando la mascota por state
const handleEditMascota = (mascota: Mascota) => {
  navigate('/agregar-mascota', { state: { mascota, isEditing: true } });
};

// Eliminar y actualizar el estado local sin recargar
const handleDeleteMascota = async (id: number) => {
  await api.delete(`/v1/mascotas/eliminar/${id}`);
  setMascotas(mascotas.filter(m => m.idMascota !== id));
};
```

codesnap.dev

- El delete actualiza el estado local inmediatamente (optimistic update simplificado): filtra la mascota eliminada del array local, dando respuesta visual instantánea sin esperar un nuevo GET al servidor.
- CRUD de Mascotas — AgregarMascota.tsx
 - Este componente es el más complejo del frontend web. Funciona en dos modos (crear y editar) determinados por el estado de navegación recibido desde el dashboard, e implementa un flujo completo de carga, recorte y subida de imagen.
- Modo dual: Crear vs. Editar

```
// El componente detecta su modo al montarse
const isEditing = location.state?.isEditing || false;
const mascotaToEdit: Mascota | undefined = location.state?.mascota;

// Pre-rellena todos los campos si viene en modo edición
const [nombre, setNombre] = useState(mascotaToEdit?.nomMascota || '');
const [tipo, setTipo] = useState(mascotaToEdit?.especie || '');
const [raza, setRaza] = useState(mascotaToEdit?.raza || '');
// ... etc para todos los campos
```

codesnap.dev

- Esta arquitectura reutiliza el mismo componente para crear y editar, reduciendo duplicación de código. El título del formulario, el texto del botón y el endpoint de submit cambian dinámicamente según el modo:

```
// Submit adapta el endpoint según el modo
if (isEditing && mascotaToEdit?.idMascota) {
  await api.put(`/v1/mascotas/actualizar/${mascotaToEdit.idMascota}`, payload);
} else {
  await api.post(`/v1/mascotas/registrar`, payload);
}
```

codesnap.dev

- Flujo de imagen con recorte circular
 - AgregarMascota implementa un flujo completo de gestión de imagen en 4 etapas usando la librería react-easy-crop:

Etapas	Descripción	Implementación
1. Selección	El usuario hace click en el avatar circular, que activa un input file oculto	ref={fileInputRef} + fileInputRef.current?.click()
2. Recorte	Se abre un modal con el componente Cropper superpuesto en pantalla completa	position: fixed, z-index: 9999, cropShape='round', aspect={1}
3. Aplicar	getCroppedImg() convierte el área seleccionada en un File que se guarda en estado	setImagenCropped(croppedFile), setImagenPreview(URL.createObjectURL(...))
4. Subida	Al hacer submit, se sube el File a Cloudinary y se guarda la URL resultante en el backend	subirImagenCloudinary(imagenCropped, 'mascotas') → URL → payload

```
// AgregarMascota.tsx – flujo de imagen resumido
// Paso 2: abre modal de recorte
const handleImageChange = (e) => {
  const imageUrl = URL.createObjectURL(e.target.files[0]);
  setImageToCrop(imageUrl);
  setIsCropping(true);
};

// Paso 3: guarda resultado del recorte como File
const handleSaveCrop = async () => {
  const croppedFile = await getCroppedImg(imageToCrop, croppedAreaPixels);
  setImagenCropped(croppedFile);
  setImagenPreview(URL.createObjectURL(croppedFile));
  setIsCropping(false);
};

// Paso 4: sube a Cloudinary en el submit
if (imagenCropped) {
  imageUrl = await subirImagenCloudinary(imagenCropped, 'mascotas');
}
```

codesnap.dev

- El recorte es circular (cropShape='round', aspect={1}), lo que garantiza que todas las fotos de mascotas tienen proporciones cuadradas 1:1, consistente con la presentación en los PetCards del dashboard.
- Servicio Cloudinary — [cloudinaryService.ts](#)

```
// cloudinaryService.ts — subida directa desde el navegador
export const subirImagenCloudinary = async (file: File, tipo: 'mascotas' | 'usuarios') => {
  const data = new FormData();
  data.append('file', file);
  data.append('upload_preset', tipo === 'mascotas' ? 'mascotas_preset' : 'usuarios_preset');

  const response = await fetch(
    'https://api.cloudinary.com/v1_1/dkryb2g4m/image/upload',
    { method: 'POST', body: data }
  );
  const result = await response.json();
  return result.secure_url; // URL HTTPS lista para guardar en BD
};
```

codesnap.dev

- La subida va directamente del navegador a Cloudinary (bypass del backend), lo que evita que imágenes pesadas pasen por el servidor Spring Boot y respeta el límite de 5 MB configurado en application.properties. El servicio acepta un parámetro tipo para usar diferentes upload_presets de Cloudinary según si la imagen es de mascota o de usuario.
- Payload final al backend

```
// Construcción del payload: imagen nueva, imagen existente, o sin imagen
const payload = {
  nomMascota: nombre,
  especie: tipo,
  raza: raza,
  sexo: sexo, // number: 1=Macho, 0=Hembra
  fecNac: fecNac || null,
  estFecNac: estFecNac, // number: 0=exacta, 1=estimada
  peso: peso ? parseFloat(peso) : null,
  imagenMascota: imageUrl // nueva URL de Cloudinary
  || mascotaToEdit?.imagenMascota // URL existente en edición
  || undefined // sin imagen
};
```

codesnap.dev

- La lógica de tres alternativas para imagenMascota garantiza que en modo edición, si el usuario no selecciona una nueva foto, se preserva la URL existente de la mascota en lugar de sobrescribirla con null.
- Componente Reutilizable — PetCard

- PetCard es el componente visual que representa a una mascota en el dashboard. Es completamente controlado desde el exterior mediante props tipadas en TypeScript:

```
// PetCard.types.ts – contrato del componente
export interface PetCardProps {
  mascota: Mascota;
  onEdit?: (mascota: Mascota) => void;
  onDelete?: (id: number) => Promise<void>;
  onClick?: () => void;
}

// Mascota interface – espejo del modelo del backend
export interface Mascota {
  idMascota?: number;
  nomMascota: string;
  especie: string;
  raza?: string;
  sexo: number; // 1=Macho, 0=Hembra
  fecNac?: string;
  estFecNac?: number;
  peso?: number;
  imagenMascota?: string;
  estadoMasc?: number;
}
```

codesnap.dev

- El componente implementa su propio modal de confirmación para el delete, con estado interno showDeleteModal y control de doble clic accidental (stopPropagation para que el click en los botones no active el onClick del card):

```
// PetCard.tsx — modal de confirmación interno
const handleDeleteClick = (e: React.MouseEvent) => {
  e.stopPropagation(); // evita propagar al onClick del card
  setShowDeleteModal(true);
};

// El modal muestra el nombre de la mascota para confirmar
{showDeleteModal && (
  <div className="modal-overlay">
    <h2>¿Seguro que quieres eliminar a {nombreDisplay}?</h2>
    <button onClick={handleConfirmDelete}>
      {isDeleting ? 'Eliminando...' : 'Eliminar'}
    </button>
  </div>
)}

```

codesnap.dev

- Los botones de editar y eliminar se superponen sobre la imagen con posicionamiento absoluto (CSS), apareciendo solo al pasar el cursor. Si el prop onEdit u onDelete no se proporciona, los botones no se renderizan, haciendo el componente flexible para distintos contextos de uso.
- Resumen de Funcionalidades Implementadas

Funcionalidad	Componente	Estado	Observación
Inicio de sesión (JWT)	Login.tsx	Completo	Token en localStorage, redirección a dashboard
Registro de usuario	Registro.tsx	Completo	Validación client-side de contraseña (regex)
Recuperación contraseña	RequestPassword.tsx + ResetPassword.tsx	Completo	Flujo OTP de 3 pasos, datos pasados via localStorage
Dashboard Tutor	DashboardTutor.tsx	Completo	Carga de mascotas real desde la API
Dashboard Médico	DashboardMedico.tsx	Estructura	Endpoints de médico pendientes en backend
Dashboard Admin	DashboardAdmin.tsx	Estructura	Endpoints de admin pendientes en backend

Agregar mascota	AgregarMascota.tsx	Completo	Incluye recorte circular de imagen con Cloudinary
Editar mascota	AgregarMascota.tsx	Completo	Mismo componente, modo dual via navigation state
Eliminar mascota	PetCard + DashboardTutor	Completo	Soft delete + modal de confirmación
Ver mascotas (PetCards)	PetCard.tsx	Completo	Componente reutilizable con foto, nombre, especie
Agendar cita	SeleccionDia + Formulario	Estructura	Pantallas creadas, integración con backend pendiente

2. Validaciones, Seguridad y Buenas Prácticas

- **Encriptación de Contraseñas (Hashing Unidireccional)**

Para cumplir con el requerimiento de seguridad **RNF 2.1** y alinearse con el marco legal chileno (Ley N° 19.628 sobre Protección de la Vida Privada y la Ley Marco de Ciberseguridad), el sistema prohíbe el almacenamiento de contraseñas en texto plano. En su lugar, utiliza el algoritmo **BCrypt** a través de `PasswordEncoder` en Spring Boot.

Se seleccionó BCrypt por dos características técnicas fundamentales: primero, incorpora un **factor de costo adaptativo** que ralentiza computacionalmente la verificación, haciéndolo resistente a ataques modernos de fuerza bruta; segundo, aplica un **salt aleatorio dinámico** a cada clave, asegurando que contraseñas idénticas generen hashes distintos para anular ataques de diccionario. Esta implementación garantiza la confidencialidad absoluta de los expedientes clínicos frente a posibles filtraciones, mitigando riesgos de suplantación y eximiendo al proyecto de sanciones legales.

- **Implementación de Seguridad con JWT (JSON Web Tokens)**

Para dar cumplimiento al requerimiento de autenticación segura **RNF 2.2**, el sistema implementa un esquema basado en tokens JWT. La arquitectura de este módulo se diseñó considerando la resiliencia del servidor y la experiencia del usuario (renovación de sesiones), dividiéndose en tres componentes clave:

- **Paso A: Gestión Criptográfica y Resiliencia (JwtUtil)**

A diferencia de configuraciones básicas que pierden las sesiones si el servidor se reinicia, la clase `JwtUtil` se configuró utilizando una semilla estática (**SECRET_SEED**). Esto garantiza que las firmas criptográficas sobrevivan a los despliegues en AWS. Además, la clase administra tiempos de expiración dinámicos,

diferenciando si la petición proviene de un entorno web (8 horas) o móvil (30 días), optimizando la persistencia de la sesión según el cliente.

- **Paso B: Filtro de Intercepción y Renovación de Sesión (JwtRequestFilter)**

El filtro **JwtRequestFilter** intercepta cada petición HTTP. Una de sus características clave en la arquitectura es la renovación automática del token. Si un usuario envía un token válido, el filtro no solo lo autentica, sino que intercepta la petición, genera un token completamente nuevo y lo inyecta en las cabeceras de respuesta (**New-Token**). Esto evita desconexiones abruptas mientras el usuario utiliza la aplicación.

- **Paso C: Cadena de Seguridad y Manejo de Excepciones (SecurityConfig)**

Finalmente, la clase **SecurityConfig** define el comportamiento de los endpoints. Todas las rutas públicas (como Swagger, login y recuperación de contraseñas) son liberadas mediante **permitAll()**, mientras que el resto exige autenticación obligatoria. Como buena práctica adicional, se interceptó el punto de entrada de autenticación (**authenticationEntryPoint**) para sobrescribir el error HTML por defecto de Spring Boot. Si una petición carece de un token válido, el servidor estructura y devuelve explícitamente un objeto JSON con estado HTTP 401, facilitando su consumo por parte del frontend.

- **Sanitización de Entradas y Prevención de Inyecciones (SQLi / XSS)**

Para garantizar la robustez del sistema, proteger la integridad de los expedientes clínicos y cumplir con el estándar de seguridad de la plataforma (RNF 11.3), la arquitectura del backend implementa una estrategia orientada a mitigar las vulnerabilidades más críticas identificadas por OWASP.

- **Prevención de Inyección SQL (SQLi):**

La capa de persistencia se delega completamente a Spring Data JPA (Hibernate). Este framework utiliza nativamente Prepared Statements. Esto significa que cualquier dato enviado por el usuario (como el nombre o la raza de la mascota) es tratado estrictamente como un parámetro literal y nunca como un comando ejecutable, neutralizando intentos de inyección en MySQL.

- **Sanitización Estructural:**

Para mitigar inyecciones maliciosas en las peticiones, la API utiliza el patrón DTO gestionado a través de Lombok (**@Data**) y combinado con Jakarta Bean Validation. El sistema evalúa que el tipo de dato y el formato sean exactamente los esperados. Validaciones estrictas como **@NotBlank** y **@DecimalMin** (para evitar pesos

negativos o corruptos) bloquean el paso de datos anómalos o scripts camuflados antes de que alcancen la lógica de negocio.

- **Validación Restrictiva de Archivos (Límite de 5MB y Protección del Servidor)**

Para cumplir con la validación restrictiva de recursos (**RNF 11.2**) y salvaguardar la estabilidad de la infraestructura, el sistema implementa un control sobre el tamaño de los archivos multimedia. Permitir la carga de archivos sin límite expone al servidor a riesgos de saturación de memoria y posibles ataques de Denegación de Servicio (DoS) al agotar el almacenamiento.

Para mitigar este riesgo, el límite de 5MB se configuró a nivel del Servlet de Spring Boot directamente en el archivo de propiedades del entorno. De esta manera, el servidor web (Tomcat) rechaza automáticamente cualquier petición que exceda el tamaño máximo permitido, bloqueando la amenaza en la capa de red antes de que sature la memoria de la aplicación.

- **Buenas Prácticas de Código**

Para asegurar la mantenibilidad y la calidad del software a largo plazo, el proyecto adopta principios de ingeniería que garantizan un código limpio, legible y desacoplado, alineándose con estándares industriales de desarrollo en Spring Boot.

1. **Arquitectura por Capas (Separación de Responsabilidades):**

El backend se estructuró bajo el patrón de Arquitectura por Capas (N-Tier). Esta organización aísla las responsabilidades del sistema:

- **Capa de Controladores (**controller**):** Se limita exclusivamente a recibir peticiones HTTP y retornar respuestas (códigos 200, 201, 401), delegando toda la ejecución a servicios para evitar código "espagueti".
- **Capa de Servicios (**service**):** Concentra la lógica de negocio pura. Esto permite que, si en el futuro cambian las reglas del negocio, el código esté en un único lugar sin afectar la infraestructura de red.
- **Capa de Persistencia (**repository**):** Encapsula el acceso a datos mediante Spring Data JPA, manteniendo la lógica de la base de datos totalmente independiente de la lógica de la aplicación.

2. **Uso de Patrones de Diseño y Código Limpio:**

- **Reducción de Boilerplate con Lombok:** Se utilizó la anotación **@Data** de la librería Lombok en los DTOs. Esto elimina la necesidad de escribir manualmente métodos **getters**, **setters**, **equals** y **hashCode**, reduciendo

drásticamente la cantidad de líneas de código y facilitando la legibilidad sin sacrificar la funcionalidad.

- **Inmutabilidad de Datos con DTOs:** La API emplea DTOs para el intercambio de datos. Al utilizar esta capa, garantizamos que las entidades internas de la base de datos (Modelos JPA) no queden expuestas directamente, lo que aumenta la seguridad del sistema y permite modificar la estructura de la base de datos sin romper la API que consumen las aplicaciones cliente.

3. Inyección de Dependencias y Código Desacoplado:

Siguiendo los principios de **SOLID**, todos los componentes del sistema (servicios, filtros y controladores) utilizan **Inyección de Dependencias a través de constructores** en lugar de inyección de campos (`@Autowired` directo). Esto hace que las clases sean mucho más fáciles de testear unitariamente y asegura que todas las dependencias sean obligatorias para que el componente funcione, evitando errores de punteros nulos (`NullPointerExceptions`) en tiempo de ejecución.

- **Especificación de Endpoints**

Como materialización de la arquitectura por capas y las buenas prácticas de desarrollo implementadas, la API RESTful de NodeVet expone los siguientes endpoints. Esta interfaz sirve como contrato oficial para la comunicación entre el frontend (Web/Móvil) y el backend, garantizando una interacción estandarizada, segura y documentada mediante el esquema de seguridad basado en JWT.

Método	Endpoint	Descripción	Acceso
POST	/api/auth/login	Autenticación y generación de token JWT	Público
POST	/api/auth/usuarios/registro	Registro de usuario (Tutor/Médico)	Público
POST	/api/auth/recuperacion	Solicitud de recuperación de contraseña	Público
POST	/api/auth/verificacion	Validación de código de recuperación	Público
POST	/api/auth/password	Establecimiento de nueva contraseña	Público
POST	/api/v1/mascotas	Registro de nueva mascota	Privado (JWT)
GET	/api/v1/mascotas	Listado de mascotas del usuario	Privado (JWT)

Método	Endpoint	Descripción	Acceso
POST	/api/auth/login	Autenticación y generación de token JWT	Público
POST	/api/auth/usuarios/registro	Registro de usuario (Tutor/Médico)	Público
POST	/api/auth/recuperacion	Solicitud de recuperación de contraseña	Público
POST	/api/auth/verificacion	Validación de código de recuperación	Público
PUT	/api/v1/mascotas/{id}	Actualización de perfil de mascota	Privado (JWT)

3. Integración Front-Back

Integración Web:

- Tecnologías utilizadas:
 - Axios: Cliente HTTP principal utilizado para la comunicación con el backend
 - API Fetch Nativa: Utilizada para consumo específico de servicios externos, como la subida de imágenes a Cloudinary.
 - Vite Proxy: Configuración de proxy en el servidor de desarrollo local para evitar problemas de CORS y unificar la ruta base.
- Configuración del entorno de Desarrollo (Vite)
 - En el archivo "[vite.config.ts](#)", se configura un proxy inverso para el servidor de desarrollo. Todas las solicitudes que comiencen con el prefijo "/api" son redirigidas al backend en "<http://localhost:8080>". Esto permite al cliente Axios configurar simplemente "/api" como "baseURL", haciendo que la aplicación sea agnóstica a la ubicación final del backend cuando se construye para producción.
- Cliente HTTP Centralizado (Axios)
 - El cliente API principal se encuentra modularizado en "src/api/client.ts". Esta centralización proporciona un único punto de entrada para configurar comportamientos globales en todas las peticiones al backend.
 - Instancia Base
 - Se crea una instancia de Axios con la URL base de acuerdo a la configuración del proxy:

```
const api = axios.create({
  baseURL: '/api'
});
```

codesnap.dev

- Interceptores
 - El cliente centralizado implementa dos interceptores clave para la gestión de autenticación basada en JWT:
 - Interceptor de Solicitud (Request):
 - Garantiza que todas las solicitudes dirigidas al backend incluyan el token de acceso si el usuario está autenticado.
 - Obtiene el token de “localStorage”.
 - Si existe, inyecta el encabezado “Authorization: Bearer <token>” en la solicitud saliente.
 - Interceptor de Respuesta (Response):
 - Maneja de manera global la rotación de tokens y los flujos de error relacionados con la autenticación.
 - Rotación de Token: Inspecciona los encabezados de la respuesta en busca de un “new-token”. Si el backend envía un nuevo token en alguna respuesta exitosa, este se actualiza automáticamente en el “localStorage”.
 - Manejo de Error 401 (Unauthorized): Si el servidor responde con un estado 401 (token expirado o inválido), se intercepta el error globalmente:

1. Se elimina el token obsoleto de “localStorage”.
 2. Se redirige forzosamente al usuario a la vista de “/login” vía “window.location.href”, purgando el estado en memoria para mantener la consistencia en el cliente web
- Patrón de Consumo en Componentes
 - Los componentes y páginas de la aplicación importan la instancia configurada (“api”) para realizar sus operaciones CRUD

```
import api from '../../../api/client';

// Ejemplo genérico extraído de Dashboards (DashboardTutor / DashboardAdmin)
const fetchData = async () => {
  try {
    const response = await api.get('/endpoint');
    // Manejo de la respuesta
  } catch (error) {
    // Axios lanza excepciones automáticamente para códigos de estado ≥ 400
  }
}
```

codesnap.dev

- El manejo de las llamadas se realiza típicamente dentro de los hooks “useEffect” (para inicializar vistas) o dentro de manejadores de eventos asíncronos (“onSubmit”, “onClick”).
- Integraciones Externas (Cloudinary)
 - La plataforma utiliza el servicio Cloudinary para almacenar y servir imágenes (ej. fotos de mascotas y perfiles). Este consumo se aísla en el servicio “src/services/[cloudinaryService.ts](#)”.
 - Para este caso particular no se emplea la instancia de Axios centralizada porque:
 1. Es un dominio distinto (“https://api.cloudinary.com/...”).
 2. No requiere la inyección del token JWT de la aplicación local.

En su lugar, utiliza la API “fetch” nativa

```
// src/services/cloudinaryService.ts
export const subirImagenCloudinary = async (file: File, tipo: 'mascotas' | 'usuarios'): Promise<string | null> => {
  // Construcción de FormData con el 'upload_preset' específico
  const data = new FormData();
  data.append('file', file);
  data.append('upload_preset', nombrePreset);

  const response = await fetch('https://api.cloudinary.com/v1_1/dkryb2g4m/image/upload', {
    method: 'POST',
    body: data,
  });

  // Parseo y retorno de la URL segura
  const result = await response.json();
  return result.secure_url;
};
```

codesnap.dev

Esta clara separación de responsabilidades asegura que las peticiones al backend fluyan por un canal asegurado y estricto, mientras que las subidas de archivos externas se comuniquen de forma independiente y eficiente.

Frontend Móvil:

- Arquitectura de la Integración Cliente-Servidor
 - La arquitectura del sistema se fundamenta en un modelo Cliente-Servidor, donde la aplicación móvil (Frontend) desarrollada en React Native (Expo) actúa como cliente, consumiendo los servicios expuestos por la API RESTful alojada en el Backend (Spring Boot).
 - La comunicación entre ambas capas se realiza de manera asíncrona a través del protocolo HTTP/HTTPS. El formato estándar para el intercambio de datos (cargas útiles de peticiones y respuestas) es exclusivamente JSON (JavaScript Object Notation), garantizando una alta interoperabilidad y un procesamiento ligero en los dispositivos móviles.

- Gestión de Seguridad y Control de Sesiones

La protección de los endpoints y la persistencia de la sesión del usuario se gestionan mediante el estándar JWT (JSON Web Tokens). El flujo de autenticación opera de la siguiente manera:

- **Emisión del Token:** Tras la validación exitosa de las credenciales (Login) en el endpoint /auth/login, el backend genera y devuelve un token JWT firmado.
- **Almacenamiento Local:** El frontend, mediante un contexto global de React (AuthContext), almacena este token de manera persistente en la memoria del dispositivo utilizando AsyncStorage.
- **Inyección Automática (Interceptores):** Para aislar la lógica de autenticación del resto de las peticiones, se configuró una instancia personalizada de la librería Axios. Se implementó

un interceptor de peticiones que adjunta automáticamente el token en la cabecera HTTP (Authorization: Bearer <token>) para todas las solicitudes dirigidas a rutas protegidas.

- **Validación de Origen:** El servidor Spring Boot, configurado con JwtRequestFilter, intercepta cada petición entrante, verifica la firma y vigencia del token, y extrae la identidad del usuario para autorizar la transacción.
- **Flujo de Datos, Sincronización y Consumo de API**

Para garantizar que la interfaz de usuario refleje siempre el estado actual de la base de datos, se implementaron diversas estrategias de consumo de datos:

 - **Manejo Asíncrono y Feedback:** Todas las transacciones de red manejan estados de carga (loading) y captura de excepciones (catch), retroalimentando al usuario en tiempo real a través del componente global CustomAlert.
 - **Sincronización de Vistas:** Se emplea el hook useFocusEffect de React Navigation en pantallas críticas (como el listado de mascotas o la agenda). Esto asegura que se ejecuten peticiones GET para refrescar la información cada vez que la pantalla vuelve a estar en primer plano.
 - **Integridad Estructural (DTOs):** Las peticiones de escritura (POST, PUT) envían objetos JSON que mapean exactamente con los Data Transfer Objects (DTO) requeridos por el backend. Esto previene errores de parseo y asegura la integridad de los datos.
 - **Estandarización de Zonas Horarias:** Se implementó una capa de formateo de fechas en el frontend antes de la serialización de datos. Esto asegura que las fechas (como el cumpleaños de una mascota) se envíen en formato local puro (ej. YYYY-MM-DD), evitando desfases de días provocados por la conversión automática a la zona horaria del servidor (UTC/GMT-4).
- **Integraciones Híbridas con Servicios de Terceros**

La arquitectura delega procesos complejos a servicios externos para optimizar la carga del servidor principal:

 - **Gestión Multimedia (Cloudinary):** El almacenamiento de imágenes (fotos de perfil y mascotas) no satura el backend. El flujo consiste en que el frontend captura la imagen y la sube directamente a la API de Cloudinary mediante un objeto FormData. Cloudinary procesa el archivo y retorna una URL segura (secure_url), la cual es finalmente enviada e indexada en la base de datos del backend a través de un endpoint ligero (ej. PUT /v1/usuarios/perfil/foto).
 - **Pasarela de Pagos (Flow):** El agendamiento transaccional requiere una validación externa. Al solicitar una reserva (POST /v1/reservas), el backend registra el intento de agendamiento en base de datos y se comunica con la API de Flow para generar una orden de pago. El backend responde al móvil con la urlPago generada, y el frontend utiliza el módulo Linking para redirigir al usuario al navegador web nativo del dispositivo, permitiendo completar la transacción de forma segura fuera del entorno de la app.

Fase 2.4: Métricas del proyecto

Para evaluar la calidad del software construido y asegurar que la plataforma cumpla con los estándares técnicos exigidos, el desempeño del proyecto se mide a través de las siguientes métricas, basadas en los Requisitos No Funcionales (RNF) y la metodología de desarrollo:

- Cobertura de Pruebas (Testing):
 - Se estableció como métrica de calidad alcanzar al menos un 80% de cobertura del código fuente. Esto se verifica a través de pruebas unitarias y de integración construidas con JUnit y Mockito.
- Rendimiento y Latencia (Performance):
 - El sistema debe garantizar un tiempo de respuesta ágil. Se mide que la latencia para consultar y visualizar el historial clínico completo o los dashboards no exceda los 2 segundos, debiendo mantener esta estabilidad incluso bajo un estrés de 500 usuarios concurrentes.
- Velocidad de Integración y Despliegue (CI/CD):
 - El tiempo total medido desde que inicia el proceso de compilación del código hasta que la aplicación se encuentra operativa y respondiendo en el entorno de producción no debe superar los 15 minutos.
- Disponibilidad y Conmutación (Uptime):
 - El sistema debe mantenerse operativo casi ininterrumpidamente. Se medirá que las caídas no planificadas sumen un máximo de 43 minutos al mes, para no sobrepasar las 9 horas anuales de inaccesibilidad permitidas. La recuperación ante fallas se gestiona mediante el reinicio automático del contenedor de la aplicación utilizando la política restart: unless-stopped de Docker sobre una única instancia en la nube (EC2). Esto garantiza que, si el proceso del backend se detiene de forma inesperada, el servicio intente levantarse automáticamente de inmediato.
- Cumplimiento de Sprints y Avance Ágil:
 - El rendimiento del equipo se mide mediante el cumplimiento de los objetivos iterativos trazados en Trello, validando que el desarrollo de los Sprints (1 al 4) se ejecute dentro del plazo global de 4 a 5 meses asignado al proyecto.

Etapa 3: Pruebas del software

Fase 3.1: Enfoque:

Las pruebas realizadas en este primer punto de la etapa 3, corresponden en su totalidad a pruebas funcionales manuales, ejecutadas en dos niveles complementarios. En el primer nivel se validó el comportamiento de la API REST del backend de forma aislada, verificando que cada endpoint respondiera con los códigos HTTP correctos, que las reglas de negocio se aplicaran adecuadamente y que el sistema manejara correctamente tanto los flujos exitosos como los casos de error. En el segundo nivel se validó el comportamiento de las interfaces de usuario, tanto en la aplicación móvil como en la web, comprobando que los flujos de interacción, las validaciones en cliente y la comunicación con el backend funcionaran de manera integrada y coherente desde la perspectiva del usuario final.

Este enfoque fué elegido considerando que las funcionalidades implementadas en el proyecto de NodeVet giran en torno a formularios de registro, autenticación, y gestión de datos, los cuales se prestan naturalmente para la validación manual e iterativa. En este tipo de aplicaciones, la prioridad es asegurar que las validaciones de entrada se apliquen correctamente, que los mensajes de error sean claros y oportunos, y que los flujos completen su ciclo de forma coherente entre el frontend y el backend. Las pruebas fueron diseñadas a partir de los casos de uso definidos en la etapa de análisis, cubriendo tanto los caminos exitosos como los escenarios de error más relevantes de cada módulo.

Fase 3.2: Plan de pruebas funcionales

1. Tipos de pruebas aplicadas

Dada la naturaleza del equipo y los tiempos del proyecto, se optó por **pruebas manuales** como estrategia principal de validación, descartando por ahora la automatización (unit testing) para esta ronda. Esto se debe a que el enfoque de esta etapa estuvo puesto en verificar el comportamiento funcional end-to-end de cada módulo (backend, frontend web y frontend móvil) antes de invertir tiempo en suites automatizadas, las cuales se contemplan para una fase posterior una vez que los flujos de negocio (agendamiento, pagos) estén completamente estabilizados.

Las pruebas se dividieron en dos enfoques según el componente:

- **Backend:** pruebas manuales de API, ejecutadas directamente sobre los endpoints REST mediante **Thunder Client** y **Swagger UI**, verificando códigos de respuesta HTTP, estructura de los payloads JSON, validaciones de negocio y manejo de errores.
- **Frontend (Web y Móvil):** pruebas manuales de interfaz, enfocadas en el comportamiento de los formularios (registro, login, recuperación de contraseña, gestión de mascotas) desde la perspectiva del usuario final, validando validaciones visuales, mensajes de error y flujos de navegación.

2. Estrategia de asignación

Las pruebas se distribuyeron según la responsabilidad de desarrollo de cada integrante:

Responsable	Componente	Herramientas
Cruz Muñoz, Ricardo	Backend (API RESTful)	Thunder Client / Swagger UI
Aranda Cardenas, Catalina	Frontend Móvil	Dispositivo Android / Expo Go
Santander Leal, Sebastian	Frontend Web	Navegador / DevTools

Esta división permitió que cada responsable probara directamente el código que construyó, aprovechando el conocimiento previo de la lógica interna de su módulo para diseñar casos de prueba más certeros.

3. Cobertura de módulos

En el caso del backend, las pruebas no se limitaron a los módulos base (Autenticación y Mascotas), sino que se extendieron a **todos los endpoints disponibles a la fecha**, incluyendo Agendamiento/Reservas, Pagos y Consultas/Ficha Clínica, cubriendo así la totalidad de la API construida hasta este punto del proyecto.

En total se ejecutaron **90 casos de pruebas**, distribuidos en los siguientes módulos:

- Autenticación (Login, Registro, Recuperación de contraseña)
- Gestión de Mascotas (CRUD)
- Agendamiento / Reservas
- Pagos
- Veterinarios / Consultas
- Integración Pagos-Agendamiento

4. Criterios de aceptación

Un caso de prueba se considera **aprobado** cuando:

- El sistema responde con el código HTTP esperado según el escenario (éxito o error controlado).
- El cuerpo de la respuesta (JSON) contiene la estructura y los datos esperados.
- Las validaciones de negocio (formato, duplicados, autorización, límites de tamaño, etc.) se aplican correctamente.

- En el caso de frontend, la interfaz refleja visualmente el resultado de la operación (mensajes de error, redirecciones, actualización de listas) sin errores de consola.

Un caso se considera **fallido** cuando el comportamiento observado difiere de lo especificado en el criterio de aceptación del RF/RNF correspondiente.

Fase 3.3: Herramientas y Software utilizados

1. Thunder Client:

- Fué la herramienta principal para las pruebas del backend. Se trata de una extensión de Visual Studio Code que permite construir y enviar peticiones HTTP de forma manual, revisar los códigos de respuesta, los headers y el body del response. A través de ella se ejecutaron todos los casos de prueba de la API REST, incluyendo autenticación, gestión de mascotas, jornadas, bloques horarios, veterinarios, reservas y pagos.

2. Flow Sandbox:

- Utilizado para simular el flujo de pago en las pruebas relacionadas con la pasarela de pagos. Al ser el entorno de pruebas oficial de la plataforma Flow, permitió generar órdenes de pago reales y obtener tokens de transacción para verificar el funcionamiento del webhook sin procesar cobros reales.

3. MySql Workbench:

- Empleado como herramienta de verificación directa sobre la base de datos. En particular, se utilizó para confirmar el comportamiento del soft delete en la eliminación de mascotas, consultando directamente el campo “estado_masc” de la tabla “mascota” tras ejecutar la petición DELETE.

4. Consola del navegador (DevTools):

- Herramienta de apoyo principal durante las pruebas de frontend web. Permitted verificar las peticiones HTTP enviadas al backend, los códigos de respuesta obtenidos y los mensajes de error retornados, complementando la observación visual del comportamiento de la interfaz.

5. Dispositivo físico Android:

- Entorno de ejecución para las pruebas de frontend móvil, corriendo la aplicación desarrollada en React Native/Expo. Las pruebas fueron realizadas de forma manual interactuando directamente con la aplicación instalada en el dispositivo, y los resultados fueron evidenciados mediante capturas de pantalla.

Fase 3.4: Requisitos involucrados

- Requisitos Funcionales:

Código	Nombre	Caso de prueba asociado
RF-01	Crear Perfil Tutor	CP 19 - CP 20 - CP 21 - CP 22 - CP 23 - CP 44 - CP 45 - CP 46 - CP 47 - CP 48
RF-02	Modificar Perfil	CP 73 - CP 74 - CP 75 - CP 76
RF-03	Adjuntar Imagen	CP 33 - CP 34 - CP 58 - CP 59
RF-04	Inhabilitar Perfil	CP 77 - CP 78 - CP 79
RF-05	Crear Perfil Médico	CP 12 - CP 13 - CP 14 - CP 65 - CP 66 - CP 67 - CP 68 - CP 69 - CP 70
RF-06	Iniciar Sesión	CP 1 - CP 2 - CP 24 - CP 25 - CP 49 - CP 50 - CP 86
RF-07	Recuperar Contraseña	CP 26 - CP 27 - CP 28 - CP 29 - CP 30 - CP 31 - CP 51 - CP 52 - CP 53 - CP 54 - CP 55 - CP 56
RF-08	Agregar Mascota	CP 3 - CP 4 - CP 5 - CP 32 - CP 33 - CP 34 - CP 35 - CP 57 - CP 58 - CP 59
RF-09	Editar Mascota	CP 6 - CP 36 - CP 37 - CP 61 - CP 62
RF-12	Ver Calendario Disponible	CP 40 - CP 41
RF-13	Iniciar Agendamiento	CP 15 - CP 16 - CP 42 - CP 43
RF-14	Dashboard Médico	CP 86
RF-16	Registrar Evolución Médica	CP 87 - CP 88 - CP 89 - CP 90
RF-18	Eliminar Mascota	CP 7 - CP 38 - CP 39 - CP 63 - CP 64
RF-19	Confirmar Pago / Webhook	CP 17 - CP 18
RF-20	Generar Agenda (Jornadas y Bloques)	CP 8 - CP 9 - CP 10 - CP 11 - CP 80 - CP 81 - CP 82 - CP 83 - CP 84 - CP 85
RF-21	Crear Especialidad	CP 71 - CP 72

- Requisitos No Funcionales:

Código	Nombre	Caso de prueba asociado
RNF-1.1	Bloqueo Temporal de Espacio en Agenda	CP 16 - CP 43
RNF-2.1	Hashing Unidireccional de Contraseñas	CP 1 - CP 2 - CP 24 - CP 25 - CP 49 - CP 50

RNF-2.2	Autenticación Segura mediante Tokens JWT	CP 1 - CP 2 - CP 5 - CP 15 - CP 24 - CP 25 - CP 49 - CP 50
RNF-3.2	Flujo Guiado de Recuperación de Contraseña	CP 26 - CP 27 - CP 28 - CP 29 - CP 30 - CP 31 - CP 51 - CP 52 - CP 53 - CP 54 - CP 55 - CP 56
RNF-3.3	Accesibilidad y Claridad Visual en el Calendario	CP 40 - CP 41
RNF-4.1	Compatibilidad Multiplataforma y Diseño Responsivo	CP 19 al CP 43 (móvil) - CP 44 al CP 90 (web)
RNF-6.2	Comunicación Segura con Pasarela de Pagos	CP 16 - CP 17 - CP 18
RNF-6.3	Integración Automatizada de Correo Electrónico (SMTP)	CP 26 - CP 51
RNF-7.1	Control de Concurrencia en Reserva de Citas	CP 16 - CP 43
RNF-7.2	Auditoría y Soft Delete en Expedientes	CP 7 - CP 39 - CP 64 - CP 77 - CP 78 - CP 79
RNF-7.3	Validación de Integridad de Datos Numéricos en Backend	CP 4
RNF-11.1	Reversión Automática de Citas por Fallo de Pago	CP 18
RNF-11.3	Sanitización de Entradas y Prevención de Inyecciones	CP 4 - CP 6
RNF-12.2	Desacoplamiento del Proveedor de Pagos	CP 16 - CP 17 - CP 18
RNF-13.2	Encapsulación de Validaciones Transversales	CP 19 - CP 22 - CP 23 - CP 44 - CP 47 - CP 48
RNF-13.3	Manejo Global e Interceptado de Excepciones	CP 4 - CP 6 - CP 13 - CP 68
RNF-15.1	Cobertura Mínima de Pruebas	CP 1 al CP 90

Fase 3.5: Casos de pruebas

A continuación se detallan los casos de prueba ejecutados sobre la plataforma NodeVet, abarcando los tres componentes del sistema: backend, frontend web y frontend móvil. En total se ejecutaron 90 casos de prueba, distribuidos según el alcance funcional de cada componente y organizados por módulo (Autenticación, Mascotas, Agendamiento, Gestión de Veterinarios y Pagos para el backend; formularios y flujos de interfaz para los frontend web y móvil). Cada caso fue ejecutado de forma manual, contrastando el resultado obtenido contra el comportamiento esperado según los requisitos funcionales y no funcionales definidos en la Etapa 1.

El detalle completo de la ejecución, incluyendo las peticiones, respuestas e interfaces evaluadas para cada caso, se encuentra disponible en el siguiente enlace: [Planilla Casos de Prueba NodeVet.xlsx](#)

1. Backend

Entorno de ejecución: Backend Spring Boot en **localhost:8080**, Thunder Client como cliente HTTP, MySQL para verificación directa en base de datos y Sandbox de Flow para transacciones de pago simuladas. Imágenes de evidencia sobre las pruebas: [Capturas pruebas backend](#)

Módulo 1: Autenticación

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-01	Inicio de sesión con credenciales válidas (Tutor/Admin/Veterinario)	RF-06 / RNF 2.1, 2.2	Autenticación correcta y generación de token JWT válido	✓
CP-02	Registro de nuevo usuario	RF-01 / RF-05	Registro correcto con contraseña cifrada (BCrypt)	✓

Módulo 2: Mascotas

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-03	Registro de mascota con datos válidos	RF-08	Mascota creada y asociada al ID del tutor	✓
CP-04	Registro de mascota con peso negativo	RNF 7.3	Rechazo de la petición con HTTP 400 y mensaje de validación	✓
CP-05	Edición de mascota propia	RF-09	Actualización correcta de los datos editables	✓
CP-06	Tutor intenta modificar mascota de otro Tutor (Anti-IDOR)	RNF 2.2	Rechazo de la petición con HTTP 403 Forbidden	✓
CP-07	Eliminación lógica (soft delete) de mascota	RF-04 / RNF 7.2	Mascota marcada como inactiva, sin borrado físico	✓

Módulo 3: Agendamiento

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-08	Consulta de disponibilidad horaria (bloques)	RF-12	Listado correcto de bloques disponibles	✓
CP-09	Tutor intenta crear jornada laboral (acción exclusiva de Admin)	RNF 2.2	Rechazo de la petición por falta de permisos	✓
CP-10	Generación de bloques horarios mensuales	RNF 1.1 / RNF 7.1	Creación de bloques sin duplicados para el periodo procesado	✓
CP-11	Creación de reserva con datos válidos	RF-13	Reserva registrada y bloque marcado como ocupado	✓

Módulo 4: Gestión de Veterinarios

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-12	Registro de veterinario con datos válidos	RF-05	Cuenta de veterinario creada correctamente	✓
CP-13	Registro de veterinario con especialidad asociada	RF-05	Especialidad registrada y vinculada al perfil	✓
CP-14	Registro de veterinario con correo ya existente	RF-05	Rechazo de la petición con HTTP 409 Conflict	✓
CP-15	Usuario no-Admin intenta crear cuenta de veterinario	RNF 2.2	Rechazo de la petición por falta de permisos	✓

Módulo 5: Pagos

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-16	Generación de orden de pago hacia Flow (Sandbox)	RNF 6.2	Orden generada correctamente en formato JSON sobre HTTPS	✓
CP-17	Confirmación de pago exitoso vía webhook	RF-13 / RNF 7.1	Reserva actualizada a estado "Agendada" tras confirmación	✓
CP-18	Manejo de pago rechazado o fallido	RNF 11.1	Bloque horario revertido a "Disponible" automáticamente	✓

2. Frontend Móvil:

Entorno de ejecución: Aplicación React Native/Expo ejecutada en dispositivo físico Android, conectada al backend Spring Boot en localhost:8080. Las pruebas fueron ejecutadas de forma manual interactuando directamente con la aplicación instalada en el dispositivo. Evidencias: [Frontend Movil](#)

Módulo 1: Autenticación

ID	Descripción	Requisito Asociado	Resultado Esperado	Resultado Obtenido	Estado
CP-19	Registro con contraseña sin complejidad mínima	RF-1 RNF 3.2	El sistema NO envía petición HTTP. Muestra lista de errores a corregir	Éxito	✓
CP-20	Registro con correo ya existente	RF-1	El backend retorna HTTP 409 Conflict. El frontend muestra mensaje informando que el correo ya tiene cuenta registrada	Éxito	✓
CP-21	Registro exitoso	RF-1 RNF 3.2	Aparece el modal con el mensaje de éxito y el botón para ir al Login	Éxito	✓
CP-22	Registro con celular en formato incorrecto	RF-1 RNF 13.2	Aparece mensaje bajo el campo indicando el formato correcto y el botón "Crear Cuenta" no funciona	Éxito	✓
CP-23	Registro con nombre menor a 3 caracteres	RF-1 RNF 13.2	Aparece el mensaje "Mínimo 3 caracteres" bajo el campo y el botón "Crear Cuenta" no funciona	Éxito	✓
CP-24	Login con contraseña incorrecta	RF-06 RNF 2.2	El backend retorna HTTP 401. El frontend muestra modal con mensaje de credenciales incorrectas	Éxito	✓
CP-25	Login exitoso	RF-06 RNF 2.2	Se ingresa al HomeScreen	Éxito	✓
CP-26	Solicitar código de recuperación con correo registrado	RF-07 RNF 3.2	Se muestra mensaje de confirmación del envío del código y se habilita el campo para ingresarlo	Éxito	✓

CP-27	Verificación del código de recuperación correcto	RF-07 RNF 3.2	Redirige a la pantalla para ingresar la nueva contraseña y su confirmación	Éxito	✓
CP-28	Verificación del código de recuperación erróneo	RF-07	Aparece el modal informando que el código es incorrecto	Éxito	✓
CP-29	Verificar largo mínimo del código de verificación	RF-07 RNF 3.2	El botón "Verificar" se mantiene desactivado	Éxito	✓
CP-30	Restablecimiento exitoso de contraseña	RF-07 RNF 3.2	Aparece el modal de éxito y al aceptar redirige al inicio de sesión	Éxito	✓
CP-31	Restablecimiento con contraseñas que no coinciden	RF-07 RNF 3.2	El campo "Confirmar contraseña" se rodea de rojo y el botón "Guardar" se mantiene inactivo	Éxito	✓

Módulo 2: Mascotas

ID	Descripción	Requisito Asociado	Resultado Esperado	Resultado Obtenido	Estado
CP-32	Registro con campos obligatorios vacíos	RF-08 RNF 13.2	Aparece un modal indicando que se deben completar los campos obligatorios, impidiendo la petición POST	El sistema simplemente no hace nada: no muestra modal ni mensaje de error alguno	✗
CP-33	Registro de mascota sin foto	RF-08 RF-03	La mascota se registra correctamente y en la PetCard se muestra sin imagen	Éxito	✓
CP-34	Registro de mascota válido	RF-08 RF-03	Se retrocede a "Mis Mascotas" con un modal de éxito	Redirige correctamente a "Mis Mascotas" pero no muestra el modal de éxito	✗

CP-35	Campo "Raza" bloqueado hasta seleccionar "Especie"	RF-08	La opción de raza se mantiene desactivada hasta que se selecciona una especie	Éxito	✓
CP-36	Edición exitosa de mascota	RF-09 RNF 13.2	Retrocede a la información básica de la mascota con un modal de éxito	No muestra modal de confirmación ni actualiza la pantalla de forma inmediata; el usuario debe retroceder manualmente para ver los cambios	✗
CP-37	Editar con el nombre en blanco	RF-09 RNF 13.2	Aparece un modal indicando que el nombre de la mascota no puede quedar en blanco	No aparece modal de ningún tipo; el botón "Guardar Cambios" simplemente no funciona	✗
CP-38	Cancelar la eliminación desde el modal de confirmación	RF-18	La PetCard se mantiene en el listado	Éxito	✓
CP-39	Confirmar la eliminación desde el modal de confirmación	RF-18 RNF 7.2	La PetCard desaparece del listado	La eliminación se ejecuta en BD correctamente (soft delete) pero la PetCard no desaparece del listado hasta recargar la sección manualmente	✗

Módulo 3: Agendamiento

ID	Descripción	Requisito Asociado	Resultado Esperado	Resultado Obtenido	Estado
CP-40	El calendario no permite seleccionar fechas anteriores a la actual	RF-12 RF-13	Las fechas anteriores permanecen inactivas desde el momento de acceder a la pantalla	Éxito	✓
CP-41	El calendario muestra profesionales con bloques horarios disponibles	RF-12	Los días sin disponibilidad aparecen vacíos; los que sí tienen muestran al profesional con las horas disponibles	Éxito	✓
CP-42	Agendar sin seleccionar una mascota	RF-13	El botón "Continuar" se mantiene deshabilitado	Éxito	✓
CP-43	El modal de confirmación muestra la información correcta del agendamiento	RF-13 RNF 3.2	El modal muestra la información seleccionada en el agendamiento	Éxito	✓

3. Frontend Web

Entorno de ejecución: aplicación web (React + Vite) servida en localhost:5173 con proxy /api hacia el backend en localhost:8080, validada en navegador apoyándose en la consola de DevTools para inspeccionar las peticiones HTTP, los códigos de respuesta y los mensajes de error retornados. Las pruebas abarcan los formularios y flujos de interfaz de los tres roles del sistema (Tutor, Administrador y Veterinario), cubriendo tanto los caminos exitosos como los escenarios de validación y error. En total se ejecutaron 46 casos (CP-44 a CP-90), todos con resultado aprobado.

Módulo 1: Autenticación (Registro, Login y Recuperación)

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-44	Registro con contraseña sin complejidad mínima	RF-01 / RNF-13.2	El sistema no envía la petición HTTP y muestra los requisitos de contraseña pendientes	✓
CP-45	Registro con correo ya existente	RF-01	El backend responde HTTP 409 Conflict y el formulario informa que el correo ya tiene cuenta	✓
CP-46	Registro exitoso con datos válidos	RF-01	Usuario creado; se muestra el modal de éxito con acceso al Login	✓
CP-47	Registro con celular en formato incorrecto	RF-01 / RNF-13.2	Mensaje bajo el campo con el formato correcto; el botón no envía la petición	✓
CP-48	Registro con nombre de menos de 3 caracteres	RF-01 / RNF-13.2	Mensaje "Mínimo 3 caracteres" bajo el campo; se bloquea el envío	✓
CP-49	Login con contraseña incorrecta	RF-06 / RNF-2.1, 2.2	El backend responde HTTP 401 y el frontend muestra un mensaje de credenciales incorrectas	✓
CP-50	Login exitoso	RF-06 / RNF-2.1, 2.2	Autenticación correcta y redirección al dashboard según el rol	✓
CP-51	Solicitar código de recuperación con correo registrado	RF-07 / RNF-3.2	HTTP 200; mensaje de envío del código y habilitación del campo para ingresarlo	✓
CP-52	Verificación del código de recuperación válido	RF-07 / RNF-3.2	HTTP 200; redirige a la pantalla de nueva contraseña	✓
CP-53	Verificación del código de recuperación erróneo	RF-07 / RNF-3.2	HTTP 400; mensaje "Código inválido o expirado" sin avanzar	✓

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-54	Código de recuperación con menos de 6 dígitos	RF-07 / RNF-3.2	Mensaje "El código debe contener 6 dígitos numéricos"; no se envía la petición	✓
CP-55	Restablecimiento exitoso de contraseña	RF-07 / RNF-3.2	HTTP 200; modal de éxito y redirección al inicio de sesión	✓
CP-56	Restablecimiento con contraseñas que no coinciden	RF-07 / RNF-3.2	Mensaje "Las contraseñas no coinciden"; no se envía la petición	✓

Módulo 2: Gestión de Mascotas (Tutor)

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-57	Registro de mascota con campos obligatorios vacíos	RF-08	Se exige completar los campos obligatorios y se impide la petición POST	✓
CP-58	Registro de mascota sin foto	RF-08 / RF-03	La mascota se crea y la PetCard muestra el placeholder "Sin foto"	✓
CP-59	Registro de mascota válido (con foto)	RF-08 / RF-03	Mascota creada; se vuelve a "Mis Mascotas" con modal de éxito	✓
CP-61	Edición de mascota exitosa	RF-09	Datos actualizados; se vuelve a la ficha con modal de éxito	✓
CP-62	Edición de mascota con el nombre en blanco	RF-09	Alerta en el campo vacío; no se envía la petición	✓
CP-63	Cancelar la eliminación desde el modal de confirmación	RF-18	La PetCard se mantiene; no se realiza la petición	✓
CP-64	Confirmar la eliminación (soft delete) de una mascota	RF-18 / RNF-7.2	La PetCard desaparece del listado de mascotas activas	✓

Módulo 3: Gestión de Veterinarios y Especialidades (Administrador)

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-65	Crear veterinario con contraseña sin complejidad mínima	RF-05	Mensaje con los requisitos bajo el campo; no se envía la petición POST	✓
CP-66	Crear veterinario con RUN inválido	RF-05	Mensaje "El RUN debe ser numérico y tener al menos 7 dígitos"; se bloquea el envío	✓

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-67	Crear veterinario con dígito verificador inválido	RF-05	El sistema no permite más de 1 carácter en el DV; se bloquea el envío	✓
CP-68	Crear veterinario sin seleccionar especialidad	RF-05 / RNF-13.3	HTTP 400; el formulario exige asignar al menos una especialidad	✓
CP-69	Crear veterinario con correo ya existente	RF-05	HTTP 409 Conflict; el formulario informa que el correo ya está registrado	✓
CP-70	Registro de veterinario exitoso	RF-05	HTTP 201; se cierra el modal y la tabla de usuarios se actualiza	✓
CP-71	Crear especialidad duplicada	RF-21	Mensaje "Esa especialidad ya existe"; no se agrega a la lista	✓
CP-72	Crear una nueva especialidad	RF-21	HTTP 201; la especialidad se agrega y queda seleccionada automáticamente	✓

Módulo 4: Gestión de Usuarios (Administrador)

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-73	Modificar usuario con nombre o apellido vacío	RF-02	Mensaje de campo obligatorio; se bloquea el envío (sin petición PUT)	✓
CP-74	Modificar usuario con correo en formato inválido	RF-02	Mensaje de validación de formato; no se realiza la petición PUT	✓
CP-75	Modificar usuario a un correo ya en uso por otra cuenta	RF-02	HTTP 400; el formulario informa que el correo ya está en uso	✓
CP-76	Modificación de usuario exitosa	RF-02	HTTP 200; se cierra el modal y la tabla refleja los datos actualizados	✓
CP-77	El administrador no puede eliminar su propia cuenta	RF-04 / RNF-7.2	El botón "Eliminar" de la propia fila aparece deshabilitado	✓
CP-78	Cancelar la eliminación desde el cuadro de confirmación	RF-04 / RNF-7.2	El usuario se mantiene en la lista; no se realiza la petición DELETE	✓
CP-79	Confirmar la eliminación (soft delete) de un usuario	RF-04 / RNF-7.2	HTTP 204; la cuenta se desactiva y desaparece del listado de activos	✓

Módulo 5: Generación de Agenda (Administrador)

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-80	Guardar jornada con ID de veterinario inválido o vacío	RF-20	Mensaje "Ingresa un ID de veterinario válido"; no se envía la jornada	✓
CP-81	Guardar jornada con hora de inicio igual o posterior a la de fin	RF-20	Mensaje "La hora de inicio debe ser anterior a la hora de fin"; no se envía	✓
CP-82	Guardar jornada sin días laborales seleccionados	RF-20	Mensaje "Selecciona al menos un día laboral"; no se envía la jornada	✓
CP-83	Guardar jornada laboral exitosamente	RF-20	HTTP 201 por cada día; mensaje de jornada guardada lista para generar bloques	✓
CP-84	Generar bloques horarios exitosamente	RF-20	HTTP 201; mensaje "Se generaron N bloque(s)" para el mes seleccionado	✓
CP-85	Generar bloques sin haber guardado la jornada	RF-20	HTTP 400; mensaje indicando que no existe jornada base para el veterinario	✓

Módulo 6: Acceso y Consultas del Veterinario

ID	Descripción	Requisito Asociado	Resultado Esperado	Estado
CP-86	Login de veterinario exitoso	RF-06 / RF-14	HTTP 200 con rol VETERINARIO; redirección al dashboard del médico	✓
CP-87	Cerrar consulta sin ID de reserva	RF-16	Mensaje "Ingresa el ID de la reserva que estás atendiendo"; no se envía	✓
CP-88	Cerrar una consulta vacía (sin diagnóstico, notas ni receta)	RF-16	Mensaje que exige al menos un campo clínico; se bloquea el envío	✓
CP-89	Cierre de consulta exitoso	RF-16	HTTP 201; mensaje "Consulta cerrada exitosamente" y formulario limpio	✓
CP-90	Cerrar consulta con ID de reserva inexistente o no pagada	RF-16	Error 4xx; se muestra el mensaje del backend sin cerrar la consulta	✓

[Evidencias de pruebas](#)

Etapa 4: Cierre Final del proyecto

Fase 4.1: Estado final del producto:

Al cierre del proyecto, NodeVet se encuentra en un estado funcional y desplegable, con los tres componentes de la arquitectura (backend, frontend web y aplicación móvil) operativos y comunicándose entre sí a través de una API REST común. El sistema cubre el ciclo completo del negocio: registro y autenticación de usuarios, gestión de mascotas, agendamiento de horas con integración de pago real, atención clínica veterinaria con control de asistencia, y administración de veterinarios, jornadas y usuarios. A continuación se detalla el estado de cada componente.

Backend (Spring Boot / MySQL):

- El backend expone la totalidad de la API REST consumida por ambos frontends, organizada en controladores por dominio: autenticación (AuthController), usuarios (UsuarioController), mascotas (MascotaController), veterinarios (VeterinarioController), especialidades (EspecialidadController), agenda y jornadas (AgendaController, JornadaController), reservas (ReservaController), pagos (PagoController), consultas clínicas (ConsultaController) y administración (AdminController). El modelo de datos implementa más de 13 entidades en tercera forma normal, con borrado lógico (is_deleted / estado_*) en las tablas críticas y control de concurrencia sobre los bloques horarios para evitar doble reserva.
- La seguridad está resuelta con Spring Security y JWT: el login diferencia la duración del token según el origen de la sesión (8 horas para web, 30 días para móvil), y un filtro de renovación automática extiende la sesión en cada petición válida agregando el header new-token. La recuperación de contraseña funciona mediante un flujo de tres pasos con código OTP de 6 dígitos y expiración de 15 minutos. Todas las contraseñas se almacenan con BCrypt y las validaciones de entrada se aplican a nivel de DTO mediante anotaciones de Jakarta Validation (@NotBlank, @DecimalMin, etc.), devolviendo errores estructurados a través de un GlobalExceptionHandler centralizado.
- La integración de pagos con Flow está completamente operativa: al crear una reserva, el backend genera la orden de pago y retorna la URL de checkout en la misma respuesta; un webhook (/api/v1/pagos/webhook) recibe y valida la confirmación de la transacción de forma automática, actualizando el estado de la reserva sin intervención manual. El almacenamiento de archivos (fotos de perfil, de mascotas y adjuntos médicos como exámenes o radiografías) se resuelve íntegramente a través de Cloudinary; el backend solo persiste la URL resultante.

Frontend Web (React + TypeScript):

- El portal web cubre los tres roles del sistema mediante rutas y dashboards diferenciados: DashboardTutor (agendamiento, mascotas, perfil), DashboardMedico (agenda del día, atención de consultas) y DashboardAdmin (gestión de veterinarios, generación de agenda, configuración de precio). El flujo de agendamiento (AgendarHora.tsx) implementa un calendario mensual propio y se integra con Flow de la misma forma que la app móvil, incluyendo la pantalla PagoResultado.tsx para

procesar el retorno desde la pasarela de pago. La gestión de veterinarios (`GestionVeterinarios.tsx`, `DetalleVeterinario.tsx`, `GenerarAgenda.tsx`) permite al administrador crear profesionales, configurar su jornada laboral y generar los bloques horarios mensuales derivados de esa jornada.

Aplicación Móvil (React Native / Expo):

- La app móvil implementa navegación diferenciada por rol mediante tres Bottom Tab Navigators independientes: uno para el Tutor (5 pestañas: Mascotas, Ordenes, Home, Agenda, Perfil), uno para el Veterinario (4 pestañas: Inicio, Atención, Historial, Perfil) y uno para el Administrador (4 pestañas sin etiqueta: InicioAdmin, Veterinarios, Usuarios, PerfilAdmin), seleccionados automáticamente según el rol devuelto al autenticarse. El Tutor puede registrar y editar mascotas, consultar su historial clínico clasificado por tipo (consulta, vacuna, examen, cirugía), agendar horas con el mismo flujo de pago vía Flow (usando expo-linking para abrir y retornar del checkout), y revisar sus órdenes médicas. El Veterinario cuenta con su agenda diaria, registro de evolución médica y un mecanismo explícito para marcar la asistencia del paciente ("No se presentó (Ausente)"), que es el que determina si la reserva cierra como Completada o Ausente. El Administrador puede gestionar veterinarios, configurar jornadas y bloques horarios, y activar/desactivar cuentas de tutores.

Estado por módulo:

- **Completo y funcional:** autenticación y recuperación de contraseña (los 3 roles), gestión de mascotas y su historial clínico, agendamiento con pago real vía Flow, registro de consultas con control de asistencia, gestión de veterinarios/jornadas/bloques horarios, gestión de usuarios (activar/desactivar) desde el rol Administrador.
- **Parcial o simplificado:** el perfil del Administrador en la app móvil (`PerfilAdminScreen`) es aún una pantalla mínima, sin edición de datos ni foto, a diferencia de los perfiles de Tutor y Veterinario; la configuración de precio de la cita en el panel web (`ConfigPrecio.tsx`) es actualmente informativa y se guarda en `localStorage` del navegador, sin persistir en el backend ni afectar el monto real cobrado por Flow (el monto real proviene de la tabla `VALOR`, congelado en la reserva al momento de la creación).
- **No implementado / pendiente:** pruebas automatizadas de cobertura relevante (solo existe un test de repositorio; el resto de la validación del sistema se realizó de forma manual mediante el plan de 90 casos de prueba documentado en la Fase 3), y una infraestructura de alta disponibilidad (el despliegue actual corre en una única instancia EC2 con reinicio automático de contenedor vía Docker, sin balanceo de carga ni instancia de respaldo).

Fase 4.2: Atributos de calidad consolidados:

Esta sección resume, para cada atributo de calidad definido en los Requisitos No Funcionales (Fase 1), cuál es el estado real y verificable al cierre del proyecto, contrastando lo planificado con lo efectivamente implementado en el código.

Seguridad:

El sistema implementa autenticación basada en JWT con Spring Security, contraseñas cifradas con BCrypt, y un mecanismo de recuperación de contraseña mediante código OTP de 6 dígitos con expiración de 15 minutos. La duración del token se diferencia según el cliente (8 horas en web, 30 días en móvil) al momento del login, aunque la renovación automática del token en peticiones sucesivas aplica siempre la duración de 30 días, sin distinguir el origen de la sesión original — un punto a corregir en una futura iteración. La política de contraseñas (mínimo 6 caracteres, mayúscula, minúscula y carácter especial) se valida únicamente en el cliente (formularios de registro web y móvil); el backend no repite esta validación a nivel de DTO, por lo que la robustez de la contraseña depende actualmente de que la petición provenga de los clientes oficiales. El flujo de recuperación de contraseña protege correctamente contra enumeración de usuarios en su primer paso (solicitud de código), aunque el segundo paso (verificación de código) sí revela si un correo está registrado ante una solicitud inválida, quedando como mejora pendiente. No se implementó control de rate-limiting ni bloqueo por intentos fallidos de login.

Rendimiento:

No se ejecutaron pruebas de carga ni de estrés formales (herramientas como JMeter o k6) durante el proyecto; las metas de tiempo de respuesta y concurrencia definidas en la etapa de planificación no cuentan con evidencia empírica que las respalde, por lo que se documentan como objetivos de diseño más que como resultados medidos. El rendimiento observado durante las pruebas manuales fue adecuado para el volumen de datos de prueba utilizado, sin incidentes de lentitud reportados en los casos de prueba ejecutados.

Disponibilidad y Fiabilidad:

El despliegue actual corre sobre una única instancia EC2 de AWS, con el backend y frontend web contenedorizados mediante Docker y la política de reinicio restart: unless-stopped, que reinicia automáticamente el contenedor ante una caída del proceso. No existe balanceador de carga, réplicas del backend, ni una instancia de respaldo activa; por lo tanto, la disponibilidad depende de un único punto de falla a nivel de infraestructura. La confirmación de pagos vía webhook de Flow funciona de manera asíncrona y desacoplada del flujo del usuario, lo que da cierta resiliencia ante caídas puntuales de conexión del cliente durante el proceso de pago.

Compatibilidad e Interoperabilidad:

El frontend web y el backend evitan por diseño la necesidad de configurar CORS: en desarrollo, Vite actúa como proxy inverso (/api → localhost:8080), y en producción Nginx cumple el mismo rol (/api/ → backend), de modo que ambas peticiones se resuelven bajo el mismo origen desde la perspectiva del navegador. Además,

se resolvió un problema real de interoperabilidad con la pasarela Flow: esta retorna al usuario mediante una petición POST a la URL de retorno, incompatible con una SPA que solo sirve contenido por GET; tanto Nginx (location /pago/resultado) como el middleware de desarrollo de Vite interceptan este POST y responden con una redirección 303 para forzar un GET válido. La app móvil, al no estar sujeta a la política de mismo origen de los navegadores, se conecta directamente a la IP del backend sin necesidad de estos ajustes.

Usabilidad:

Los tres roles del sistema (Tutor, Veterinario, Administrador) cuentan con navegación, paleta de colores y tipografía (Fredoka) consistentes en ambas plataformas (web y móvil), lo que da una identidad visual unificada. La app móvil incorpora patrones de usabilidad esperables (indicadores de carga en botones durante peticiones asíncronas, pull-to-refresh, validación de formularios en tiempo real con mensajes bajo cada campo). Como excepción, el perfil del rol Administrador en la app móvil es aún una pantalla mínima (solo título y cierre de sesión), sin el nivel de funcionalidad de edición que sí tienen los perfiles de Tutor y Veterinario.

Mantenibilidad:

El backend separa responsabilidades en capas (controller / service / repository / DTO), con manejo centralizado de errores mediante GlobalExceptionHandler y documentación de la API vía Swagger/OpenAPI (anotaciones @Operation en los controladores). La cobertura de pruebas automatizadas es mínima: solo existe un test de repositorio (UsuarioRepositoryTest), y el propio proceso de build del backend (Dockerfile) compila con -DskipTests, por lo que la validación de regresiones recae actualmente por completo en el plan de pruebas manuales de la Fase 3 (90 casos de prueba). Esto representa la principal deuda técnica del proyecto de cara a un mantenimiento a largo plazo.

Portabilidad:

El backend y el frontend web se distribuyen como imágenes Docker multi-stage, lo que permite desplegarlos en cualquier entorno compatible con contenedores sin depender de la configuración manual del sistema operativo anfitrión. La app móvil se distribuye como archivo APK generado con EAS Build y alojado en Amazon S3 para descarga directa, sin pasar por Google Play ni App Store, lo que limita su portabilidad de instalación a Android y a la distribución manual del archivo.

Almacenamiento de Archivos:

Todo el almacenamiento externo de imágenes y documentos (fotos de perfil, fotos de mascotas, exámenes y adjuntos médicos) se resuelve mediante Cloudinary, tanto desde el backend (que solo persiste la URL resultante) como desde ambos frontends. No se utiliza Amazon S3 para este propósito; su único uso real dentro del proyecto es como repositorio de descarga del instalador APK de la app móvil.

Cierre y Conclusión

Conclusión etapa 1:

El proyecto NodeVet nace como una respuesta directa a las ineficiencias del modelo veterinario tradicional. A lo largo de la planificación y estructuración arquitectónica detallada en este documento, se ha demostrado que la plataforma no solo moderniza el registro clínico mediante la digitalización centralizada, sino que ataca directamente un problema crítico del negocio: el ausentismo y la pérdida económica por citas no concretadas.

La decisión de implementar un sistema interactivo con bloqueo temporal y pago obligatorio, respaldado por tecnologías robustas (Java Spring Boot, React, AWS), garantiza a las clínicas veterinarias la rentabilidad de su tiempo médico. Al mismo tiempo, la entrega de una aplicación móvil nativa empodera a los dueños de mascotas, fidelizándolos y otorgándoles un control sin precedentes sobre la salud de sus animales.

Al finalizar los 4 meses de desarrollo, se entregará una solución de software escalable, segura y altamente disponible, cumpliendo a cabalidad con los objetivos trazados en el acta de constitución.

Conclusión etapa 2:

La etapa 2 del proyecto de NodeVet, enfocada en los “Cimientos del software”, ha concluido de manera exitosa, logrando establecer la primera parte del mismo, el cuál corresponde a la creación de usuarios (tutor) y que el mismo logre añadir alguna mascota. Además de los siguientes puntos:

- **Arquitectura Consolidada y Desacoplada:** Se implementó de manera efectiva la arquitectura en capas (N-Tier), logrando una separación estricta entre la capa de presentación desarrollada en React y React Native, la lógica de negocio construida sobre Spring Boot, y la persistencia de datos administrada por MySQL. Este diseño garantiza que los clientes (Web y Móvil) operen con total independencia del backend, lo que facilita el mantenimiento y habilita la escalabilidad horizontal mediante balanceadores de carga.
- **Seguridad y cumplimiento normativo:** El uso del algoritmo BCrypt para el hashing unidireccional de contraseñas y la implementación de JWT con interceptores para la renovación de sesiones aseguran un flujo de autenticación robusto. Asimismo, la integración de borrado lógico (soft delete) en la base de datos preserva la trazabilidad y la integridad referencial de los expedientes.
- **Eficiencia en la Integración Front-Back:** La comunicación entre las interfaces de usuario y el servidor se resolvió de manera óptima mediante Axios, configurando interceptores automatizados para la inyección y rotación de tokens. Destaca positivamente la estrategia de optimización para la carga de archivos multimedia, la cual delega la subida de imágenes directamente desde el navegador hacia Cloudinary, evitando así cuellos de botella en el servidor principal de Spring Boot y respetando las restricciones de tamaño.

- Preparación Operativa: Con el entorno de pruebas configurado (utilizando H2 para el aislamiento de tests), los flujos críticos de gestión de usuarios y el CRUD de mascotas desarrollados, y la API documentada a través de Swagger UI, el núcleo de NodeVet se encuentra tecnológicamente validado.

En síntesis, las decisiones de ingeniería adoptadas y ejecutadas en esta fase aseguran que la plataforma cuenta con la solvencia técnica necesaria para soportar las reglas de negocio más complejas de la siguiente etapa, como el agendamiento concurrente y la integración de la pasarela de pagos.

Conclusión etapa 3:

La etapa 3 del proyecto NodeVet, enfocada en las pruebas de software, permitió validar el comportamiento funcional de la plataforma a través de un proceso de pruebas manuales aplicado tanto al backend como a los frontend web y móvil. Mediante el uso de Thunder Client y Swagger UI para la verificación de los endpoints REST, y la validación directa de interfaces para los clientes web y móvil, se ejecutaron 90 casos de prueba que cubrieron la totalidad de los módulos disponibles a la fecha: Autenticación, Gestión de Mascotas, Agendamiento, Pagos y Consultas/Ficha Clínica.

Durante la ejecución se identificaron 5 bugs reales, los cuales fueron corregidos satisfactoriamente dentro de la misma ronda de trabajo, logrando que la totalidad de los módulos evaluados quedara con resultado aprobado al cierre de esta etapa. Este resultado refleja un nivel de madurez adecuado en la implementación actual del sistema, validando que las reglas de negocio críticas (como las validaciones de autenticación, el manejo de mascotas y los flujos de agendamiento y pago) operan conforme a los requisitos funcionales y no funcionales definidos en la planificación inicial del proyecto.

La estrategia de dividir las pruebas según la responsabilidad de desarrollo de cada integrante (backend, frontend web y frontend móvil) permitió una validación más eficiente, al aprovechar el conocimiento técnico de cada responsable sobre la lógica interna de su componente. Si bien las pruebas realizadas fueron exclusivamente manuales, este enfoque resultó suficiente para detectar y corregir oportunamente los errores funcionales más relevantes en esta fase del desarrollo.

De cara a la siguiente etapa, queda pendiente incorporar pruebas automatizadas (unitarias y de integración) que complementen la validación manual, así como extender la cobertura de pruebas a los módulos que aún se encuentran en estado de estructura inicial, como los dashboards de Veterinario y Administrador.

Conclusión final del proyecto:

Al cierre de las cuatro etapas de desarrollo, NodeVet se consolida como una solución de software funcional que cumple el objetivo central planteado desde el acta de constitución: combatir el ausentismo y la pérdida económica por citas no concretadas en clínicas veterinarias, mediante un sistema de agendamiento con bloqueo temporal de horarios e integración real de pago a través de la pasarela Flow. Lo que en la Etapa 1 se proyectó como una arquitectura N-Tier robusta sobre Java Spring Boot, React y React Native se materializó efectivamente en un producto donde los tres roles del negocio —Tutor, Veterinario y Administrador— operan de forma independiente tanto en el portal web como en la aplicación móvil nativa, compartiendo un mismo backend documentado con Swagger UI.

El núcleo de negocio quedó completamente implementado: el ciclo de vida de una cita —desde su agendamiento, pasando por la confirmación de pago vía webhook, hasta el registro de la evolución médica— incorpora el mecanismo de control de asistencia (marcado de "Ausente" por parte del veterinario) que constituye el diferencial estratégico del proyecto frente al modelo tradicional. A esto se suma la gestión completa de mascotas y su historial clínico, la administración de veterinarios, especialidades y jornadas laborales, y la generación automática de bloques horarios mensuales, todo respaldado por un modelo de datos normalizado con borrado lógico y control de concurrencia sobre los horarios.

En materia de seguridad, la autenticación JWT diferenciada por cliente, el cifrado de contraseñas con BCrypt y el flujo de recuperación mediante código OTP dotan a la plataforma de una base sólida, aunque quedan identificadas oportunidades de mejora puntuales —como la validación server-side de la política de contraseñas y el cierre completo de la protección contra enumeración de usuarios en el flujo de recuperación— que se documentan como trabajo futuro más que como riesgos bloqueantes para la entrega.

A diferencia de la proyección inicial de una infraestructura con balanceo de carga y alta disponibilidad activa, el despliegue final operó sobre una única instancia en AWS con reinicio automático de contenedores Docker, una decisión pragmática y suficiente para el alcance de un proyecto de título, pero que se reconoce como una simplificación respecto de lo planteado en la etapa de planificación. De igual manera, la validación de calidad se apoyó íntegramente en el plan de pruebas manuales ejecutado en la Etapa 3 —90 casos de prueba distribuidos por responsabilidad de desarrollo, con 5 bugs detectados y corregidos—, quedando la incorporación de pruebas automatizadas (unitarias y de integración) como la principal deuda técnica a resolver en una eventual continuidad del producto más allá del ámbito académico.

En síntesis, NodeVet cumple con los requisitos funcionales y no funcionales definidos como prioritarios en la planificación original, entregando una plataforma coherente, segura en sus aspectos esenciales y alineada con la problemática real del negocio veterinario. Las brechas identificadas entre lo planificado y lo finalmente implementado —infraestructura de disponibilidad, cobertura de pruebas automatizadas y algunas pantallas aún en estado inicial, como el perfil del Administrador en la app móvil— no comprometen la validez de la solución entregada, sino que trazan una hoja de ruta clara para su evolución posterior, evidenciando un cierre de proyecto maduro, autocrítico y fundamentado en el estado real del código por sobre lo únicamente proyectado.

Anexos

- **Uriol Paredes, R. M. (2016).**
 - Sistema de Información Veterinario Vía Web y Móvil para Mejorar la Gestión de Información del Área Veterinaria de la Subgerencia de Salud de la Municipalidad de Trujillo 2016
 - Respalda un enfoque de desarrollo dual (aplicaciones web y móviles), justificando cómo el acceso centralizado a la información clínica agiliza los diagnósticos.
- **Silva, J. (2021).**
 - El uso de aplicaciones Web para la Gestión de clínicas veterinarias y su incidencia en la mejora de procesos administrativos. Ecuadorian Science Journal, 5(Esp.4), 114-125
 - Fundamenta directamente cómo la transición hacia aplicaciones web modernas reduce la carga administrativa del personal del recinto.
 - En la lista de conclusiones, el autor estipula explícitamente los beneficios: "Genera información o reportes que permitirá analizar [...] detectar fallas y tener mejor control", y concluye que una aplicación web busca "gestionar de mejor manera los procesos administrativos y brindar un servicio calificado, asegurar la información y fidelizar sus clientes".
- **Loor García, Y. Y. (2021).**
 - Desarrollo de aplicación web para la gestión de consultas y agendamiento de citas de mascota de la clínica veterinaria Burgos
 - Se enfoca en la automatización y rendimiento eficiente de las agendas médicas para evitar choques de horarios y mejorar la atención de las mascotas
- **Córdova Urriola, J. W. (2018).**
 - Sistema web para el proceso de control de inventario en la empresa veterinaria Mi Mascota [Tesis de pregrado, Universidad César Vallejo]. Repositorio Institucional UCV
 - Esta tesis detalla el uso de metodologías ágiles (como Scrum) para el desarrollo del backend y frontend
- **Semisud. (2017)**
 - Diseño de sistema web de historias clínicas de mascotas para la veterinaria "Mordiscos" [Tesis de Licenciatura en Sistemas de Información]. Repositorio de la Biblioteca Virtual Semisud
 - Resumen (Abstract) y Conclusiones del Estudio de Campo: "El sistema busca que todas las personas puedan tener a su alcance el expediente de la mascota y así obtener información del dueño de la mascota, observar las citas médicas, vacunas, fichas médicas completas, y realizar consultas en caso de que exista el cambio de veterinario"

- **Padilla Espino, M. F. (2021).**

- Sistema web para la gestión de historias clínicas de la veterinaria [Tesis de pregrado, Universidad Autónoma de Ica]. Repositorio Institucional UAI.
 - Capítulo 1: Planteamiento del Problema, Apartado "1.2. Formulación del Problema: "la ficha es entregada al dueño de la mascota, pero es común que estos pierdan algunas o todas las fichas que se les entregan. En tal sentido, el problema debe solucionarse cuanto antes para eliminar las causas que generan estos procesos ineficientes".

- **Chávez Yagual, D. S. (2022).**

- Desarrollo de una aplicación web para el agendamiento de citas y control de procesos de la clínica veterinaria Animal Health [Tesis de Tecnología de la Información, Universidad Estatal Península de Santa Elena]. Repositorio UPSE.
 - Resumen e Introducción (Justificación del Proyecto): En la justificación (primeras páginas), se argumenta que el proyecto "busca mejorar el control de los servicios de atención al cliente y administrar todos los procesos", eliminando los cuellos de botella en la recepción mediante un calendario interactivo.